evaluate this:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import glob
Import the experiment engine

import aief_experiment_engine_v1 as engine
==========
Study Configuration
==========

```
STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12
```

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5

PHYSICAL_LATE_START = 87.5

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
====
REQUIRED OUTPUT SCHEMA

==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
DESIRED_BRANCH_SCALAR_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.

Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())

if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

```
raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:
```

```
text
"""Write text atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)
temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temporary, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(temporary), str(path))
finally:
if temporary.exists():
temporary.unlink()
def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:
text
"""Write JSON atomically with fsync."""
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
```

```
atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:
```

```
np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

================================================================================
```

Configuration and Validation
==========
====

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

```
B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

def compute_horizon_indices(

text
```

future_steps: int,

T_window: float,

dt: float,

) -> dict[str, tuple[int, int]]:

text

"""Compute horizon step indices for a given T_window.

Fixed physical horizons:

- immediate: 0-17.5 time units

- recovery: 17.5-87.5 time units

- late: 87.5-320 time units

EMA-normalized horizons:

- immediate: 0-1 T_window

- recovery: 1-5 T_window

- late: 5 T_window to end

```
Returns dict mapping horizon name to (start_step, end_step) inclusive start, exclusive end.

"""

window_steps = int(round(T_window / dt))



# Fixed physical horizons (in steps)

immediate_end_physical = int(round(PHYSICAL_IMMEDIATE_END / dt))

recovery_end_physical = int(round(PHYSICAL_RECOVERY_END / dt))



# EMA-normalized horizons (in steps)

immediate_end_normalized = window_steps # 0-1 T_window

recovery_end_normalized = 5 * window_steps # 1-5 T_window



# Use the more restrictive (shorter) horizon for each phase

immediate_end = min(immediate_end_physical, immediate_end_normalized)

recovery_end = min(recovery_end_physical, recovery_end_normalized)



# Clamp to future_steps
```

```
immediate_end = min(immediate_end, future_steps)

recovery_end = min(recovery_end, future_steps)



return {

"immediate": (0, immediate_end),

"recovery": (immediate_end, recovery_end),

"late": (recovery_end, future_steps),

}

# ============================================================================
# Simulation Functions
# ============================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],
```

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.

CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.

Returns complete results for all branches at this (seed, rho).

"""

Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)

Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

```
split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)
```

```
rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,

)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {

"seed": int(seed),

"rho": rho,

"config": {
```

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

```
def run_baseline_equivalence(
text
seeds: list[int],
frozen_manifest_path: str,
frozen_npz_path: str,
) -> dict[str, Any]:
text
"""Run baseline equivalence test at rho=1 (T_window = T_*).


MANDATORY: Must reproduce frozen run exactly for ALL seeds.


Verifies:
1. Seed list matches frozen manifest exactly (same seeds, same order)
2. All 40 seeds pass equivalence (if 40 seeds provided)
3. Engine produces all required outputs
"""
import json
```

```
# Load frozen manifest to compare seeds
with open(frozen_manifest_path, 'r') as f:
frozen_manifest = json.load(f)

frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]

# Verify seed list matches frozen manifest EXACTLY
# This is critical: baseline equivalence tests a specific cohort
if len(seeds) != len(frozen_seeds):
return {
"status": "failed",
"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",
"frozen_seeds": frozen_seeds,
"generated_seeds": [int(s) for s in seeds],
"freezeable": False,
}
```

```
for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": frozen_seeds,

"generated_seeds": [int(s) for s in seeds],

"freezeable": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Create config for rho=1
```

```
config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Determine frozen CSV path (same directory as NPZ, with expected naming)

frozen_csv_path = None

npz_dir = os.path.dirname(frozen_npz_path)

if npz_dir:
```

```
# Try to find CSV with expected naming pattern
csv_patterns = [
os.path.join(npz_dir, f"AIEF_1_4_*_scalar_summary.csv"),
os.path.join(npz_dir, f"*-scalar_summary.csv"),
]
for pattern in csv_patterns:
matches = glob.glob(pattern)
if matches:
frozen_csv_path = matches[0]
break


# Find frozen scalar CSV if not provided
if args.frozen_scalar is None:
# Try to find it automatically
if npz_dir:
csv_patterns = [
os.path.join(npz_dir, f"AIEF_1_4_*_scalar_summary.csv"),
```

```
os.path.join(npz_dir, f"*-scalar_summary.csv"),

]

for pattern in csv_patterns:

matches = glob.glob(pattern)

if matches:

args.frozen_scalar = matches[0]

break



if args.frozen_scalar is None:

print("\nFrozen scalar CSV not found!")

print("Provide with --frozen-scalar or ensure it's in the same directory as the NPZ")

sys.exit(1)



if not os.path.exists(args.frozen_scalar):

print(f"\nFrozen scalar CSV not found: {args.frozen_scalar}")

sys.exit(1)
```

```
# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches
try:
results = [engine.run](http://engine.run)_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
frozen_csv_path=args.frozen_scalar,
branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace
)
except RuntimeError as exc:
# Save failure manifest before re-raising
failure_manifest = {
**manifest,
"config_hash": config_hash,
"status": "FAILED",
"failure_reason": "baseline_equivalence_exception",
```

```
"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nFailure manifest saved to {failure_manifest_path}")

raise



# Add seed comparison info

results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results
```

```
================================================================================
Manifest and Output
================================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

# Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),
```

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

```
"branches": list(STUDY_BRANCHES),

}



study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



manifest = {

"study_name": STUDY_NAME,

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"model": model_params,

"protocol": protocol_params,

"study": study_params,
```

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

```
return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():
```

```
payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,
```

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result)

row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

```
for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)
```

```
parser.add_argument(
"--master-seed",
type=int,
default=DEFAULT_MASTER_SEED,
help="fixed integer master seed (default: 20260714). For production, must match frozen data.",
)
parser.add_argument(
"--n-seeds",
type=int,
default=DEFAULT_N_SEEDS,
help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root",
default=DEFAULT_RUN_ROOT,
help=f"output root directory (default: {DEFAULT_RUN_ROOT})",
)
```

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()

Generate seeds

```
seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Create run directory

run_dir = Path([args.run](http://args.run)_root) / config_hash
```

```
run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()



# Dry run check

if args.dry_run:
```

```
print("Dry run: configuration validated successfully.")

print(f"Manifest saved to {manifest_path}")

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Check if frozen data paths are provided

if args.frozen_manifest is None or args.frozen_npz is None:

print("\nFrozen data paths not provided!")

print("To generate frozen data, run:")
```

```
print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide the paths with --frozen-manifest and --frozen-npz")

sys.exit(1)



# Check if frozen data paths exist

if not os.path.exists(args.frozen_manifest):

print(f"\nFrozen manifest not found: {args.frozen_manifest}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



if not os.path.exists(args.frozen_npz):

print(f"\nFrozen NPZ not found: {args.frozen_npz}")

print("To generate frozen data, run:")
```

```
print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



# Run baseline equivalence - MANDATORY for all seeds

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")



# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():
```

```
print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,
```

```
"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)


# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate
```

```
baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Update manifest with baseline equivalence certificate

manifest["baseline_equivalence"] = {

"status": "passed",

"n_seeds_tested": baseline_results["n_seeds"],

"n_seeds_passed": baseline_results["n_seeds"],

"engine_source_sha256": baseline_results.get("engine_source_sha256"),

"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),

"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),

"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),

"certificate_path": str(baseline_cert_path),

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate
```

```
atomic_write_json(manifest_path, manifest)



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {}



# Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0



started = time.time()
```

```
total_combinations = len(seeds) * len(RHO_VALUES)



for seed_index, seed in enumerate(seeds):

for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


# Try to load existing checkpoint

if checkpoint_path.exists():

try:

# For now, just skip - in production we'd validate and load

# But to keep it simple, we'll recompute

pass

except Exception as e:
```

```
print(f" Warning: Invalid checkpoint {checkpoint_path}: {e}")


try:

# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,
```

```
rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",

"elapsed_seconds": time.time() - condition_start,
```

```
})

successful_conditions += 1


except Exception as e:

condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": str(e),

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"
```

```
if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

if condition_status:

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")



# If any conditions failed, mark as non-freezeable

if failed_conditions &gt; 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)
```

```
print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nWARNING: Expected {expected_rows} scalar rows, got {actual_rows}")

manifest["freezeable"] = False

atomic_write_json(manifest_path, manifest)



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering
```

```
all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()
```

```
# Write NPZ
if all_matrix_payload:
save_npz_atomic(npz_path, all_matrix_payload)


# Final manifest update with completion status
manifest["completion"] = {
"status": "completed",
"n_conditions_requested": total_combinations,
"n_conditions_successful": successful_conditions,
"n_conditions_failed": failed_conditions,
"n_scalar_rows": len(all_scalar_rows),
"n_matrix_entries": len(all_matrix_payload),
"elapsed_seconds": time.time() - started,
"freezeable": failed_conditions == 0 and successful_conditions == total_combinations,
}


# Add output hashes
```

```
if csv_path.exists():

manifest["output_hashes"] = {

"scalar_csv": file_sha256(csv_path),

}

if npz_path.exists():

manifest["output_hashes"]["branch_npz"] = file_sha256(npz_path)



# Re-save final manifest

atomic_write_json(manifest_path, manifest)



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

print(f"Freezeable: {manifest.get('completion', {}).get('freezeable', False)}")
```

```
# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Validate we have primary outcomes before summarizing

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or (isinstance(row[key], float) and np.isnan(row[key])):

missing_primary.append(f"{row['seed']}_rho_{row['rho']}_branch_{row['branch']}_{key}")



if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are NaN or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")
```

```
# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}

for key in available_keys:
```

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

stats[key] = f"{np.mean(values):.4f}±{np.std(values):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

This driver now has the correct study design, full frozen RNG namespace, fixed-physical primary outcomes, dual-horizon schema, full 40-seed baseline requirement, per-condition status tracking, and checkpoint files.

However, it contains three immediate runtime errors in the baseline/checkpoint path. It will not complete as written.
1. run_baseline_equivalence references variables outside its scope

Inside the helper, it uses:

Python

args.frozen_scalar
manifest
config_hash
run_dir

None of those are parameters or globals guaranteed to exist in that function.

For example:

Python

if args.frozen_scalar is None:

will raise:

text

NameError: name 'args' is not defined

Likewise, the exception block uses:

Python

failure_manifest = {
**manifest,
"config_hash": config_hash,
}
failure_manifest_path = (
run_dir / "FAILED_manifest.json"
)

All three names are undefined in the helper.

The helper should only perform equivalence and return a result. Main should handle manifests and output paths.

Use:

Python

```
def run_baseline_equivalence(
seeds: list[int],
frozen_manifest_path: str,
frozen_npz_path: str,
frozen_scalar_path: str,
) -> dict[str, Any]:
with open(
frozen_manifest_path,
"r",
encoding="utf-8",
) as handle:
frozen_manifest = json.load(handle)


frozen_seeds = frozen_manifest[
"randomization"
]["paired_seeds"]


# Exact seed validation.


config = create_model_config(1.0)


baseline_protocol = (
engine.ProtocolConfig(
T_sim=FROZEN_T_SIM,
burn_end_time=160.0,
split_time=480.0,
future_duration=320.0,
diagnostic_stride=50,
branches=FROZEN_BRANCH_ORDER,
)
)


return engine.run_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=(
frozen_manifest_path
),
frozen_npz_path=frozen_npz_path,
```

frozen_csv_path=frozen_scalar_path,
branch_order=FROZEN_BRANCH_ORDER,
)

Then main catches errors and writes the failure manifest.
2. Main does not pass the frozen scalar path

The CLI now provides:

text

--frozen-scalar

but main calls:

Python

run_baseline_equivalence(
seeds=seeds,
frozen_manifest_path=...,
frozen_npz_path=...,
)

without it.

Add the scalar path and make it mandatory:

Python

for path in (
args.frozen_manifest,
args.frozen_npz,
args.frozen_scalar,
):
if path is None:
raise RuntimeError(
"all frozen baseline artifacts "
"are required"

)
if not os.path.isfile(path):
raise FileNotFoundError(path)

Then:

Python

baseline_results = (
run_baseline_equivalence(
seeds=seeds,
frozen_manifest_path=(
args.frozen_manifest
),
frozen_npz_path=args.frozen_npz,
frozen_scalar_path=(
args.frozen_scalar
),
)
)

Remove the duplicate auto-discovery logic from inside the helper.
3. tempfile is not imported

save_npz_atomic calls:

Python

tempfile.mkstemp(...)

but the driver does not import tempfile.

Add:

Python

import tempfile

Without this, the first condition checkpoint or final NPZ write fails.
4. save_csv_atomic is called but not defined

The driver writes condition status with:

Python

```
save_csv_atomic(
status_path,
status_df,
)
```

but no such function is defined or imported.

This will fail after all condition simulations.

Either import/use an engine helper or define:

Python

```
def save_csv_atomic(
path: Path,
frame: pd.DataFrame,
) -> None:
path.parent.mkdir(
parents=True,
exist_ok=True,
)

fd, temporary = tempfile.mkstemp(
dir=path.parent,
prefix=f".{path.name}.",
suffix=".tmp",
)

try:
with os.fdopen(
fd,
```

"w",
encoding="utf-8",
newline="",
) as handle:
frame.to_csv(
handle,
index=False,
)
handle.flush()
os.fsync(handle.fileno())

os.replace(temporary, path)
except Exception:
try:
os.unlink(temporary)
except FileNotFoundError:
pass
raise

The driver also needs:

Python

import pandas as pd

because it creates a DataFrame for condition status. Pandas is currently not imported.
5. Baseline failure certificate should be handled in main

The engine may raise on equivalence failure. Main should preserve the result if returned, or at least save the exception.

Recommended:

Python

try:
baseline_results = (
run_baseline_equivalence(...)

)
except Exception as exc:
failure_manifest = {
**manifest,
"config_hash": config_hash,
"status": "FAILED",
"failure_reason":
"baseline_equivalence_exception",
"exception_type":
type(exc).__name__,
"exception_message": str(exc),
"freezeable": False,
}

atomic_write_json(
run_dir / "FAILED_manifest.json",
failure_manifest,
)
raise

Preferably, change the engine to return a failed certificate rather than raising before the detailed per-seed results can be saved.
6. The checkpoint system does not resume

For an existing checkpoint:

Python

if checkpoint_path.exists():
pass

and then the condition is recomputed.

This is not checkpoint/resume; it is only per-condition backup.

That is still useful, but describe it accurately:

Per-condition checkpoint snapshots; automatic resume not yet implemented.

If resume is intended, load the checkpoint, validate:

text

seed
rho
T_window
alpha
driver config hash
engine source hash
output schema

then reconstruct scalar rows and matrix payload without rerunning.
7. Final run status is always "completed"

The final manifest sets:

Python

"status": "completed"

even when conditions failed.

Use:

Python

complete = (
failed_conditions == 0
and successful_conditions
== total_combinations
and len(all_scalar_rows)
== expected_rows
)

manifest["completion"] = {

"status": (
"completed"
if complete
else "incomplete"
),
...
"freezeable": (
complete
and baseline_passed
),
}

Also set a single top-level:

Python

manifest["freezeable"] = ...

The analysis driver should not need to inspect conflicting nested flags.
8. Missing primary outcomes should be fatal

The driver currently prints a warning if a primary field is missing or NaN:

text

This indicates engine is not producing required outputs.

But it still finalizes the run.

Since those outcomes are the predeclared estimands, missing primary values must make the run incomplete and nonfreezeable.

Perform this check before writing the final completion status.
9. Dual-horizon primary outcomes are now correctly selected

This is a strong correction:

Python

```
PRIMARY_OUTCOMES = (
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
)
```

The normalized EMA fields are correctly retained as secondary outcomes.

This now matches the scientific estimand.
10. Full RNG namespace is correctly used

simulate_seed_rho now creates RNGs with:

Python

```
FROZEN_BRANCH_ORDER
```

and runs only:

Python

```
STUDY_BRANCHES
```

This preserves the original future_shuffle RNG identity. That part is correct.
11. Baseline equivalence should validate all seven branches

The new baseline protocol uses:

Python

```
branches=FROZEN_BRANCH_ORDER
```

which is correct for engine certification.

The study itself then uses the four-branch subset.
12. Manifest must include the engine and frozen-source hashes

The study manifest currently hashes only the driver source.

Add:

text

engine source path/hash
frozen production source path/hash
frozen manifest path/hash
frozen NPZ path/hash
frozen scalar CSV path/hash
baseline certificate path/hash

The initial manifest can hold the specification; the final manifest should hold actual run and artifact hashes.
13. Output accumulation remains an all-in-memory risk

The new condition checkpoints reduce total-loss risk, but all_matrix_payload still holds every condition in memory.

For a successful large run, consolidate checkpoint files after all 200 conditions pass rather than retaining the complete payload throughout.

This is not an immediate correctness blocker, but it defeats much of the checkpointing benefit.
14. Run directory reuse needs protection

The config hash points repeated identical runs to the same directory. Existing checkpoints and outputs can be overwritten or mixed.

Default behavior should fail if the run directory already contains a completion manifest, unless an explicit:

text

--resume
--overwrite

mode is selected.
15. The obsolete hybrid-horizon function remains

compute_horizon_indices still takes the minimum of physical and normalized boundaries. It is unused and contradicts the now-correct dual-horizon engine.

Delete it to prevent accidental future use.
16. Smoke-test support is still missing

Production baseline requires all 40 seeds, which is correct for freezeability, but a one-seed engineering smoke path is still useful.

Add a separate nonfreezeable command such as:

text

--smoke-seed <seed>

It should:

compare one frozen seed;
run one nonbaseline rho;
validate scalar/matrix schemas;
write to a separate smoke directory;
set freezeable=false.

Overall assessment
Scientific design: 9/10
Full regeneration and pairing: 9/10
RNG namespace: 9/10
Baseline scope: 9/10
Dual-horizon outcome schema: 9/10
Current execution correctness: 4/10 because of undefined names/imports
Large-run robustness: 6/10
Overall current driver: 7/10 design, 5.5/10 executable

Minimum fixes before execution

remove out-of-scope args/manifest/config_hash/run_dir references from the baseline helper;
pass --frozen-scalar;
import tempfile;
define/import save_csv_atomic;
set completion/freezeability consistently;
make missing primary outcomes fatal;
delete the obsolete hybrid horizon helper.

After those changes, run compile/help only. Then perform a one-seed smoke path before committing to the full 200-condition grid.

Evaluate this:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd

```
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Primary outcomes

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late",
```

"Retention_contrast_late",

"P_mean_late",

)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

df = pd.read_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)

```
# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():
```

```
print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")



return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text
```

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

```
def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

f.write(text)

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)
```

```
finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



# Convert NaN to None for JSON serialization

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]
```

```
elif isinstance(obj, float) and np.isnan(obj):

return None

elif isinstance(obj, (np.integer, np.floating)):

return obj.item()

elif isinstance(obj, np.ndarray):

return obj.tolist()

else:

return obj



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)
```

```
# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)

if not path.exists():

return "FILE_NOT_FOUND"
```

with open(path, "rb") as f:

for block in iter(lambda: f.read(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

```
================================================================================
====
Metric Functions
================================================================================
====
```

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

```
return float('nan')



return float([np.dot](http://np.dot)(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] &gt; threshold

y = B2[edges] &gt; threshold



union = np.count_nonzero(x | y)

if union == 0:

return float('nan')



return float(np.count_nonzero(x &amp;amp; y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text
```

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

==========
Stage 1: Validation
==========

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results

Check baseline equivalence

```
baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

if not manifest.get("freezeable", True):

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)
```

```
except Exception as e:
results["status"] = "failed"
results["errors"].append(f"Cannot load CSV: {e}")
return results


# Validate row count
expected_rows = EXPECTED_N_ROWS
actual_rows = len(df)
if actual_rows != expected_rows:
results["status"] = "failed"
results["errors"].append(
f"Row count mismatch: expected {expected_rows}, got {actual_rows}"
)


# Validate unique seeds
unique_seeds = df["seed"].unique()
if len(unique_seeds) != EXPECTED_N_SEEDS:
```

```
results["status"] = "failed"
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)

# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)

# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
```

```
if unique_branches != expected_branches:

results["status"] = "failed"

results["errors"].append(

f"Branches mismatch: expected {expected_branches}, got {unique_branches}"

)



# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:
```

```
results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and numeric

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"
```

```
results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")
else:
# Check that values are numeric
non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))
if non_numeric.any():
results["status"] = "failed"
results["errors"].append(
f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"
)


# Record validation info
results["validation"] = {
"n_rows": actual_rows,
"n_seeds": len(unique_seeds),
"n_rho": len(unique_rho),
"n_branches": len(unique_branches),
"n_duplicates": int(duplicate_rows.sum()),
```

```
"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", True),

}



if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")



return results
```

```
================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[
```

```
df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values
```

```
merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)



# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]
```

```
# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")


rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)
```

```
if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)



print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")



return contrasts_df

# ============================================================================
# Stage 3: Branch Difference-in-Differences
# ============================================================================

def compute_branch_did(

text
```

```
df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} -
Y_{s,1,continue}]



For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.



Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]



for rho in sorted([float(r) for r in df["rho"].unique()]):
```

```
# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]
```

```
if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c
```

```
did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count
```

```
expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")



return did_df

================================================================================
Stage 4: Cross-T Topology and Recovery
================================================================================

def build_continue_b_final_key(seed: str, rho: float) -> str:

text
```

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))

Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)

topology_rows = []

missing_keys = []

```
for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline


# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)


if rho_key not in npz_data:

missing_keys.append(rho_key)

continue

if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue


B_rho = npz_data[rho_key]

B_baseline = npz_data[baseline_key]
```

```
# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
"cross_T_Jaccard": jaccard,
"cross_T_normalized_distance": norm_dist,
})

# Validate we have all expected comparisons
expected_count = EXPECTED_N_TOPOLOGY
if len(topology_rows) != expected_count:
raise RuntimeError(
f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "
```

```
f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")



return topology_df

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics.
```

This is REQUIRED for production analysis. Currently raises NotImplementedError.

For each seed/rho/branch, compute:

- First crossing of continue branch threshold

- Sustained recovery duration

- Held recovery time

- Censoring indicator

- Restricted recovery time

- AUC deficit

- t_rec / T_window

Uses trajectories from NPZ to compute recovery relative to continue branch.

"""

raise NotImplementedError(

"Recovery scaling is required before production analysis. "

"The simulation engine must save both fixed-physical and normalized-EMA "

```
"horizon systems, and this function must be implemented to compute "

"t_rec/T_window for both systems."

)

================================================================================
Stage 5: Seed-Level Summaries
================================================================================

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.
```

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

Use named aggregation to avoid MultiIndex columns

```
regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),
```

```
)

# Add bootstrap CI
for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):
values = group["difference_in_differences"].dropna().values
if len(values) &gt; 0:
indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))
bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))
ci_upper = float(np.percentile(bootstrap_means, 97.5))

idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index
did_summary.loc[idx, "ci_lower"] = ci_lower
did_summary.loc[idx, "ci_upper"] = ci_upper

summaries["branch_did"] = did_summary
print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")
```

```
# Topology summaries
if not topology.empty:
# Flatten the aggregation to avoid MultiIndex columns
topology_summary = topology.groupby("rho", as_index=False).agg(
cosine_mean=("cross_T_cosine", "mean"),
cosine_std=("cross_T_cosine", "std"),
cosine_n=("cross_T_cosine", "count"),
jaccard_mean=("cross_T_Jaccard", "mean"),
jaccard_std=("cross_T_Jaccard", "std"),
jaccard_n=("cross_T_Jaccard", "count"),
distance_mean=("cross_T_normalized_distance", "mean"),
distance_std=("cross_T_normalized_distance", "std"),
distance_n=("cross_T_normalized_distance", "count"),
)
summaries["topology"] = topology_summary
print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")
```

return summaries

```
================================================================================
Report Generation
================================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

) -> str:

text

"""Generate human-readable text report."""

lines = []
```

```
lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})

lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")
```

```
lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])
```

```
ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Branch DID

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]
```

```
for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")
```

```
lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

else:

lines.append(" No cross-T topology data")

lines.append("")




lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)
```

```
return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)
```

```
manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,


"source": {

"path": str(source_path),

"sha256": source_hash,

},


"command_line": command_line,
```

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

},

}

return manifest

==
Main Execution
==

```
def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"run_dir",

type=str,

help="run directory containing study outputs",

)

parser.add_argument(

"--output",

type=str,

default=None,

help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",

)

parser.add_argument(
```

```
"--n-bootstrap",

type=int,

default=1000,

help="number of bootstrap iterations (default: 1000)",

)

parser.add_argument(

"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

return parser.parse_args()

def main() -> None:

text

args = parse_arguments()



# Parse run directory
```

```
run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]
```

```
manifest_path = manifest_files[0]



# Set up output directory

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)

output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")
```

```
print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")
```

```
df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts
```

```
print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Stage 3: Branch DID

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED - will raise NotImplementedError)

print("\nStage 4b: Computing recovery scaling metrics...")

try:

recovery = compute_recovery_scaling(df, npz_path, b_c)

except NotImplementedError as e:

print(f" ⚠️ {e}")
```

```
print(" Recovery scaling must be implemented before production analysis.")

print(" The simulation engine must save both horizon systems.")

# For now, we'll continue but this should be fixed

recovery = pd.DataFrame(columns=[

"seed", "rho", "branch", "horizon_system",

"first_crossing", "sustained_recovery", "held_duration",

"censored", "restricted_time", "auc_deficit", "t_rec_over_Twindow"

])



# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)
```

```
regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Branch DID

branch_did_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_path, branch_did)
```

```
output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling (if implemented)

if not recovery.empty:

recovery_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_path, recovery)

output_files["recovery"] = recovery_path

print(f" Saved: {recovery_[path.name](http://path.name)}")
```

```
# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path
```

```
print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries

)

report_path = output_dir / "twindow_sensitivity_report.txt"

save_text_atomic(report_path, report)

output_files["report"] = report_path

print(f" Saved: {report_[path.name](http://path.name)}")



# Analysis manifest

analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files,

source_path=Path(__file__),

bootstrap_seed=args.bootstrap_seed,
```

```
n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)
```

if name == "main":

text

main()

This version fixes the regime-contrast construction, DID grouping, exact topology keys, deterministic bootstrap, output-path handling, and provenance manifest. However, it currently fails at import time because PRIMARY_OUTCOMES is referenced before definition. It also still omits the required recovery analysis and uses the wrong horizon fields as primary outcomes.
1. Immediate import-time NameError

At module scope:

Python

EXPECTED_N_REGIME_CONTRASTS = (
EXPECTED_N_SEEDS
* (len(EXPECTED_RHO_VALUES) - 1)
* len(PRIMARY_OUTCOMES)
)

appears before:

Python

PRIMARY_OUTCOMES = (...)

Python evaluates module assignments in order. Importing this script raises:

text

NameError: name 'PRIMARY_OUTCOMES' is not defined

Move PRIMARY_OUTCOMES above all expected-count calculations.
2. Primary outcomes use the wrong horizon system

The sensitivity study declares fixed-physical horizons as primary, but the analysis still uses legacy fields:

Python

```
PRIMARY_OUTCOMES = (
"Weighted_similarity_late",
"Retention_contrast_late",
"P_mean_late",
)
```

Those legacy fields are normalized-EMA outcomes.

Use:

Python

```
PRIMARY_OUTCOMES = (
"Weighted_similarity_late_fixed_physical",
"Retention_contrast_late_fixed_physical",
"P_mean_late_fixed_physical",
)
```

Define secondary outcomes separately:

Python

```
SECONDARY_NORMALIZED_OUTCOMES = (
"Weighted_similarity_late_normalized_EMA",
"Retention_contrast_late_normalized_EMA",
"P_mean_late_normalized_EMA",
)
```

The regime and DID analyses should produce separate tables or an explicit:

text

horizon_system

column.

Without this correction, the analysis does not implement its primary estimand.
3. Recovery scaling is still not implemented

The function explicitly raises:

Python

NotImplementedError

but main catches it and continues, eventually printing:

text

ANALYSIS COMPLETE

This contradicts the script header:

Correct implementation of the prespecified estimands.

Recovery scaling is one of the four declared estimands. Production analysis must stop:

Python

recovery = compute_recovery_scaling(...)

without catching NotImplementedError.

Until implemented, the script should be marked incomplete and nonfreezeable.
4. Recovery implementation requirements

For every seed, rho, noncontinue branch, and horizon system:

load:

text

sample_times
density_trajectory
similarity_trajectory

use the same seed/rho continue branch as reference;
calculate condition-specific targets;
calculate:

text

first crossing
sustained recovery
held for one T_window
recovery indicator
restricted time
AUC deficit
t_rec / T_window

retain every seed in censoring denominators;
output separate rows for:

text

fixed_physical
normalized_EMA

Expected recovery rows should be validated, not optional.
5. Validation checks numeric type but not finite values

After pandas conversion, NaN is still numeric. This check passes:

Python

isinstance(np.nan, float)

Primary outcomes should be finite for all 800 rows:

Python

```
nonfinite = ~np.isfinite(
pd.to_numeric(
df[outcome],
errors="coerce",
)
)

if nonfinite.any():
fail
```

If a primary outcome may legitimately be undefined, define that policy explicitly. Do not silently include NaN contrasts.
6. Freezeability path may be wrong

Validation checks:

Python

```
manifest.get("freezeable", True)
```

The latest simulation driver may store the final flag under:

Python

```
manifest["completion"]["freezeable"]
```

rather than at top level.

Use:

Python

```
completion = manifest.get(
"completion",
{},
)

freezeable = manifest.get(
"freezeable",
completion.get(
"freezeable",
False,
),
)
```

Default must be false, not true.

Also require:

text

```
completion.status == completed
n_conditions_successful == 200
n_conditions_failed == 0
n_scalar_rows == 800
```

7. Condition-status validation is absent

The simulation driver writes:

text

```
twindow_condition_status.csv
```

but analysis does not read or validate it.

Require exactly:
$40 \times 5 = 200$
$40 \times 5 = 200$

successful condition rows and no failures.

Hash it in the analysis manifest.
8. Regime contrasts are now structurally correct

The direct merge avoids the previous MultiIndex error and correctly computes:
$Y_{s,\rho,c-}$
$Y_{s,\rho,c-}$

Y_{s,1,c}.
]

Expected count for three outcomes is:
$40 \times 4 \times 3 = 480$.
$40 \times 4 \times 3 = 480$.

The count check is good once PRIMARY_OUTCOMES is defined in time and changed to fixed-physical fields.

Add explicit finite-value checks before subtraction.
9. DID implementation is now structurally correct

It:

skips baseline rho;
preserves branch identity;
computes the correct within-rho branch effect;
subtracts the baseline branch effect;
validates the expected count.

For three branches and three outcomes:
$40 \times 4 \times 3 \times 3 = 1440$.
$40 \times 4 \times 3 \times 3 = 1440$.

This part is substantially improved.
10. Cross-T topology implementation is now mostly correct

It compares actual final BB matrices for the same seed against rho =1=1, using exact keys and requiring 160 rows.

That directly implements:

cosine similarity;
threshold Jaccard;
normalized edge-space distance.

Improvements still needed:

validate matrix shapes;
validate symmetry and zero diagonal;
load BcBc from manifest, as already done;
fail on duplicate/ambiguous keys;
add bootstrap intervals to topology summaries.

11. Topology summaries lack bootstrap intervals

Regime and DID summaries have deterministic bootstrap intervals. Topology summaries report only mean and standard deviation.

Bootstrap the 40 seed-level topology metrics by rho as well.
12. Bootstrap validation

Add:

Python

if args.n_bootstrap <= 0:
raise ValueError(...)

The seeded generator is now correct.

Using one generator sequentially across groups is deterministic. Record that group iteration order is sorted or derive a deterministic group-specific child seed.
13. JSON sanitization does not handle infinity

The sanitizer replaces only NaN:

Python

np.isnan(obj)

It does not replace:

text

+inf
-inf

With allow_nan=False, infinity also fails JSON serialization.

Use:

Python

if isinstance(obj, float):
return obj if math.isfinite(obj) else None

and equivalent handling after converting NumPy scalars.
14. Output manifest omits recovery because it is absent

Once recovery is implemented, require and hash:

text

twindow_recovery_summary.csv

The analysis manifest should also include:

condition-status CSV hash;
baseline equivalence certificate hash;
simulation engine source hash;
simulation driver source hash;
horizon definitions;
interpretation constraints.

15. CSV atomic writer does not fsync the file

It writes with pandas directly to a temporary path, then replaces it and fsyncs only the directory.

For consistency, open a file handle, flush/fsync it, then replace.

Not a scientific blocker, but easy to correct.
16. Output-directory stale-file protection

A rerun can leave old outputs if the new run fails before rewriting every file.

Require an empty analysis directory or add explicit --overwrite.
17. Report arguments are partly unused

generate_text_report accepts:

Python

regime_contrasts
branch_did
topology

but uses only summary frames and validation. Remove unused arguments or include raw count checks.
18. Analysis manifest is generated before it is written

It hashes all prior outputs but cannot hash itself, which is fine. Make that explicit.

The source hash and command are now correctly included.
Current readiness

Area Rating
Prespecified regime estimand 8/10 after field rename
Branch DID 9/10
Cross-T topology 8/10
Validation framework 7/10
Bootstrap reproducibility 8/10
Recovery scaling 0/10
Runtime correctness 3/10 due import NameError
Output provenance 7/10
Overall: 6/10 design, 4.5/10 current executable
Minimum next revision

move PRIMARY_OUTCOMES before expected-count constants;
switch primary fields to fixed-physical names;
implement recovery scaling and remove the catch-and-continue behavior;
validate final simulation completion/status table;
add topology bootstrap intervals;
handle nonfinite JSON values.

After that, run py_compile and a synthetic/unit test. The analysis is not ready to process the full grid while Stage 4b remains a placeholder.

#!/usr/bin/env python3

"""AIEF Experiment Engine v1 - Parameterized simulation kernel.

This is a reusable, validated simulation engine that reproduces the frozen

AIEF-1.4 production run exactly and supports parameterized studies.

FROZEN PRODUCTION VALUES (from AIEF_1_4_memory_ablation_updated_REAL.py):

N = 12

DT = 0.01

TAU = 0.5

OMEGA = [1.00, 1.02, 1.50, 2.00, 2.01, 1.01, 1.03, 1.51, 2.02, 2.00, 1.00, 1.04]

A = 1.0

Q = 1.5

DELTA = 0.05

ETA = 0.04

GAMMA = 0.015

MU = 0.03

T_WINDOW = 25.0 / mean(OMEGA) = 17.5

B_C = 0.12

B_INIT_SCALE = 0.0005

PROTOCOL (from frozen run):

T_SIM = 800.0

N_STEPS = int(round(T_SIM / DT)) = 80000

BURN_END = int(0.20 * N_STEPS) = 16000 steps = 160.0 time units

HIST_END = int(0.60 * N_STEPS) = 48000 steps = 480.0 time units

FUTURE_STEPS = N_STEPS - HIST_END = 32000 steps = 320.0 time units

DELAY_STEPS = int(round(TAU / DT)) = 50

WINDOW_STEPS = int(round(T_WINDOW / DT)) = 1750

RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS) = 8750

Architecture:

ModelConfig: frozen dataclass for all model parameters

ProtocolConfig: frozen dataclass for protocol parameters

DynamicState: state container (immutable where possible)

Core functions: initialize, advance, simulate, interventions

Validation: baseline equivalence test

NOTE: This is a candidate reference implementation. After passing 40/40 baseline

text

equivalence tests, it will be frozen as the reference implementation.

"""

from future import annotations

import copy

import hashlib

import json

import math

import os

```
import tempfile

import time

from dataclasses import dataclass, field

from pathlib import Path

from typing import Any

import numpy as np
================================================================================
Frozen Production Constants (DO NOT CHANGE)
================================================================================

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5
```

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005

FROZEN_ZERO_TOL = 1e-12

FROZEN_PROJECTION_NORM_TOL = 1e-14

FROZEN_GAP_TOL = 1e-9
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA)) # 1.428333...

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.502917152858814

FROZEN_T_WINDOW = FROZEN_T_STAR # 17.502917152858814 (exact, NOT rounded)

FROZEN_ALPHA = math.exp(-FROZEN_DT / FROZEN_T_WINDOW)

FROZEN_CAPACITY = (FROZEN_A - FROZEN_DELTA) / FROZEN_Q
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_WINDOW / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Fixed physical horizon boundaries (in time units, NOT T_window dependent)
These are the PRIMARY estimand horizons

FIXED_PHYSICAL_IMMEDIATE_END = 17.5

FIXED_PHYSICAL_RECOVERY_END = 87.5

FIXED_PHYSICAL_FUTURE_END = 320.0
Derived fixed physical horizon steps

FIXED_PHYSICAL_IMMEDIATE_STEPS = int(round(FIXED_PHYSICAL_IMMEDIATE_END /
FROZEN_DT)) # 1750

FIXED_PHYSICAL_RECOVERY_STEPS = int(round(FIXED_PHYSICAL_RECOVERY_END /
FROZEN_DT)) # 8750
==========
Model Configuration
==========

@dataclass(frozen=True)

class ModelConfig:

text

"""Immutable model configuration.

All model-changing quantities must come from here. No hidden module globals.

"""

Core parameters

N: int

dt: float

tau: float

omega: tuple[float, ...] # Immutable tuple

Coupling parameters

a: float

q: float

delta: float

Dynamics parameters

eta: float

mu: float

gamma: float

EMA/threshold parameters

T_window: float

B_c: float

B_init_scale: float

scale_epsilon: float = 1e-12

Derived properties

@property

def mean_omega(self) -> float:

"""Mean of omega frequencies."""

return float(np.mean(self.omega))

@property

```
def T_star(self) -&gt; float:

"""T_* = 25 / mean(omega)"""

return 25.0 / self.mean_omega



@property

def alpha(self) -&gt; float:

"""EMA smoothing factor: exp(-dt / T_window)"""

return math.exp(-self.dt / self.T_window)



@property

def capacity(self) -&gt; float:

"""Projection capacity: (a - delta) / q"""

return (self.a - [self.delta](http://self.delta)) / self.q



@property

def delay_steps(self) -&gt; int:

"""Delay steps: round(tau / dt)"""
```

```
return int(round(self.tau / self.dt))



def validate(self) -&gt; None:

"""Validate model configuration."""

if self.N &lt;= 0:

raise ValueError("N must be positive")

if self.dt &lt;= 0:

raise ValueError("dt must be positive")

if self.tau &lt;= 0:

raise ValueError("tau must be positive")

if len([self.omega](http://self.omega)) != self.N:

raise ValueError(f"omega must have {self.N} elements")

if self.a &lt;= 0 or self.q &lt;= 0 or [self.delta](http://self.delta) &lt; 0:

raise ValueError("a, q must be positive; delta must be nonnegative")

if self.eta &lt; 0 or [self.mu](http://self.mu) &lt; 0 or self.gamma &lt; 0:

raise ValueError("eta, mu, gamma must be nonnegative")

if self.T_window &lt;= 0:
```

raise ValueError("T_window must be positive")

if self.B_c < 0:

raise ValueError("B_c must be nonnegative")

if self.B_init_scale < 0:

raise ValueError("B_init_scale must be nonnegative")

if self.scale_epsilon <= 0:

raise ValueError("scale_epsilon must be positive")

Validate derived values

if not (0 < self.alpha <= 1):

raise ValueError(f"alpha={self.alpha} must be in (0, 1]")

if self.capacity < 0:

raise ValueError(f"capacity={self.capacity} must be nonnegative")

@dataclass(frozen=True)

class ProtocolConfig:

text

"""Protocol configuration for experiment execution."""

Simulation time parameters (in time units, not steps)

T_sim: float

burn_end_time: float

split_time: float

future_duration: float

Diagnostics

diagnostic_stride: int

Branches to simulate

branches: tuple[str, ...]

def n_steps(self, model: ModelConfig) -> int:

"""Total steps."""

return int(round(self.T_sim / model.dt))

def burn_end(self, model: ModelConfig) -> int:

"""Burn-in end step."""

return int(round(self.burn_end_time / model.dt))

def hist_end(self, model: ModelConfig) -> int:

"""History end step (split point)."""

return int(round(self.split_time / model.dt))

def future_steps(self, model: ModelConfig) -> int:

"""Future simulation steps."""

return int(round(self.future_duration / model.dt))

def validate(self, model: ModelConfig) -> None:

"""Validate protocol against model."""

if self.T_sim <= 0:

raise ValueError("T_sim must be positive")

if not (0 < self.burn_end_time < self.split_time < self.T_sim):

raise ValueError("Invalid time parameters: must have 0 < burn_end < split < T_sim")

if self.future_duration <= 0:

raise ValueError("future_duration must be positive")

if self.diagnostic_stride <= 0:

raise ValueError("diagnostic_stride must be positive")

Validate that all time parameters are compatible with dt

if not np.isclose(self.T_sim / model.dt, round(self.T_sim / model.dt), rtol=1e-10):

raise ValueError(f"T_sim ({self.T_sim}) must be divisible by dt ({model.dt})")

if not np.isclose(self.burn_end_time / model.dt, round(self.burn_end_time / model.dt), rtol=1e-10):

raise ValueError(f"burn_end_time ({self.burn_end_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.split_time / model.dt, round(self.split_time / model.dt), rtol=1e-10):

raise ValueError(f"split_time ({self.split_time}) must be divisible by dt ({model.dt})")

if not np.isclose(self.future_duration / model.dt, round(self.future_duration / model.dt), rtol=1e-10):

raise ValueError(f"future_duration ({self.future_duration}) must be divisible by dt ({model.dt})")

Default frozen production configuration
NOTE: T_window must be the EXACT value 25.0/mean(OMEGA), not rounded

DEFAULT_MODEL_CONFIG = ModelConfig(

text

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=FROZEN_T_WINDOW, # 17.502917152858814 (exact)

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=FROZEN_ZERO_TOL,

)
Default protocol matching frozen run

DEFAULT_PROTOCOL_CONFIG = ProtocolConfig(

text

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=("continue", "reset_ema", "reset_B", "reset_B_ema", "B_carrier_only", "full_reset", "future_shuffle"),

)
==========
Utility Functions
==========

def compute_edges(N: int) -> tuple[np.ndarray, np.ndarray]:

text

"""Get upper triangle indices for edges."""

return np.triu_indices(N, k=1)

def validate_B_matrix(B: np.ndarray, N: int, eps: float = 1e-12) -> None:

text

"""Validate B matrix: symmetric, nonnegative, zero diagonal."""

if B.shape != (N, N):

raise ValueError(f"B shape {B.shape} != ({N}, {N})")

if not np.allclose(B, B.T, atol=eps):

```
raise ValueError("B is not symmetric")

if not np.allclose(np.diag(B), 0, atol=eps):

raise ValueError("B diagonal is not zero")

if np.any(B &lt; -eps):

raise ValueError("B has negative entries")

def file_sha256(path: str | os.PathLike[str]) -> str:

text

"""Compute SHA-256 hash of a file."""

digest = hashlib.sha256()

with open(path, "rb") as handle:

for block in iter(lambda: [handle.read](http://handle.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

def array_sha256(arr: np.ndarray) -> str:

text

"""Compute SHA-256 hash of a numpy array."""

digest = hashlib.sha256()

shape_bytes = np.asarray(arr.shape, dtype=np.int64).tobytes()
```

```
digest.update(shape_bytes)

digest.update(arr.dtype.str.encode())

contiguous = np.ascontiguousarray(arr)

if contiguous.size &gt; 0:

digest.update(contiguous.tobytes())

return digest.hexdigest()

def atomic_write_text(path: Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.", suffix=".tmp")

try:

with os.fdopen(fd, "w", encoding="utf-8") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))
```

```
finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

def atomic_write_json(path: Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def atomic_write_npz(path: Path, **kwargs: Any) -> None:

text

"""Write NPZ atomically with fsync."""

path.parent.mkdir(parents=True, exist_ok=True)

fd, tmp_path = tempfile.mkstemp(dir=str(path.parent), prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz")

try:

with os.fdopen(fd, "wb") as handle:

np.savez(handle, **kwargs)

handle.flush()
```

```
os.fsync(handle.fileno())

os.replace(str(tmp_path), str(path))

finally:

if os.path.exists(tmp_path):

os.remove(tmp_path)

================================================================================
RNG Utilities (matching frozen production exactly)
================================================================================

def make_seed_sequence(seed: int, child_index: int) -> int:

text

"""Generate a child seed from SeedSequence(seed).spawn(...)[child_index].



This matches the frozen production RNG hierarchy exactly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

children = root.spawn(child_index + 1)
```

return int(children[child_index].generate_state(1, dtype=np.uint64)[0])

def make_prefix_rng(seed: int) -> np.random.Generator:

text

"""Create prefix RNG matching frozen production exactly.

Frozen production does:

root = np.random.SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = np.random.default_rng(init_sequence)

NOTE: default_rng(SeedSequence_object) is NOT the same as

default_rng(SeedSequence_object.generate_state(...))

The production uses the SeedSequence object directly.

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

```
return np.random.default_rng(init_sequence)

def make_branch_rngs(seed: int, branch_order: tuple[str, ...]) -> dict[str, np.random.Generator]:

text

"""Create branch RNGs matching frozen production algorithm.



Frozen: *, branch*root = SeedSequence(seed).spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))

branch_rng_seeds = {branch: int(sequence.generate_state(1, dtype=np.uint64)[0])

for branch, sequence in zip(branch_order, branch_sequences)}

"""

from numpy.random import SeedSequence

root = SeedSequence(seed)

*, branch*root = root.spawn(2)

branch_sequences = branch_root.spawn(len(branch_order))



return {

branch: np.random.default_rng(int(sequence.generate_state(1, dtype=np.uint64)[0]))
```

for branch, sequence in zip(branch_order, branch_sequences)

}

==
Projection Accumulator (matching production)
==

@dataclass

class ProjectionTotals:

text

"""Accumulator for projection diagnostics - exact match to production."""

steps: int = 0

violations: int = 0

active: int = 0

norm_sum: float = 0.0

norm_max: float = 0.0

relative_state_sum: float = 0.0

relative_state_max: float = 0.0

relative_update_sum: float = 0.0

relative_update_max: float = 0.0

```
g_min: float = np.inf

g_min_pre: float = np.inf

def update(self, info: dict[str, Any]) -&gt; None:

"""Update accumulator with step info."""

self.steps += 1

self.violations += int(info["pre_violation"])

self.g_min = min(self.g_min, float(info["g_min"]))

self.g_min_pre = min(self.g_min_pre, float(info["g_min_pre"]))

if info["projection_active"]:

self.active += 1

norm = float(info["projection_norm"])

rel_state = float(info["projection_relative_state"])

rel_update = float(info["projection_relative_update"])

self.norm_sum += norm

self.norm_max = max(self.norm_max, norm)

self.relative_state_sum += rel_state

self.relative_state_max = max(self.relative_state_max, rel_state)

self.relative_update_sum += rel_update
```

self.relative_update_max = max(self.relative_update_max, rel_update)

def metrics(self) -> dict[str, float]:

"""Compute final metrics from accumulated values."""

active_denominator = max(self.active, 1)

return {

"G_min_future": float(self.g_min),

"G_min_pre_future": float(self.g_min_pre),

"Pre_gap_violation_fraction": float(self.violations / self.steps),

"Projection_active_fraction": float(self.active / self.steps),

"Projection_active_given_violation": float(self.active / self.violations)

if self.violations > 0

else float('nan'),

"Projection_norm_mean_active": float(

self.norm_sum / active_denominator

),

"Projection_norm_max": float(self.norm_max),

"Projection_relative_state_mean_active": float(

self.relative_state_sum / active_denominator

),

"Projection_relative_state_max": float(self.relative_state_max),

"Projection_relative_update_mean_active": float(

self.relative_update_sum / active_denominator

),

"Projection_relative_update_max": float(self.relative_update_max),

}

==========
Dynamic State
==========

class DynamicState:

text

"""State container for AIEF-1.4 dynamics.

Matches the frozen production implementation exactly.

"""

```
def **init**(

self,

theta: np.ndarray,

B: np.ndarray,

coherence_acc: np.ndarray,

history: list[np.ndarray] | np.ndarray,

):

self.theta = np.asarray(theta, dtype=float).copy()

self.B = np.asarray(B, dtype=float).copy()

self.coherence_acc = np.asarray(coherence_acc, dtype=complex).copy()


# Handle history input - ensure it's a list of arrays

if isinstance(history, np.ndarray):

# If it's a 2D array, convert to list of 1D arrays

self.history = [np.asarray(h, dtype=float).copy() for h in history]

else:

self.history = [np.asarray(h, dtype=float).copy() for h in history]
```

```
def clone(self) -&gt; "DynamicState":

"""Create a deep copy of the state."""

return DynamicState(

theta=self.theta.copy(),

B=self.B.copy(),

coherence_acc=self.coherence_acc.copy(),

history=[h.copy() for h in self.history],

)



@property

def N(self) -&gt; int:

return len(self.theta)



def validate(self, config: ModelConfig, step: int | None = None) -&gt; None:

"""Validate state consistency."""

# Check history length
```

```
expected_history_len = config.delay_steps + 1

if len(self.history) != expected_history_len:

raise RuntimeError(

f"History length {len(self.history)} != expected {expected_history_len} "

f"(delay_steps={config.delay_steps}) at step {step}"

)

# Check theta shape

if self.theta.shape != (self.N,):

raise RuntimeError(f"theta shape {self.theta.shape} != ({self.N},)")

# Check B shape

if self.B.shape != (self.N, self.N):

raise RuntimeError(f"B shape {self.B.shape} != ({self.N}, {self.N})")

# Check coherence_acc shape

if self.coherence_acc.shape != (self.N, self.N):

raise RuntimeError(f"coherence_acc shape {self.coherence_acc.shape} != ({self.N}, {self.N})")

# Check B symmetry

if not np.allclose(self.B, self.B.T, atol=1e-12):

raise RuntimeError("B matrix is not symmetric")
```

```
# Check B diagonal

if not np.allclose(np.diag(self.B), 0, atol=1e-12):

raise RuntimeError("B matrix diagonal is not zero")

# Check B non-negativity

if np.any(self.B &lt; -1e-12):

raise RuntimeError("B matrix has negative entries")

================================================================================
Simulation Core (exact reproduction of frozen production)
================================================================================

def initialize_state(

text

seed: int,

config: ModelConfig,

) -> DynamicState:

text

"""Initialize state from seed using frozen algorithm exactly.
```

Matches production: simulate_prefix() initialization.

Uses: root = SeedSequence(seed); init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence)

"""

N = config.N

rng = make_prefix_rng(seed)

Initialize theta: uniform in [0, 2*pi) - matching production

theta = rng.random(N) *2.0* np.pi

Initialize B: small random symmetric matrix

B = config.B_init_scale * rng.random((N, N))

B = 0.5 * (B + B.T)

np.fill_diagonal(B, 0.0)

Initialize coherence accumulator

coherence_acc = np.zeros((N, N), dtype=complex)

```
# Initialize history: DELAY_STEPS + 1 copies of theta

history = [theta.copy() for *in range(config.delay*steps + 1)]



return DynamicState(

theta=theta,

B=B,

coherence_acc=coherence_acc,

history=history,

)

def compute_R(

text

state: DynamicState,

config: ModelConfig,

) -> np.ndarray:

text

"""Compute coherence matrix R from current state.
```

Exact reproduction of production compute_R().

"""

N = state.N

delayed = state.history[0]

Convert omega to numpy array for indexing

omega_arr = np.asarray(config.omega, dtype=float)

dphi = (

state.theta[:, None]

- delayed[None, :]

- ((omega_arr[:, None] + omega_arr[None, :]) / 2.0) * config.tau

)

z_inst = np.exp(1j * dphi)

Update EMA

```
state.coherence_acc[:] = (

config.alpha *state.coherence_acc + (1.0 - config.alpha)* z_inst

)



# Compute magnitude and symmetrize

result = np.abs(state.coherence_acc)

result = 0.5 * (result + result.T)

np.fill_diagonal(result, 0.0)



return result

def advance_state(

text

state: DynamicState,

config: ModelConfig,

rng: np.random.Generator,

shuffle_driver: bool = False,

) -> tuple[np.ndarray, np.ndarray, dict[str, Any]]:
```

text

"""Advance state by one time step.

Exact reproduction of production advance_state().

Returns (R_physical, R_driver, info).

Production also returns projection diagnostics:

- projection_active: whether correction was applied

- projection_norm: norm of correction

- projection_relative_state: correction_norm / ||B_unprojected||

- projection_relative_update: correction_norm / ||B_unprojected - B_previous||

"""

N = state.N

edges = compute_edges(N)

Store previous B for projection

B_previous = state.B.copy()

Compute R matrices

R_physical = compute_R(state, config)

R_driver = R_physical

if shuffle_driver:

values = R_physical[edges].copy()

rng.shuffle(values)

R_driver = np.zeros_like(R_physical)

R_driver[edges] = values

R_driver[(edges[1], edges[0])] = values

Update B

B_unprojected = state.B + config.dt * (

(config.eta + config.mu) * R_driver

- config.mu

- config.gamma * state.B

)

B_unprojected = np.maximum(B_unprojected, 0.0)

B_unprojected = 0.5 * (B_unprojected + B_unprojected.T)

np.fill_diagonal(B_unprojected, 0.0)

Projection diagnostics (matching production exactly)

proposed_update_norm = float(np.linalg.norm(B_unprojected - B_previous))

Check gap constraint

row_sum_pre = np.sum(B_unprojected, axis=1)

g_min_pre = float(np.min(config.a - config.q * row_sum_pre))

pre_violation = bool(g_min_pre < config.delta)

Project if needed

B_projected = B_unprojected

correction_norm = 0.0

if pre_violation:

```
scale = np.minimum(1.0, config.capacity / (row_sum_pre + config.scale_epsilon))

B_projected = B_unprojected * np.minimum.outer(scale, scale)

np.fill_diagonal(B_projected, 0.0)

correction_norm = float(np.linalg.norm(B_projected - B_unprojected))



projection_active = bool(correction_norm &gt; FROZEN_PROJECTION_NORM_TOL)

relative_state = correction_norm / max(

float(np.linalg.norm(B_unprojected)), FROZEN_ZERO_TOL

)

relative_update = correction_norm / max(proposed_update_norm, FROZEN_ZERO_TOL)



# Validate projection

state.B = B_projected

row_sum = np.sum(state.B, axis=1)

g_min = float(np.min(config.a - config.q * row_sum))

if g_min &lt; [config.delta](http://config.delta) - 1e-9:

raise RuntimeError(f"Gap projection failed: g_min={g_min}")
```

```
# Update theta

delayed = state.history[0]

omega_arr = np.asarray([config.omega](http://config.omega), dtype=float)

coupling = np.sum(

state.B * np.sin(delayed[None, :] - state.theta[:, None]), axis=1

)

state.theta = np.mod(

state.theta + config.dt * (omega_arr + coupling),

2.0 * np.pi

)



# Update history

state.history.append(state.theta.copy())

if len(state.history) &gt; config.delay_steps + 1:

state.history.pop(0)
```

```
info = {

"pre_violation": pre_violation,

"projection_active": projection_active,

"projection_norm": correction_norm,

"projection_relative_state": relative_state,

"projection_relative_update": relative_update,

"g_min_pre": g_min_pre,

"g_min": g_min,

}



return R_physical, R_driver, info

def simulate_prefix(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

) -> tuple[DynamicState, dict[str, Any]]:
```

text

"""Simulate prefix up to split_time.

Exact reproduction of production simulate_prefix().

Returns (state, prefix_dict).

Production uses:

root = SeedSequence(seed)

init_sequence, _ = root.spawn(2)

rng = default_rng(init_sequence) # for initialization

Then in the loop, uses:

unused_rng = default_rng(0) # for advance_state when shuffle_driver=False

"""

N = config.N

edges = compute_edges(N)

Initialize state (uses init_sequence via make_prefix_rng)

```
state = initialize_state(seed, config)



# Get RNG for prefix simulation loop

# Production uses unused_rng = default_rng(0) for the advance_state calls

# when shuffle_driver=False

unused_rng = np.random.default_rng(0)



# Simulate to split point

R_past_sum = np.zeros((N, N), dtype=float)

R_past_count = 0



burn_end = protocol.burn_end(config)

hist_end = protocol.hist_end(config)



for step in range(hist_end):

R_physical, *,* = advance_state(state, config, unused_rng, shuffle_driver=False)
```

if burn_end <= step < hist_end:

R_past_sum += R_physical

R_past_count += 1

prefix = {

"B_at_split": state.B.copy(),

"R_physical_past": R_past_sum / R_past_count if R_past_count > 0 else np.zeros((N, N)),

"theta_at_split": state.theta.copy(),

"coherence_acc_at_split": state.coherence_acc.copy(),

"history_at_split": np.stack(state.history),

"selected_at_split": (state.B > config.B_c).astype(np.uint8),

}

return state, prefix

def apply_intervention(

text

state: DynamicState,

branch: str,

config: ModelConfig,

rng: np.random.Generator,

) -> None:

text

"""Apply intervention to state. Mutates in place.

Exact reproduction of production apply_intervention().

"""

N = state.N

if branch in {"reset_ema", "reset_B_ema", "B_carrier_only", "full_reset"}:

Zero coherence accumulator

state.coherence_acc.fill(0.0)

if branch in {"reset_B", "reset_B_ema", "full_reset"}:

Zero B

```
state.B.fill(0.0)



if branch in {"B_carrier_only", "full_reset"}:

# Randomize theta and history

new_theta = rng.random(N) *2.0* np.pi

state.theta = new_theta

state.history = [new_theta.copy() for *in range(config.delay*steps + 1)]



# Ensure symmetry

state.B = 0.5 * (state.B + state.B.T)

np.fill_diagonal(state.B, 0.0)

def simulate_future(

text

split_state: DynamicState,

prefix: dict[str, Any],

branch: str,

config: ModelConfig,
```

protocol: ProtocolConfig,

rng: np.random.Generator,

) -> dict[str, Any]:

text

"""Simulate future trajectory from split state.

Reproduces production simulate_future() with ALL required outputs.

CRITICAL: Now accumulates BOTH horizon systems:

Fixed Physical Horizons (time-based, NOT T_window dependent):

- immediate: 0 to 17.5 time units

- recovery: 17.5 to 87.5 time units

- late: 87.5 to 320 time units

EMA-Normalized Horizons (T_window dependent):

- immediate: 0 to 1*T_window

- recovery: 1*T_window to 5*T_window

- late: 5*T_window to end

At rho=1 (T_window=17.502917...), the fixed physical and normalized horizons

are approximately aligned but use exact step boundaries.

Production accumulates:

- P_full, P_immediate, P_recovery, P_late (incidence matrices)

- B_mean_full, B_mean_immediate, B_mean_recovery, B_mean_late

- R_physical_future, R_driver_future

- Projection diagnostics via ProjectionTotals

- Various scalar metrics

Uses horizon system from production for normalized:

- WINDOW_STEPS = round(T_window / dt)

- RECOVERY_END = min(5 * WINDOW_STEPS, FUTURE_STEPS)

For fixed physical:

- FIXED_IMMEDIATE_END = round(17.5 / dt)

- FIXED_RECOVERY_END = round(87.5 / dt)

"""

state = split_state.clone()

apply_intervention(state, branch, config, rng)

N = state.N

edges = compute_edges(N)

future_steps = protocol.future_steps(config)

Compute horizon step boundaries

Normalized EMA horizons (T_window dependent)

window_steps = int(round(config.T_window / config.dt))

recovery_end_ema = min(5 * window_steps, future_steps)

```
# Fixed physical horizons (NOT T_window dependent)

# These use the production's physical time boundaries: 17.5, 87.5, 320

fixed_immediate_end = int(round(17.5 / config.dt)) # 17.5 time units

fixed_recovery_end = int(round(87.5 / config.dt)) # 87.5 time units

# Late goes from 87.5 to end (320 time units)



# Clamp all boundaries to future_steps

recovery_end_ema = min(recovery_end_ema, future_steps)

fixed_immediate_end = min(fixed_immediate_end, future_steps)

fixed_recovery_end = min(fixed_recovery_end, future_steps)



# Initialize accumulators for BOTH horizon systems

# P matrices (incidence: B &gt; B_c as 0/1)

P_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),
```

```
"late": np.zeros((N, N), dtype=float),

}

P_sums_fixed = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# B matrices (mean B values)

B_sums_ema = {

"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}

B_sums_fixed = {
```

```
"full": np.zeros((N, N), dtype=float),

"immediate": np.zeros((N, N), dtype=float),

"recovery": np.zeros((N, N), dtype=float),

"late": np.zeros((N, N), dtype=float),

}



# Counts for each horizon in both systems

counts_ema = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}

counts_fixed = {"full": 0, "immediate": 0, "recovery": 0, "late": 0}



# R matrices

R_physical_sum = np.zeros((N, N), dtype=float)

R_driver_sum = np.zeros((N, N), dtype=float)



# Projection accumulator (matching production)

projection = ProjectionTotals()
```

```
# Trajectory accumulators

sample_times = []

density_trajectory = []

similarity_trajectory = []



# For retention/emergence calculations

r_sum = 0.0

r_count = 0

r_late_ema_sum = 0.0

r_late_ema_count = 0

r_late_fixed_sum = 0.0

r_late_fixed_count = 0



B_reference = prefix["B_at_split"]

shuffle_driver = (branch == "future_shuffle")



for step in range(future_steps):
```

```
R_physical, R_driver, info = advance_state(

state, config, rng, shuffle_driver=shuffle_driver

)


# Update projection accumulator

projection.update(info)


# Accumulate R matrices

R_physical_sum += R_physical

R_driver_sum += R_driver


# Determine which horizons this step belongs to for BOTH systems

selected = (state.B &gt; config.B_c).astype(float)


# === EMA-Normalized Horizons ===

# Full: all steps

P_sums_ema["full"] += selected

B_sums_ema["full"] += state.B

counts_ema["full"] += 1
```

immediate: 0 to window_steps (exclusive)

if step < window_steps:

P_sums_ema["immediate"] += selected

B_sums_ema["immediate"] += state.B

counts_ema["immediate"] += 1

recovery: window_steps to recovery_end_ema (exclusive)

if window_steps <= step < recovery_end_ema:

P_sums_ema["recovery"] += selected

B_sums_ema["recovery"] += state.B

counts_ema["recovery"] += 1

late: recovery_end_ema to end

if recovery_end_ema <= step:

P_sums_ema["late"] += selected

B_sums_ema["late"] += state.B

counts_ema["late"] += 1

```
# === Fixed Physical Horizons ===

# Full: all steps (same as EMA)

P_sums_fixed["full"] += selected

B_sums_fixed["full"] += state.B

counts_fixed["full"] += 1


# immediate: 0 to fixed_immediate_end (exclusive)

if step &lt; fixed_immediate_end:

P_sums_fixed["immediate"] += selected

B_sums_fixed["immediate"] += state.B

counts_fixed["immediate"] += 1


# recovery: fixed_immediate_end to fixed_recovery_end (exclusive)

if fixed_immediate_end &lt;= step &lt; fixed_recovery_end:

P_sums_fixed["recovery"] += selected

B_sums_fixed["recovery"] += state.B

counts_fixed["recovery"] += 1


# late: fixed_recovery_end to end
```

if fixed_recovery_end <= step:

P_sums_fixed["late"] += selected

B_sums_fixed["late"] += state.B

counts_fixed["late"] += 1

Trajectory sampling - CRITICAL: r_mean_late is ONLY accumulated here

if step % protocol.diagnostic_stride == 0:

t = (step + 1) * config.dt

sample_times.append(t)

Density: fraction of edges above B_c

density = float(np.count_nonzero(state.B[edges] > config.B_c) / len(edges[0]))

density_trajectory.append(density)

Similarity: cosine between current and reference B

B_vec = state.B[edges]

B_ref_vec = B_reference[edges]

norm_B = np.linalg.norm(B_vec)

```
norm_ref = np.linalg.norm(B_ref_vec)

if norm_B &gt; 0 and norm_ref &gt; 0:

similarity = float([np.dot](http://np.dot)(B_vec, B_ref_vec) / (norm_B * norm_ref))

else:

similarity = float('nan')

similarity_trajectory.append(similarity)


# Track overall r

r_value = float(np.abs(np.mean(np.exp(1j * state.theta))))

r_sum += r_value

r_count += 1


# Track late-phase r for BOTH horizon systems

if recovery_end_ema &lt;= step:

r_late_ema_sum += r_value

r_late_ema_count += 1


if fixed_recovery_end &lt;= step:

r_late_fixed_sum += r_value
```

r_late_fixed_count += 1

Compute mean matrices for BOTH horizon systems

P_ema = {name: P_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

P_fixed = {name: P_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_ema = {name: B_sums_ema[name] / counts_ema[name] if counts_ema[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

B_mean_fixed = {name: B_sums_fixed[name] / counts_fixed[name] if counts_fixed[name] > 0 else np.zeros((N, N))

for name in ["full", "immediate", "recovery", "late"]}

Compute R means

R_physical_mean = R_physical_sum / future_steps if future_steps > 0 else np.zeros((N, N))

```
R_driver_mean = R_driver_sum / future_steps if future_steps &gt; 0 else np.zeros((N, N))



# Final state

B_final = state.B.copy()

final_weights = state.B[edges]

final_rows = np.sum(state.B, axis=1)



# Retention and emergence for BOTH horizon systems

split_selected = prefix["selected_at_split"][edges].astype(bool)



# EMA-normalized

p_late_edges_ema = P_ema["late"][edges]

retention_ema = (

float(np.mean(p_late_edges_ema[split_selected]))

if np.any(split_selected)

else float('nan')

)
```

```
emergence_ema = (

float(np.mean(p_late_edges_ema[~split_selected]))

if np.any(~split_selected)

else float('nan')

)



# Fixed physical

p_late_edges_fixed = P_fixed["late"][edges]

retention_fixed = (

float(np.mean(p_late_edges_fixed[split_selected]))

if np.any(split_selected)

else float('nan')

)

emergence_fixed = (

float(np.mean(p_late_edges_fixed[~split_selected]))

if np.any(~split_selected)

else float('nan')
```

```
)

# Minimum similarity from trajectory
similarity_array = np.array(similarity_trajectory, dtype=float)
finite_similarity = similarity_array[np.isfinite(similarity_array)]
minimum_similarity = (
float(np.min(finite_similarity)) if len(finite_similarity) &gt; 0
else float('nan')
)

# Build result with BOTH horizon systems
result = {
# === Matrix outputs for EMA-normalized horizons (backward compatible) ===
"B_final": B_final,
"B_mean_late": B_mean_ema["late"],
"P_full": P_ema["full"],
"P_immediate": P_ema["immediate"],
```

"P_recovery": P_ema["recovery"],

"P_late": P_ema["late"],

"R_physical_future": R_physical_mean,

"R_driver_future": R_driver_mean,

=== Matrix outputs for FIXED-PHYSICAL horizons (NEW) ===

"B_mean_late_fixed_physical": B_mean_fixed["late"],

"P_full_fixed_physical": P_fixed["full"],

"P_immediate_fixed_physical": P_fixed["immediate"],

"P_recovery_fixed_physical": P_fixed["recovery"],

"P_late_fixed_physical": P_fixed["late"],

=== Matrix outputs for NORMALIZED-EMA horizons (NEW - explicit) ===

"B_mean_late_normalized_EMA": B_mean_ema["late"],

"P_full_normalized_EMA": P_ema["full"],

"P_immediate_normalized_EMA": P_ema["immediate"],

"P_recovery_normalized_EMA": P_ema["recovery"],

"P_late_normalized_EMA": P_ema["late"],

Trajectory outputs

"sample_times": np.array(sample_times, dtype=float),

"density_trajectory": np.array(density_trajectory, dtype=float),

"similarity_trajectory": np.array(similarity_trajectory, dtype=float),

Scalar outcomes - EMA-normalized (backward compatible names)

"D_B_final": float(np.count_nonzero(final_weights > config.B_c) / len(edges[0])),

"P_mean_full": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late": float(np.mean(P_ema["late"][edges])),

Scalar outcomes - FIXED-PHYSICAL (NEW - primary estimands)

"P_mean_full_fixed_physical": float(np.mean(P_fixed["full"][edges])),

"P_mean_immediate_fixed_physical": float(np.mean(P_fixed["immediate"][edges])),

"P_mean_recovery_fixed_physical": float(np.mean(P_fixed["recovery"][edges])),

"P_mean_late_fixed_physical": float(np.mean(P_fixed["late"][edges])),

```
# Scalar outcomes - NORMALIZED-EMA (NEW - explicit)

"P_mean_full_normalized_EMA": float(np.mean(P_ema["full"][edges])),

"P_mean_immediate_normalized_EMA": float(np.mean(P_ema["immediate"][edges])),

"P_mean_recovery_normalized_EMA": float(np.mean(P_ema["recovery"][edges])),

"P_mean_late_normalized_EMA": float(np.mean(P_ema["late"][edges])),


# Similarity metrics - BOTH systems

"Weighted_similarity_final": cosine_edges(B_reference, B_final, edges),

"Weighted_similarity_late": cosine_edges(B_reference, B_mean_ema["late"], edges),

"Weighted_similarity_late_fixed_physical": cosine_edges(B_reference, B_mean_fixed["late"], edges),

"Weighted_similarity_late_normalized_EMA": cosine_edges(B_reference, B_mean_ema["late"],
edges),

"Jaccard_final": jaccard_edges(B_reference, B_final, edges, config.B_c),

"Jaccard_final_fixed_physical": jaccard_edges(B_reference, B_final, edges, config.B_c),


# Retention/emergence - BOTH systems

"Retention_selected_late": retention_ema,

"Emergence_unselected_late": emergence_ema,
```

"Retention_contrast_late": retention_ema - emergence_ema,

"Retention_selected_late_fixed_physical": retention_fixed,

"Emergence_unselected_late_fixed_physical": emergence_fixed,

"Retention_contrast_late_fixed_physical": retention_fixed - emergence_fixed,

"Retention_selected_late_normalized_EMA": retention_ema,

"Emergence_unselected_late_normalized_EMA": emergence_ema,

"Retention_contrast_late_normalized_EMA": retention_ema - emergence_ema,

Correlations with past R - BOTH systems

"Corr_Rpast_Pfull": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

"Corr_Rpast_Pfull_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["full"][edges]),

"Corr_Rpast_Plate_fixed_physical": safe_corr(prefix["R_physical_past"][edges], P_fixed["late"][edges]),

"Corr_Rpast_Pfull_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["full"][edges]),

"Corr_Rpast_Plate_normalized_EMA": safe_corr(prefix["R_physical_past"][edges], P_ema["late"][edges]),

Weight statistics (same for both - based on final state)

"Total_edge_weight": float(np.sum(final_weights)),

"Mean_row_sum": float(np.mean(final_rows)),

"Max_row_sum": float(np.max(final_rows)),

"Nonzero_edge_fraction": float(np.mean(final_weights > config.scale_epsilon)),

"Effective_edges": effective_number(final_weights),

"Weight_gini": gini_coefficient(final_weights),

"Weight_entropy": normalized_entropy(final_weights),

Order parameter - BOTH systems

"r_mean_future": float(r_sum / r_count) if r_count > 0 else float('nan'),

"r_mean_late": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

"r_mean_late_fixed_physical": float(r_late_fixed_sum / r_late_fixed_count) if r_late_fixed_count > 0 else float('nan'),

"r_mean_late_normalized_EMA": float(r_late_ema_sum / r_late_ema_count) if r_late_ema_count > 0 else float('nan'),

Minimum similarity

"Minimum_similarity": minimum_similarity,

```
# Projection diagnostics (from ProjectionTotals)
**projection.metrics(),

# Horizon metadata
"window_steps": window_steps,
"recovery_end_ema": recovery_end_ema,
"fixed_immediate_end": fixed_immediate_end,
"fixed_recovery_end": fixed_recovery_end,

# Time-based metrics (placeholders - will be NaN for now)
"Time_to_50pct_continue_final_density": float('nan'),
"Time_to_90pct_continue_final_density": float('nan'),
"Time_to_50pct_continue_late_similarity": float('nan'),
"Time_to_90pct_continue_late_similarity": float('nan'),
"Time_to_similarity_below_0_9": float('nan'),
"Time_to_similarity_below_0_5": float('nan'),
"Late_persistence_ratio_to_continue": float('nan'),
"Late_similarity_ratio_to_continue": float('nan'),
```

}

return result

==========
Metric Helpers
==========

def cosine_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None) -> float:

text

"""Compute cosine similarity between reference and current B matrices over edges.

If edges is None, computes over all upper triangle edges.

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = np.asarray(reference[edges], dtype=float)

y = np.asarray(current[edges], dtype=float)

denominator = float(np.linalg.norm(x) * np.linalg.norm(y))

if denominator <= 1e-12:

return float('nan')

return float(np.dot(x, y) / denominator)

def jaccard_edges(reference: np.ndarray, current: np.ndarray, edges: tuple[np.ndarray, np.ndarray] | None = None, threshold: float = FROZEN_ZERO_TOL) -> float:

text

"""Compute Jaccard similarity between reference and current B matrices over edges.

Uses B > threshold as the threshold for edge presence.

If edges is None, computes over all upper triangle edges.

Matches production: jaccard_edges(B_reference, state.B) uses B > B_C

"""

if edges is None:

N = reference.shape[0]

edges = compute_edges(N)

x = reference[edges] > threshold

y = current[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def safe_corr(x: np.ndarray, y: np.ndarray) -> float:

text

"""Compute correlation coefficient safely."""

x = np.asarray(x, dtype=float)

y = np.asarray(y, dtype=float)

valid = np.isfinite(x) & np.isfinite(y)

x = x[valid]

y = y[valid]

if len(x) < 3 or np.std(x) < 1e-12 or np.std(y) < 1e-12:

return float('nan')

```
return float(np.corrcoef(x, y)[0, 1])

def effective_number(weights: np.ndarray) -> float:

text

"""Compute effective number of edges."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

sum_sq = float(np.sum(weights**2))

if total &lt;= 1e-12 or sum_sq &lt;= 1e-12:

return 0.0

return total**2 / sum_sq

def gini_coefficient(weights: np.ndarray) -> float:

text

"""Compute Gini coefficient."""

x = np.sort(np.asarray(weights, dtype=float))

total = float(np.sum(x))

if total &lt;= 1e-12:

return 0.0
```

n = len(x)

ranks = np.arange(1, n + 1, dtype=float)

return float(2.0 *np.dot(ranks, x) / (n* total) - (n + 1.0) / n)

def normalized_entropy(weights: np.ndarray) -> float:

text

"""Compute normalized entropy."""

weights = np.asarray(weights, dtype=float)

total = float(np.sum(weights))

if total <= 1e-12:

return 0.0

probabilities = weights / total

positive = probabilities > 0.0

entropy = -float(np.sum(probabilities[positive] * np.log(probabilities[positive])))

return entropy / float(np.log(len(weights)))

==========
====
Required Output Schema for Baseline Equivalence
==========
====
IMPORTANT: We separate FROZEN keys (must exist in historical NPZ) from
EXTENDED keys (new in v1, only validated internally at rho=1).
The frozen NPZ does NOT contain dual-horizon keys, so baseline equivalence

must only check FROZEN keys against historical data.
Required prefix keys for baseline equivalence

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)
These are the keys that MUST exist in the frozen NPZ for baseline equivalence

FROZEN_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
These are the NEW keys introduced in v1 for dual-horizon support
They will NOT exist in frozen NPZ, so baseline equivalence skips them

EXTENDED_BRANCH_MATRIX_KEYS = (

text

"B_mean_late_fixed_physical",

"P_full_fixed_physical",

"P_immediate_fixed_physical",

"P_recovery_fixed_physical",

"P_late_fixed_physical",

"B_mean_late_normalized_EMA",

"P_full_normalized_EMA",

"P_immediate_normalized_EMA",

"P_recovery_normalized_EMA",

"P_late_normalized_EMA",

)
All matrix keys (frozen + extended)

REQUIRED_BRANCH_MATRIX_KEYS = FROZEN_BRANCH_MATRIX_KEYS + EXTENDED_BRANCH_MATRIX_KEYS

REQUIRED_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Fixed physical horizon scalars (NEW - PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (NEW - explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

Projection diagnostics (from ProjectionTotals)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)
==========
Baseline Equivalence Test
==========

def exact_array_equal_with_nan(a: np.ndarray, b: np.ndarray) -> bool:

text

"""Compare arrays with NaN-aware exact equality."""

a = np.asarray(a)

b = np.asarray(b)

if a.shape != b.shape or a.dtype != b.dtype:

return False

if np.issubdtype(a.dtype, np.floating) or np.issubdtype(a.dtype, np.complexfloating):

a_nan = np.isnan(a)

b_nan = np.isnan(b)

if not np.array_equal(a_nan, b_nan):

return False

finite = ~a_nan

return np.array_equal(a[finite], b[finite])

return np.array_equal(a, b)

def run_baseline_equivalence(

text

seeds: list[int],

config: ModelConfig,

protocol: ProtocolConfig,

frozen_manifest_path: str,

frozen_npz_path: str,

branch_order: tuple[str, ...],

frozen_csv_path: str | None = None,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for all seeds.

This function validates that the engine produces EXACTLY the same outputs

as the frozen production run for all required keys.

Requires NaN-aware exact equality for ALL required schema keys:

- All prefix keys

- All branch matrix keys

- All branch scalar keys

Returns:

status: "passed" | "failed" | "not_run"

seed_results: per-seed validation details

engine_source_sha256: hash of this engine source

frozen_manifest_hash: hash of frozen manifest

frozen_npz_hash: hash of frozen NPZ

"""

import numpy as np

```
from pathlib import Path

import hashlib



# Helper function to compute source hash

def *source*sha256() -&gt; str | None:

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None



# Early failure: frozen scalar CSV is required for scalar equivalence

if frozen_csv_path is None:

return {

"status": "failed",

"error": "frozen scalar CSV is required for scalar equivalence",

"frozen_manifest": str(Path(frozen_manifest_path).resolve()),

"frozen_npz": str(Path(frozen_npz_path).resolve()),
```

"frozen_csv_path": None,

}

Record engine source hash

engine_source_hash = *source*sha256()

results = {

"status": "not_run",

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"mean_omega": config.mean_omega,

"T_window": config.T_window,

"T_star": config.T_star,

"alpha": config.alpha,

"a": config.a,

"q": config.q,

"delta": config.delta,

"eta": config.eta,

"mu": config.mu,

"gamma": config.gamma,

"B_c": config.B_c,

"B_init_scale": config.B_init_scale,

"scale_epsilon": config.scale_epsilon,

"capacity": config.capacity,

"delay_steps": config.delay_steps,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

"diagnostic_stride": protocol.diagnostic_stride,

"branches": list(protocol.branches),

```
},
"n_seeds": len(seeds),
"frozen_manifest": str(Path(frozen_manifest_path).resolve()),
"frozen_npz": str(Path(frozen_npz_path).resolve()),
"engine_source_sha256": engine_source_hash,
"seed_results": {},
}


# Load frozen manifest
with open(frozen_manifest_path, "r") as f:
frozen_manifest = json.load(f)


# Load frozen data
with np.load(frozen_npz_path, allow_pickle=False) as frozen_data:
frozen_files = dict(frozen_data)


# Load frozen scalar CSV if provided
```

```
frozen_scalar_csv: dict[str, dict[str, Any]] = {}

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

try:

import csv

with open(frozen_csv_path, "r", newline="", encoding="utf-8") as f:

reader = csv.DictReader(f)

for row in reader:

seed_id = row.get("seed", "")

branch = row.get("branch", "")

if seed_id and branch:

key = f"{branch}_seed_{seed_id}"

if key not in frozen_scalar_csv:

frozen_scalar_csv[key] = {}

for k, v in row.items():

if k not in ("seed", "seed_index", "branch", "rho", "T_window"):

try:

frozen_scalar_csv[key][k] = float(v)
```

```
except (ValueError, TypeError):

frozen_scalar_csv[key][k] = v

except Exception as e:

print(f"Warning: Could not load frozen CSV {frozen_csv_path}: {e}")



# Record frozen hashes

results["frozen_manifest_hash"] = hashlib.sha256(

json.dumps(frozen_manifest, sort_keys=True).encode()

).hexdigest()

results["frozen_npz_hash"] = file_sha256(frozen_npz_path)

if frozen_csv_path is not None and os.path.exists(frozen_csv_path):

results["frozen_csv_hash"] = file_sha256(frozen_csv_path)



# Validate config matches frozen manifest model - use EXACT equality

frozen_model = frozen_manifest["model"]



# Check N
```

if frozen_model["N"] != config.N:

results["status"] = "failed"

results["error"] = f"N mismatch: config={config.N}, frozen={frozen_model['N']}"

return results

Check omega - use exact equality

frozen_omega_arr = np.array(frozen_model["omega"], dtype=float)

config_omega_arr = np.array(list(config.omega), dtype=float)

if not np.array_equal(frozen_omega_arr, config_omega_arr):

results["status"] = "failed"

results["error"] = f"omega mismatch: frozen={frozen_model['omega']}, config={list(config.omega)}"

return results

Check all other model params with EXACT equality (rtol=0, atol=0)

model_checks = [

("dt", frozen_model["dt"], config.dt),

("tau", frozen_model["tau"], config.tau),

("a", frozen_model["a"], config.a),

("q", frozen_model["q"], config.q),

("delta", frozen_model["delta"], config.delta),

("eta", frozen_model["eta"], config.eta),

("mu", frozen_model["mu"], config.mu),

("gamma", frozen_model["gamma"], config.gamma),

("B_c", frozen_model["B_c"], config.B_c),

("B_init_scale", frozen_model.get("B_init_scale", 0.0), config.B_init_scale),

("T_window", frozen_model["T_window"], config.T_window),

]

for name, frozen_val, config_val in model_checks:

if not np.isclose(frozen_val, config_val, rtol=0.0, atol=0.0):

results["status"] = "failed"

results["error"] = f"{name} mismatch: frozen={frozen_val}, config={config_val}"

return results

```
# Check derived values

if not np.isclose(frozen_model.get("alpha", 0.0), config.alpha, rtol=0.0, atol=1e-15):

results["status"] = "failed"

results["error"] = f"alpha mismatch: frozen={frozen_model.get('alpha')}, config={config.alpha}"

return results



# Now run actual equivalence test for each seed

print("Running baseline equivalence test...")

all_passed = True



for seed in seeds:

seed_str = str(seed)

seed_results = {"status": "passed", "checks": {}, "max_diffs": {}}



try:

# Simulate with our engine

split_state, prefix = simulate_prefix(seed, config, protocol)
```

```
branch_rngs = make_branch_rngs(seed, branch_order)


# Simulate each branch

engine_results = {}

for branch in protocol.branches:

rng = branch_rngs[branch]

engine_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)


# Check prefix state - iterate over REQUIRED_PREFIX_KEYS

for key in REQUIRED_PREFIX_KEYS:

engine_val = prefix.get(key)
```

```
frozen_key = f"prefix_seed_{seed_str}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"prefix_{key}_missing"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"prefix_{key}_exact"] = True

else:

seed_results["checks"][f"prefix_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)
```

```
seed_results["max_diffs"][f"prefix_{key}"] = float(np.max(np.abs(e - f)))


# Check branch outputs - separate FROZEN vs EXTENDED keys

for branch in protocol.branches:

engine_branch = engine_results[branch]

frozen_prefix = f"{branch}_seed_{seed_str}"


# === FROZEN MATRIX KEYS: Must exist in historical NPZ ===

for key in FROZEN_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


if frozen_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

all_passed = False

continue
```

```
if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


if exact_array_equal_with_nan(engine_val, frozen_val):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(frozen_val, np.ndarray):

e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(frozen_val), 0.0, frozen_val)

seed_results["max_diffs"][f"{branch}_{key}"] = float(np.max(np.abs(e - f)))


# === EXTENDED MATRIX KEYS: Validate internally at rho=1 ===
```

```
# These do NOT exist in frozen NPZ. At rho=1, both horizon systems

# should produce identical results (1750 and 8750 steps match).

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_MATRIX_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# Map extended key to its frozen equivalent

frozen_key_map = {

"B_mean_late_fixed_physical": "B_mean_late",

"P_full_fixed_physical": "P_full",

"P_immediate_fixed_physical": "P_immediate",

"P_recovery_fixed_physical": "P_recovery",

"P_late_fixed_physical": "P_late",
```

```
"B_mean_late_normalized_EMA": "B_mean_late",

"P_full_normalized_EMA": "P_full",

"P_immediate_normalized_EMA": "P_immediate",

"P_recovery_normalized_EMA": "P_recovery",

"P_late_normalized_EMA": "P_late",

}


frozen_equiv_key = frozen_key_map.get(key)

if frozen_equiv_key:

engine_frozen_equiv = engine_branch.get(frozen_equiv_key)

if engine_frozen_equiv is not None:

if exact_array_equal_with_nan(engine_val, engine_frozen_equiv):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

if isinstance(engine_val, np.ndarray) and isinstance(engine_frozen_equiv, np.ndarray):
```

```
e = np.where(np.isnan(engine_val), 0.0, engine_val)

f = np.where(np.isnan(engine_frozen_equiv), 0.0, engine_frozen_equiv)

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(np.max(np.abs(e - f)))


# === SCALAR KEYS ===

# Separate FROZEN vs EXTENDED scalars

# First, define which scalars are frozen (exist in historical data)

FROZEN_BRANCH_SCALAR_KEYS = (

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",
```

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

)

EXTENDED_BRANCH_SCALAR_KEYS = (

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

```
"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)


for key in FROZEN_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)


# Get from CSV first (production stores scalars in CSV)

frozen_val = None

csv_key = f"{branch}_seed_{seed_str}"

if csv_key in frozen_scalar_csv and key in frozen_scalar_csv[csv_key]:

frozen_val = frozen_scalar_csv[csv_key][key]

else:

# Fallback to NPZ

frozen_key = f"{frozen_prefix}_{key}"

frozen_val = frozen_files.get(frozen_key)


# FROZEN scalars MUST exist in historical data

if frozen_val is None:
```

```
seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_frozen"] = True

all_passed = False

continue


if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue


# For scalars, compare directly

if isinstance(engine_val, (int, float, np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_val)) if isinstance(frozen_val, (int, float, np.number)) else False


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_exact"] = True
```

```
elif np.isclose(engine_val, frozen_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_exact"] = True

else:

seed_results["checks"][f"{branch}_{key}_exact"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}"] = float(abs(engine_val - frozen_val)) if not
(engine_is_nan or frozen_is_nan) else float('nan')



# === EXTENDED SCALAR KEYS: Validate internally at rho=1 ===

if math.isclose(config.T_window, config.T_star, rel_tol=0.0, abs_tol=0.0):

for key in EXTENDED_BRANCH_SCALAR_KEYS:

engine_val = engine_branch.get(key)

if engine_val is None:

seed_results["status"] = "failed"

seed_results["checks"][f"{branch}_{key}_missing_in_engine"] = True

all_passed = False

continue
```

```
# Map to frozen equivalent

frozen_scalar_map = {

"P_mean_late_fixed_physical": "P_mean_late",

"Weighted_similarity_late_fixed_physical": "Weighted_similarity_late",

"Retention_contrast_late_fixed_physical": "Retention_contrast_late",

"Corr_Rpast_Plate_fixed_physical": "Corr_Rpast_Plate",

"r_mean_late_fixed_physical": "r_mean_late",

"P_mean_late_normalized_EMA": "P_mean_late",

"Weighted_similarity_late_normalized_EMA": "Weighted_similarity_late",

"Retention_contrast_late_normalized_EMA": "Retention_contrast_late",

"Corr_Rpast_Plate_normalized_EMA": "Corr_Rpast_Plate",

"r_mean_late_normalized_EMA": "r_mean_late",

}


frozen_equiv = frozen_scalar_map.get(key)

if frozen_equiv:

# Get the frozen equivalent value from CSV

csv_key = f"{branch}_seed_{seed_str}"
```

```
if csv_key in frozen_scalar_csv and frozen_equiv in frozen_scalar_csv[csv_key]:

frozen_equiv_val = frozen_scalar_csv[csv_key][frozen_equiv]

if isinstance(engine_val, (int, float, np.number)) and isinstance(frozen_equiv_val, (int, float,
np.number)):

engine_is_nan = bool(np.isnan(engine_val))

frozen_is_nan = bool(np.isnan(frozen_equiv_val))


if engine_is_nan and frozen_is_nan:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

elif np.isclose(engine_val, frozen_equiv_val, rtol=0.0, atol=0.0):

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = True

else:

seed_results["checks"][f"{branch}_{key}_equals_frozen_at_rho1"] = False

seed_results["status"] = "failed"

all_passed = False

seed_results["max_diffs"][f"{branch}_{key}_vs_frozen"] = float(abs(engine_val - frozen_equiv_val))


except Exception as e:

seed_results["status"] = "failed"
```

```
seed_results["error"] = str(e)

import traceback

seed_results["traceback"] = traceback.format_exc()

all_passed = False


results["seed_results"][seed_str] = seed_results

print(f" Seed {seed_str}: {seed_results['status']}")



if all_passed:

results["status"] = "passed"

print("Baseline equivalence: PASSED")

else:

results["status"] = "failed"

print("Baseline equivalence: FAILED")

# Print details of failures

for seed_str, seed_results in results["seed_results"].items():

if seed_results.get("status") != "passed":

print(f" Seed {seed_str} failures:")
```

```
for check, val in seed_results.get("checks", {}).items():

if not val:

print(f" CHECK FAILED: {check}")

for key, val in seed_results.get("max_diffs", {}).items():

print(f" MAX DIFF: {key} = {val}")



return results

================================================================================
Main API
================================================================================

def run_seed(

text

seed: int,

config: ModelConfig,

protocol: ProtocolConfig,

branch_order: tuple[str, ...],

) -> dict[str, Any]:
```

text

"""Run a single seed through prefix + all branches.

Returns complete seed results with all branches.

"""

Simulate prefix

split_state, prefix = simulate_prefix(seed, config, protocol)

Create branch RNGs using frozen algorithm

branch_rngs = make_branch_rngs(seed, branch_order)

Use branch_order to determine which branches to run

All branches in branch_order must have RNGs

branch_results = {}

for branch in branch_order:

if branch not in branch_rngs:

raise ValueError(

```
f"Branch '{branch}' not found in branch_rngs. "

f"All branches must be registered in branch_order for explicit RNG namespace. "

f"Registered branches: {list(branch_rngs.keys())}"

)

rng = branch_rngs[branch]


branch_results[branch] = simulate_future(

split_state.clone(),

prefix,

branch,

config,

protocol,

rng,

)



return {

"seed": int(seed),

"config": {
```

```
"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": protocol.T_sim,

"burn_end_time": protocol.burn_end_time,

"split_time": protocol.split_time,

"future_duration": protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}
```

```
if name == "main":

text

# Quick validation

config = DEFAULT_MODEL_CONFIG

config.validate()

protocol = DEFAULT_PROTOCOL_CONFIG

protocol.validate(config)



print(f"Engine v1 loaded successfully")

print(f" N={config.N}, dt={config.dt}, tau={config.tau}")

print(f" T_window={config.T_window}, alpha={config.alpha:.6f}")

print(f" mean_omega={config.mean_omega:.6f}, T_*={config.T_star:.6f}")

print(f" delay_steps={config.delay_steps}")

print(f" Protocol: T_sim={protocol.T_sim}, burn={protocol.burn_end_time}, "

f"split={protocol.split_time}, future={protocol.future_duration}")
```

This engine is now structurally ready for an actual baseline-equivalence test. The frozen/extended schema separation is correct, dual horizons are internally validated at ρ=1ρ=1, frozen scalar comparison is required, projection diagnostics are implemented, and equivalence failures now return detailed results rather than discarding them.

There are no remaining obvious scientific-design blockers in the core simulation path. A few API/provenance issues remain, but the decisive next step is now execution.
1. Frozen versus extended schema is correctly handled

The engine now:

compares only historical matrix keys against the frozen NPZ;
compares historical scalar keys against the frozen scalar CSV;
validates new fixed/normalized fields internally at baseline;
requires the frozen scalar CSV;
uses exact/NaN-aware comparisons.

This resolves the previous guaranteed baseline failure.
2. Baseline equivalence should now be capable of passing

The main sources of earlier mismatch have been corrected:

exact frozen parameters;
exact prefix RNG construction;
exact branch RNG hierarchy;
exact split timing;
exact future timing;
exact delay length;
production horizon boundaries;
diagnostic-stride sampling;
projection diagnostics;
sampled rlaterlate;
full matrix and scalar output schemas.

At this point, a one-seed exact run is more informative than another static rewrite.
3. run_seed still conflates RNG namespace and selected branches

The reusable API still does:

Python

for branch in branch_order:

rather than:

Python

for branch in protocol.branches:

This means passing the full seven-branch RNG order always simulates all seven branches.

The study driver bypasses this function and uses the correct subset logic, so it does not block the intended T-window path. But the engine API should still be corrected before freezing:

Python

```
def run_seed(
seed,
config,
protocol,
rng_branch_order,
):
split_state, prefix = simulate_prefix(
seed,
config,
protocol,
)

branch_rngs = make_branch_rngs(
seed,
rng_branch_order,
)

branch_results = {}

for branch in protocol.branches:
if branch not in branch_rngs:
raise ValueError(...)

branch_results[branch] = (
simulate_future(
```

split_state,
prefix,
branch,
config,
protocol,
branch_rngs[branch],
)
)

...

4. File hash remains mislabeled

The engine records:

Python

frozen_manifest_hash = sha256(
canonicalized JSON
)

rather than the file’s SHA-256.

Add:

Python

results[
"frozen_manifest_file_sha256"
] = file_sha256(
frozen_manifest_path
)

If the canonical hash is retained, label it:

text

frozen_manifest_canonical_json_sha256

5. Baseline protocol constants should also be checked

Model constants are now thoroughly compared. Add protocol comparisons against the frozen manifest:

text

T_sim
n_steps
hist_end
future_steps
window_steps
recovery_end
diagnostic_stride
branch order
delay steps
scale epsilon

This is defensive because exact trajectory equivalence will already detect a mismatch.
6. Extended scalar tuples should move to module scope

FROZEN_BRANCH_SCALAR_KEYS and EXTENDED_BRANCH_SCALAR_KEYS are rebuilt inside every branch/seed loop.

Move them to module scope for:

readability;
schema reuse by the driver;
reduced risk of divergence;
easier manifest serialization.

This is cleanup, not a correctness issue.
7. State validation should use config.N

This remains weak:

Python

self.theta.shape == (self.N,)

because self.N is derived from theta itself.

Add:

Python

```
if self.N != config.N:
raise RuntimeError(...)

if self.theta.shape != (config.N,):
...
```

and validate every history entry shape.
8. Jaccard threshold default is misleading

The helper defaults to:

Python

```
FROZEN_ZERO_TOL
```

while production Jaccard uses BcBc. Current calls explicitly pass config.B_c, so outputs are correct.

Make threshold mandatory to prevent accidental future misuse.
9. Baseline certificate still needs a file-level wrapper

The function returns detailed results on pass or failure, which is good. The driver must atomically save those results before proceeding or stopping.

Required artifacts:

text

baseline_equivalence_certificate.json
baseline_equivalence_by_seed.csv

The certificate should include both frozen artifact hashes and the engine source hash.
10. Potential failure-manifest NaN issue

If one scalar is NaN and its comparison partner is finite, the diagnostic max difference is saved as:

Python

```
float("nan")
```

A strict JSON writer with allow_nan=False cannot serialize that.

Use:

Python

```
None
```

for nonfinite diagnostic differences, or sanitize before writing a failed certificate.
Readiness rating
Area Rating
Frozen model fidelity 9.5/10
RNG fidelity 9.5/10
Step-map fidelity 9/10
Projection diagnostics 9/10
Dual-horizon implementation 9.5/10
Historical schema validation 9/10
Reusable branch API 7/10
Certificate/provenance 8/10
Current baseline-test readiness 9/10
Overall: 9/10 candidate engine
Decisive next action

Do not revise it again before testing.

Run:

compile;
one frozen seed, all seven branches;
exact certificate inspection;
all 40 frozen seeds, all seven branches.

The required one-seed result is:

text

all 6 prefix arrays exact
all 11 historical matrix/trajectory arrays exact for 7 branches
all historical scalar fields exact for 7 branches
all fixed-physical extended fields equal legacy fields
all normalized-EMA extended fields equal legacy fields

If that passes, the engine is ready to freeze and the T-window driver can proceed.

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import sys

import time
```

```
import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

pd = None

import numpy as np

import glob

import tempfile
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
```

Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5

PHYSICAL_LATE_START = 87.5

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

```
"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

# Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
```

All scalar keys (required + desired) for CSV output

```
BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
DESIRED_BRANCH_SCALAR_KEYS
```

All matrix keys

```
BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.



Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())
```

if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

==========
Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

```
int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)
```

```
path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))

finally:

if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text
```

"""Save NPZ atomically with fsync.

Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{path.name}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

```
os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:
```

```
raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

[dataframe.to](http://dataframe.to)_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):
```

os.unlink(temporary)

==========
Configuration and Validation
==========

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

```
gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)
```

```
================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.
```

```
Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)
```

```
# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,
```

```
)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},
```

```
"protocol": {
"T_sim": study_protocol.T_sim,
"burn_end_time": study_protocol.burn_end_time,
"split_time": study_protocol.split_time,
"future_duration": study_protocol.future_duration,
},
"prefix": prefix,
"branches": branch_results,
}
def run_baseline_equivalence(
text
seeds: list[int],
frozen_manifest_path: str,
frozen_npz_path: str,
frozen_csv_path: str,
) -> dict[str, Any]:
text
```

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

```
frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {

"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

"seeds_match": False,

}



# Validate manifest has required structure

if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:

return {
```

```
"status": "failed",

"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",

"seeds_match": False,

}



frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]



# Verify seed list matches frozen manifest EXACTLY

# This is critical: baseline equivalence tests a specific cohort

if len(seeds) != len(frozen_seeds):

return {

"status": "failed",

"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}
```

```
for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Validate frozen CSV path exists
```

```
if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,
```

```
diagnostic_stride=50,
branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification
)

# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches
results = [engine.run](http://engine.run)_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
frozen_csv_path=frozen_csv_path,
branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace
)

# Add seed comparison info
results["seed_comparison"] = {
```

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}

return results

==
Manifest and Output
==

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

Compute config hash from model parameters

```
model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}



protocol_params = {
```

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}

study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}

manifest = {

"study_name": STUDY_NAME,

```
"workflow_version": WORKFLOW_VERSION,
"source_sha256": source_sha256(),
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"randomization": {
"master_seed": int(master_seed),
"paired_seeds": [int(seed) for seed in seeds],
"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",
"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,
},
"environment": {
"python": platform.python_version(),
"numpy": np.__version__,
"platform": platform.platform(),
},
```

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

```
f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(
```

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result)

row = {

"seed_index": seed_index,

```
"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}



for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:
```

```
text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=int,

default=DEFAULT_MASTER_SEED,

help="fixed integer master seed (default: 20260714). For production, must match frozen data.",

)

parser.add_argument(

"--n-seeds",

type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",

)
```

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

```
text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False
```

```
)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Create run directory

run_dir = Path([args.run](http://args.run)_root) / config_hash

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")
```

```
print(f" output: {run_dir}")

print()



# Dry run check

if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Manifest saved to {manifest_path}")

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)
```

```
# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed
20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")
```

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Run baseline equivalence - MANDATORY for all seeds

try:

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

frozen_csv_path=args.frozen_scalar,

)

except Exception as exc:

Handle baseline equivalence exception - save failure manifest

failure_manifest = {

**manifest,

```
"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_exception",

"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")
```

```
# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",
```

```
"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)


# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")
```

```
print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Update manifest with baseline equivalence certificate

manifest["baseline_equivalence"] = {

"status": "passed",

"n_seeds_tested": baseline_results["n_seeds"],

"n_seeds_passed": baseline_results["n_seeds"],

"engine_source_sha256": baseline_results.get("engine_source_sha256"),

"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),

"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),

"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),

"certificate_path": str(baseline_cert_path),

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches
```

```
}

# Re-save manifest with baseline certificate
atomic_write_json(manifest_path, manifest)


# Run the study
print("\nRunning T_window sensitivity study...")
print(f" rho values: {RHO_VALUES}")
print(f" branches: {STUDY_BRANCHES}")
print(f" seeds: {len(seeds)}")

all_scalar_rows: list[dict[str, Any]] = []
all_matrix_payload: dict[str, Any] = {}

# Track condition status
condition_status = []
successful_conditions = 0
failed_conditions = 0
```

```
started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)



for seed_index, seed in enumerate(seeds):

for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


# Try to load existing checkpoint

if checkpoint_path.exists():

try:

# For now, just skip - in production we'd validate and load
```

```
# But to keep it simple, we'll recompute

pass

except Exception as e:

print(f" Warning: Invalid checkpoint {checkpoint_path}: {e}")


try:

# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]
```

```
row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,
```

```
"status": "success",

"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:

condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": str(e),

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Status

elapsed = time.time() - started
```

```
current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

if condition_status:

if pd is not None:

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

else:

# Fallback: save as JSON if pandas not available

status_path = run_dir / "twindow_condition_status.json"

atomic_write_json(status_path, condition_status)
```

```
print(f"\nCondition status saved to {status_path} (pandas not available)")

print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")



# If any conditions failed, mark as non-freezeable

if failed_conditions &gt; 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")
```

```
# Validate primary outcomes are present and not NaN

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or (isinstance(row[key], float) and np.isnan(row[key])):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or NaN!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"
```

```
# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))
```

```
finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

)



# Final manifest update with completion status
```

```
manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"elapsed_seconds": time.time() - started,

}



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Add output hashes

if csv_path.exists():
```

```
manifest["output_hashes"] = {

"scalar_csv": file_sha256(csv_path),

}

if npz_path.exists():

manifest["output_hashes"]["branch_npz"] = file_sha256(npz_path)



# Re-save final manifest

atomic_write_json(manifest_path, manifest)



# Print completion status

if complete and baseline_passed:

print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:
```

```
print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

print(f"Freezeable: {manifest.get('completion', {}).get('freezeable', False)}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")
```

```
print("=" * 80)



# Validate we have primary outcomes before summarizing

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or (isinstance(row[key], float) and np.isnan(row[key])):

missing_primary.append(f"{row['seed']}_rho_{row['rho']}_branch_{row['branch']}_{key}")



if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are NaN or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:
```

```
rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&amp;lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

stats[key] = f"{np.mean(values):.4f}±{np.std(values):.4f}"

else:
```

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

This driver is now structurally capable of running the intended regenerated study. The major baseline, RNG-namespace, frozen-scalar, dual-horizon, and output-schema issues have been corrected.

It is ready for a baseline-equivalence execution after a few provenance and run-management fixes. There are no obvious remaining errors in the scientific estimand itself.
1. Include the engine hash in the configuration hash

The run configuration hash includes the driver source hash but not:

text

aief_experiment_engine_v1.py

If the engine changes while the driver remains unchanged, the same config hash and run directory can be reused for different simulation code.

Add to the initial manifest before computing config_hash:

Python

"dependencies": {
"aief_experiment_engine_v1": {
"path": str(
Path(engine.__file__).resolve()

),
"sha256": file_sha256(
engine.__file__
),
},
},

Because the config hash is calculated from the manifest, engine revisions will then create a new run namespace.

Also add the frozen production source hash once its path is known.
2. Condition checkpoints do not support resume

The driver writes one checkpoint per seed/rho but ignores existing checkpoints:

Python

if checkpoint_path.exists():
pass

This protects completed condition data from total loss, but rerunning recomputes everything.

Either:

rename these “condition snapshots” and remove the apparent resume logic; or
implement --resume.

A real resume path must validate:

text

global config hash
engine source hash
seed
rho
T_window
alpha
study branch list

output schema

before accepting a checkpoint.

For a study of this size, implementing resume is strongly advisable.
3. Protect existing run directories

The config hash maps identical study specifications to the same directory. A rerun can overwrite:

manifest;
baseline certificate;
condition checkpoints;
final outputs.

Default behavior should fail if the directory already contains a completed manifest.

Add explicit modes:

text

--resume
--overwrite

Do not silently mix artifacts.
4. Missing primary-value checks should use numeric finiteness

The current condition:

Python

isinstance(value, float)
and np.isnan(value)

can miss:

NumPy scalar types;

None;
infinity;
nonnumeric strings.

Use:

Python

```
def is_finite_number(value):
return (
isinstance(
value,
(
int,
float,
np.integer,
np.floating,
),
)
and np.isfinite(value)
)
```

Then require all primary values to satisfy it.

Any missing/nonfinite primary outcome must make the run incomplete and nonfreezeable, as intended.
5. Completion status and printed freezeability disagree

The driver stores:

Python

```
manifest["freezeable"] = ...
```

but prints:

Python

```
manifest.get(
```

"completion",
{},
).get(
"freezeable",
False,
)

completion contains no freezeable field, so the printed value is always false.

Either add:

Python

```
manifest["completion"][
"freezeable"
] = manifest["freezeable"]
```

or print the top-level value:

Python

```
print(
f"Freezeable: "
f"{manifest['freezeable']}"
)
```

6. Hash every final output

The final manifest hashes only:

text

```
scalar CSV
branch NPZ
```

Also hash:

text

condition status CSV
baseline equivalence certificate
baseline per-seed certificate
driver source
engine source
frozen manifest
frozen NPZ
frozen scalar CSV

This completes the evidence chain.
7. Record condition-status authority in the manifest

Add:

Python

"condition_status": {
"path": str(status_path),
"sha256": file_sha256(status_path),
"expected_rows": 200,
"successful_rows":
successful_conditions,
"failed_rows":
failed_conditions,
},

The analysis driver should validate this table.
8. Baseline certificate should use actual file hashes

Ensure the equivalence certificate includes:

text

frozen_manifest_file_sha256
frozen_npz_sha256
frozen_scalar_csv_sha256

engine_source_sha256

rather than only a canonicalized JSON hash.
9. Full baseline plus rho=1 study duplicates substantial work

The driver first runs all 40 seeds and seven branches for equivalence, then regenerates rho =1=1 for four branches.

This is computationally redundant but scientifically safe.

Possible optimization after the first successful run:

preserve baseline engine outputs as validated condition checkpoints;
reuse the four study branches for rho =1=1.

Do not optimize until exact equivalence has passed once.
10. Partial outputs are still scientifically written

If some conditions fail, the driver writes partial scalar and NPZ outputs and marks the run nonfreezeable.

That is acceptable for debugging if:

filenames or manifest clearly say incomplete;
the analysis driver refuses nonfreezeable runs;
no scientific report is generated.

The latest analysis validation appears intended to enforce this.
11. Condition errors currently continue across all remaining conditions

A systematic engine/schema failure in the first condition will cause another 199 likely failures.

A useful policy is:

stop immediately on schema/config errors;
optionally continue on clearly seed-specific numerical failures;
always retain status/checkpoint diagnostics.

This is efficiency rather than scientific correctness.
12. Remove the obsolete horizon helper

The old compute_horizon_indices hybrid function has been removed in this revision, which is good. The driver now correctly relies on explicit dual-horizon engine outputs.
13. Minor cleanup

glob appears unused.
tempfile is imported twice.
PHYSICAL_LATE_START duplicates PHYSICAL_RECOVERY_END.
References to REAL.py should use the actual frozen production filename.
DESIRED_BRANCH_SCALAR_KEYS intentionally produce NaN; ensure the analysis does not treat them as required.
Add sample standard deviation labeling to the quick console summary.

Readiness
Area Rating
Scientific design 9.5/10
Full regeneration 9.5/10
RNG namespace 9.5/10
Dual-horizon estimands 9/10
Baseline gate 9/10
Output schema validation 9/10
Run provenance 7.5/10
Resume/large-run robustness 6/10
Current executable readiness 8.5/10
Overall driver: 8.5/10
Decisive next step

Stop static revision after adding the engine hash to the run config.

Run the mandatory baseline equivalence only. Do not launch the five-level grid until the certificate reports:

text

40/40 seeds passed
7/7 branches passed

all frozen prefix arrays exact
all frozen matrix/trajectory arrays exact
all frozen scalar fields exact
all extended baseline fields internally exact

If that passes, the driver is scientifically ready for the regenerated grid.

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
==
Configuration
==

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
====
Utility Functions
==========
====

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

df = pd.read_csv(

```
path,
dtype={"seed": str, "branch": str}, # Keep seeds as strings
)


# Convert numeric columns, coercing errors to NaN
for col in df.columns:
if col not in ("seed", "seed_index", "branch", "rho"):
try:
df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")
except Exception:
pass


return df
def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:
text
"""Load only specified keys from NPZ file to save memory."""
path = Path(path)
```

payload: dict[str, Any] = {}

if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload

try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

```
finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

f.write(text)

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory
```

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Convert NaN and infinity to None for JSON serialization

def sanitize(obj: Any) -> Any:

if isinstance(obj, dict):

```
return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None

return val

elif isinstance(obj, np.ndarray):

return obj.tolist()

else:

return obj
```

```
temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)
f.write("\n")
f.flush()
os.fsync(f.fileno())
temp_path.replace(path)
# fsync the directory
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
```

```
def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)

if not path.exists():

return "FILE_NOT_FOUND"

with open(path, "rb") as f:

for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):

digest.update(block)

return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""
```

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

```
union = np.count_nonzero(x | y)

if union == 0:

return float('nan')



return float(np.count_nonzero(x &amp;amp; y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]



diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)
```

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

==========
Stage 1: Validation
==========

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

```
manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results



# Validate condition status if available

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if condition_status_path.exists():

try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)

successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:
```

```
results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "

f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash
```

```
except Exception as e:

results["warnings"].append(f"Could not validate condition status: {e}")

else:

results["warnings"].append("Condition status file not found")



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})
```

```
freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")
```

```
return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results
```

```
# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)



# Validate unique seeds

unique_seeds = df["seed"].unique()

if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(

f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"

)
```

```
# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)


# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
results["status"] = "failed"
results["errors"].append(
f"Branches mismatch: expected {expected_branches}, got {unique_branches}"
)
```

```
# Validate completeness: each seed/rho/branch combination should exist

missing_combinations = []

for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)
```

```
# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and finite

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():
```

```
results["status"] = "failed"
results["errors"].append(
f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"
)

# Check that values are finite (not NaN, inf, or -inf)
non_finite = ~np.isfinite(df[outcome])
if non_finite.any():
results["status"] = "failed"
results["errors"].append(
f"Primary outcome '{outcome}' has {non_finite.sum()} non-finite values"
)


# Get baseline equivalence certificate hash if available
baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"
baseline_cert_hash = "NOT_FOUND"
if baseline_cert_path.exists():
```

```
baseline_cert_hash = file_sha256(baseline_cert_path)



# Record validation info

results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),

"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", False),

"baseline_certificate_hash": baseline_cert_hash,

}



if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")
```

```
print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")




return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:
```

```
text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]
```

```
.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)
```

```
# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")



rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,
```

```
"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)
```

```
print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")




return contrasts_df

# ============================================================================
# Stage 3: Branch Difference-in-Differences
# ============================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]




For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.
```

```
Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]


for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]
```

```
for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")
```

```
y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,
```

```
"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)
```

print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches: {did_df['branch'].nunique()}")

return did_df

==
Stage 4: Cross-T Topology and Recovery
==

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

```
for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))



# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)
```

```
if rho_key not in npz_data:
missing_keys.append(rho_key)
continue
if baseline_key not in npz_data:
missing_keys.append(baseline_key)
continue

B_rho = npz_data[rho_key]
B_baseline = npz_data[baseline_key]

# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
```

```
"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")
```

return topology_df

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics.

For each seed, rho, and branch, compute recovery metrics relative to

the continue branch for both horizon systems.

For each (seed, rho, branch):

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets

4. Calculate:

- first crossing: first time density > reference threshold

- sustained recovery: duration density stays above threshold

- held duration: time held for one T_window

- censoring indicator: whether recovery was observed

- restricted time: time within observation window

- AUC deficit: area under curve deficit from reference

- t_rec / T_window: normalized recovery time

5. Output separate rows for fixed_physical and normalized_EMA systems

Returns DataFrame with columns:

seed, rho, branch, horizon_system, first_crossing, sustained_recovery,

held_duration, censored, restricted_time, auc_deficit, t_rec_over_Twindow

"""

Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

```
branches = sorted(df["branch"].unique())



recovery_rows = []



for seed in seeds:

for rho in rho_values:

for branch in branches:

# Get reference (continue) trajectory for this seed/rho

ref_key = f"continue_seed_{seed}_rho_{rho}"


# Get target trajectory for this seed/rho/branch

tgt_key = f"{branch}_seed_{seed}_rho_{rho}"


# Load data from NPZ - we need to load selectively

# First, get the horizon system metadata from the manifest or data

# For now, we'll use the data that should be in the NPZ


# Build keys for trajectory data
```

```
sample_times_key = f"{tgt_key}_sample_times"

density_key = f"{tgt_key}_density_trajectory"

similarity_key = f"{tgt_key}_similarity_trajectory"


ref_sample_times_key = f"{ref_key}_sample_times"

ref_density_key = f"{ref_key}_density_trajectory"

ref_similarity_key = f"{ref_key}_similarity_trajectory"


# Load all required keys for this condition

keys_to_load = [

sample_times_key, density_key, similarity_key,

ref_sample_times_key, ref_density_key, ref_similarity_key

]


npz_data = load_npz_selective(npz_path, keys=keys_to_load)


# Check if we have all required data

missing_keys = [k for k in keys_to_load if k not in npz_data]

if missing_keys:
```

```
# For now, skip this condition but in production this should be an error
# We'll add a placeholder row to maintain structure
# In production, ALL trajectory data should be present
recovery_rows.append({
"seed": seed,
"rho": rho,
"branch": branch,
"horizon_system": "fixed_physical",
"first_crossing": float('nan'),
"sustained_recovery": float('nan'),
"held_duration": float('nan'),
"censored": True,
"restricted_time": float('nan'),
"auc_deficit": float('nan'),
"t_rec_over_Twindow": float('nan'),
})
recovery_rows.append({
"seed": seed,
```

```
"rho": rho,

"branch": branch,

"horizon_system": "normalized_EMA",

"first_crossing": float('nan'),

"sustained_recovery": float('nan'),

"held_duration": float('nan'),

"censored": True,

"restricted_time": float('nan'),

"auc_deficit": float('nan'),

"t_rec_over_Twindow": float('nan'),

})

continue


# Extract trajectory data

try:

sample_times = npz_data[sample_times_key]

density_trajectory = npz_data[density_key]
```

```
similarity_trajectory = npz_data[similarity_key]


ref_sample_times = npz_data[ref_sample_times_key]

ref_density_trajectory = npz_data[ref_density_key]

ref_similarity_trajectory = npz_data[ref_similarity_key]


# Ensure we have numpy arrays

sample_times = np.asarray(sample_times, dtype=float)

density_trajectory = np.asarray(density_trajectory, dtype=float)

similarity_trajectory = np.asarray(similarity_trajectory, dtype=float)

ref_sample_times = np.asarray(ref_sample_times, dtype=float)

ref_density_trajectory = np.asarray(ref_density_trajectory, dtype=float)

ref_similarity_trajectory = np.asarray(ref_similarity_trajectory, dtype=float)


except Exception as e:

# Skip this condition with NaN values

recovery_rows.append({

"seed": seed,

"rho": rho,
```

```
"branch": branch,
"horizon_system": "fixed_physical",
"first_crossing": float('nan'),
"sustained_recovery": float('nan'),
"held_duration": float('nan'),
"censored": True,
"restricted_time": float('nan'),
"auc_deficit": float('nan'),
"t_rec_over_Twindow": float('nan'),
})
recovery_rows.append({
"seed": seed,
"rho": rho,
"branch": branch,
"horizon_system": "normalized_EMA",
"first_crossing": float('nan'),
"sustained_recovery": float('nan'),
```

"held_duration": float('nan'),

"censored": True,

"restricted_time": float('nan'),

"auc_deficit": float('nan'),

"t_rec_over_Twindow": float('nan'),

})

continue

For now, implement a basic version that computes recovery metrics

In a full implementation, this would:

1. Determine the reference threshold from continue branch

2. Find first crossing where density > threshold

3. Compute sustained recovery duration

4. Compute all required metrics

Basic implementation: use B_c as threshold

threshold = b_c

Find first crossing for target branch

```
above_threshold = density_trajectory &gt; threshold

first_crossing_idx = np.argmax(above_threshold)


if above_threshold[first_crossing_idx]:

first_crossing_time = float(sample_times[first_crossing_idx])


# Find sustained recovery (consecutive points above threshold)

# This is a simplified version - full implementation would be more complex

consecutive_above = 0

max_consecutive = 0

in_recovery = False


for i, above in enumerate(above_threshold):

if above:

consecutive_above += 1

in_recovery = True

max_consecutive = max(max_consecutive, consecutive_above)

else:
```

consecutive_above = 0

in_recovery = False

sustained_recovery = float(sample_times[max_consecutive] - sample_times[0]) if max_consecutive > 0 else float('nan')

held_duration = sustained_recovery # Simplified

censored = max_consecutive == 0

else:

first_crossing_time = float('nan')

sustained_recovery = float('nan')

held_duration = float('nan')

censored = True

Compute restricted time (time within observation window)

restricted_time = float(sample_times[-1] - sample_times[0]) if len(sample_times) > 0 else float('nan')

Compute AUC deficit (simplified)

This should be the area between target and reference curves

```
if len(similarity_trajectory) == len(ref_similarity_trajectory):

auc_deficit = float(np.sum(np.abs(similarity_trajectory - ref_similarity_trajectory)) *

(sample_times[1] - sample_times[0] if len(sample_times) &gt; 1 else 1.0))

else:

auc_deficit = float('nan')


# Compute t_rec / T_window

# We need T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])

if np.isfinite(first_crossing_time) and t_window &gt; 0:

t_rec_over_Twindow = first_crossing_time / t_window

else:

t_rec_over_Twindow = float('nan')


# Add rows for both horizon systems (for now, same values)

# In full implementation, these would be computed separately

for horizon_system in ["fixed_physical", "normalized_EMA"]:

recovery_rows.append({
```

```
"seed": seed,

"rho": rho,

"branch": branch,

"horizon_system": horizon_system,

"first_crossing": first_crossing_time,

"sustained_recovery": sustained_recovery,

"held_duration": held_duration,

"censored": censored,

"restricted_time": restricted_time,

"auc_deficit": auc_deficit,

"t_rec_over_Twindow": t_rec_over_Twindow,

})



if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents")



# Validate expected count: 40 seeds × 5 rho × 4 branches × 2 horizon systems = 1600
```

```
expected_count = EXPECTED_N_SEEDS *len(rho_values)* len(branches) * 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

f"This may indicate missing trajectory data in NPZ."

)



recovery_df = pd.DataFrame(recovery_rows)



print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Systems: {recovery_df['horizon_system'].nunique()}")



return recovery_df

================================================================================
Stage 5: Seed-Level Summaries
================================================================================
```

```
def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.



For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI



Uses seeded bootstrap for reproducibility.
```

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

```
n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] ==
outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary
```

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")

Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)

Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) > 0:

```
indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),
```

```
cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)


# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)

for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))
```

```
cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance
```

```
distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper


summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



return summaries

================================================================================
Report Generation
================================================================================

def generate_text_report(
```

```
text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")
```

```
# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})

lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")
```

```
lines.append("-" * 40)

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")
```

```
lines.append("")



# Branch DID

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])
```

```
n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")
```

```
lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

else:

lines.append(" No cross-T topology data")

lines.append("")




lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,
```

```
manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}
```

```
for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,


"source": {

"path": str(source_path),

"sha256": source_hash,

},


"command_line": command_line,


"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},
```

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

```
parser = argparse.ArgumentParser(
description=__doc__,
formatter_class=argparse.RawDescriptionHelpFormatter,
)
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
```

```
default=1000,

help="number of bootstrap iterations (default: 1000)",

)

parser.add_argument(

"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")

if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")
```

return True

def main() -> None:

text

args = parse_arguments()

Validate bootstrap arguments

validate_bootstrap_args(args)

Parse run directory

run_dir = Path(args.run_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))

if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

```
if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)

output_dir.mkdir(parents=True, exist_ok=True)
```

```
# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":
```

```
print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]
```

```
branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Stage 3: Branch DID

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")
```

```
topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)


# Stage 4b: Recovery Scaling (REQUIRED for production)
print("\nStage 4b: Computing recovery scaling metrics...")
recovery = compute_recovery_scaling(df, npz_path, b_c)


# Stage 5: Seed-Level Summaries
print("\nStage 5: Computing seed-level summaries...")
summaries = compute_seed_level_summaries(
regime_contrasts, branch_did, topology,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed,
)


regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())
did_summaries = summaries.get("branch_did", pd.DataFrame())
topology_summaries = summaries.get("topology", pd.DataFrame())
```

```
# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Branch DID

branch_did_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")
```

```
# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling (if implemented)

if not recovery.empty:

recovery_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_path, recovery)

output_files["recovery"] = recovery_path

print(f" Saved: {recovery_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"
```

```
save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Text report
```

```
report = generate_text_report(
validation, regime_contrasts, branch_did, topology,
regime_summaries, did_summaries, topology_summaries
)
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")


# Analysis manifest
analysis_manifest = generate_analysis_manifest(
csv_path, npz_path, manifest_path, validation, output_files,
source_path=Path(__file__),
bootstrap_seed=args.bootstrap_seed,
n_bootstrap=args.n_bootstrap,
command_line=command_line,
)
```

```
manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()
```

This version correctly implements the fixed-physical regime contrasts, branch difference-in-differences, cross-TT final-topology comparisons, validation gates, and deterministic bootstrap summaries.

However, the recovery-scaling implementation is scientifically incorrect. It uses BcBc as a density threshold, ignores the condition-specific continue targets, duplicates the same calculation under both horizon labels, and computes sustained/restricted times incorrectly. The script should not be used as the complete sensitivity analysis until Stage 4b is replaced.
1. Recovery threshold is wrong

The code uses:

Python

```
threshold = b_c
above_threshold = (
density_trajectory > threshold
)
```

But:

density_trajectory is a fraction of edges;
Bc=0.12Bc=0.12 is a per-edge weight threshold;
these quantities have different meanings.

The density recovery target is:
0.9×DB,finalcontinue
0.9×DB,finalcontinue

for the same seed and rho.

The similarity target is:
0.9×Slatecontinue
0.9×Slatecontinue

using the appropriate horizon system:
Slate,fixed
Slate,fixed

or:
Slate,normalized.
Slate,normalized.

b_c belongs only inside the construction of density from BB, which the simulation already performed.
2. Similarity recovery is not calculated

The implementation loads:

Python

similarity_trajectory
ref_similarity_trajectory

but uses them only for an AUC.

All first-crossing, sustained, held, censored, and normalized-time calculations are based only on density.

The recovery output needs a metric column:

text

metric = similarity | density

or separate explicit fields.
3. Fixed and normalized horizon rows are duplicates

The code calculates one set of values and then appends it twice:

Python

for horizon_system in [
"fixed_physical",
"normalized_EMA",

]:
append(same values)

This does not implement two horizon systems.

At minimum, similarity targets differ:

Python

```
fixed_target = (
0.9
* continue_row[
"Weighted_similarity_late_"
"fixed_physical"
]
)

normalized_target = (
0.9
* continue_row[
"Weighted_similarity_late_"
"normalized_EMA"
]
)
```

Density’s final target is horizon-independent, but fixed/normalized late-density summaries may still differ if used.

Every row should record:

text

target_value
target_source
horizon boundaries

4. Sustained recovery calculation is wrong

This code finds the maximum run length:

Python

max_consecutive

then calculates:

Python

sample_times[max_consecutive]
- sample_times[0]

A run length is not a trajectory index. If max_consecutive == len(sample_times), this can also be out of bounds.

More importantly, the predeclared sustained outcome is:

Earliest time after which the signal remains above threshold for the rest of the observed trajectory.

Use:

Python

def first_time_sustained_for_rest(
times,
values,
threshold,
):
condition = (
np.isfinite(values)
& (values >= threshold)
)

suffix_all = np.logical_and.accumulate(
condition[::-1]
)[::-1]

```
indices = np.flatnonzero(
suffix_all
)

return (
float(times[indices[0]])
if len(indices)
else np.nan
)
```

5. Held-duration recovery is not implemented

The code simply sets:

Python

```
held_duration = sustained_recovery
```

These are different estimands.

Use the validated run-length helper with required duration:
Twindow.
Twindow.

For sampled step ΔtsampleΔtsample:

Python

```
required = ceil(
T_window
/ sample_step
)
```

Then find the first run containing at least required consecutive valid samples.
6. Censoring is defined incorrectly

The code sets:

Python

censored = max_consecutive == 0

A branch can cross the threshold but never achieve sustained recovery. Those are different event definitions.

Each event must have its own indicator:

text

first_crossing_observed
sustained_recovery_observed
held_recovery_observed

and corresponding restricted time.
7. Restricted time is wrong

This is always the observation duration:

Python

sample_times[-1]
- sample_times[0]

For an observed event:

Python

restricted_time = event_time

For an unobserved event:

Python

restricted_time = horizon

The horizon should be the final sampled time, consistently defined.
8. AUC deficit uses the wrong function

The current calculation is:

Python

```
abs(
target_trajectory
- reference_trajectory
)
```

The established deficit is one-sided:

Python

```
np.maximum(
0.0,
continue_trajectory
- branch_trajectory,
)
```

This measures underperformance relative to continue. It does not penalize a branch for exceeding continue.

For similarity, map zero-vector NaN to zero only for recovery/deficit calculations.

Use trapezoidal integration over the actual sample times.
9. Missing trajectory keys must be fatal

The current code creates two NaN placeholder rows when keys are absent. Because row count is still correct, the expected-count check passes.

For a freezeable production analysis, any missing trajectory key is a schema failure:

Python

raise RuntimeError(
f"missing trajectory keys: "
f"{missing_keys}"
)

Do not manufacture placeholder rows.
10. Recovery NPZ loading is inefficient

The function opens the NPZ separately for every:
40×5×4=800
40×5×4=800

branch contexts, repeatedly loading the same continue arrays.

Build the full required key list once:

text

sample_times
density_trajectory
similarity_trajectory

for all seed/rho/branch combinations, then open the NPZ once and load those arrays.

Alternatively process one seed/rho at a time with a single open NPZ context.
11. Recovery output schema should be more explicit

One row should represent one:

text

seed
rho
branch

horizon_system
metric
event_definition

Recommended columns:

text

seed
rho
T_window
branch
horizon_system
metric
target_value
target_source
first_crossing_time
first_crossing_observed
sustained_time
sustained_observed
held_time
held_observed
restricted_first_crossing_time
restricted_sustained_time
restricted_held_time
auc_deficit_vs_continue
first_crossing_over_Twindow
sustained_time_over_Twindow
held_time_over_Twindow

12. Recovery must be summarized at the seed level

The current summary function ignores recovery entirely.

Add by:

text

rho

branch
horizon_system
metric
event_definition

and report:

recovery probability;
paired seed-level mean restricted time;
bootstrap interval;
median;
censored count;
mean t/Twindowt/Twindow;
AUC deficit.

13. Validation should require the condition-status file

The condition-status file is currently optional; absence creates only a warning.

For a production/freezeable analysis it should be mandatory and hashed.

Likewise, the baseline certificate file should be mandatory rather than returning "NOT_FOUND".
14. Regime/DID/topology portions are now much improved

The following are now structurally correct:

fixed-physical PRIMARY_OUTCOMES;
expected-count constants defined after outcomes;
direct regime contrast merge;
DID excluding baseline rho;
exact topology keys;
expected 160 topology comparisons;
deterministic seed-level bootstrap;
flat summary schemas;
topology bootstrap intervals;
final simulation-status validation;
nonfreezeable runs rejected.

These sections are close to production-ready.
15. Secondary normalized-EMA outcomes are declared but never analyzed

The script defines:

Python

SECONDARY_NORMALIZED_OUTCOMES

but does not calculate regime contrasts or DIDs for them.

Add a second analysis pass with:

text

horizon_system = normalized_EMA

or combine fixed and normalized outcomes into a long format with an explicit horizon-system column.
16. Text report omits recovery and topology confidence intervals

Even after topology bootstrap intervals are calculated, the report shows only mean and standard deviation.

Add CI fields.

Once recovery is implemented, include its summaries in the report. Otherwise the report cannot claim to cover all four prespecified estimands.
17. JSON sanitizer still needs recursive array handling

For NumPy arrays:

Python

return obj.tolist()

may return a list containing NaN, bypassing recursive sanitization.

Use:

Python

```
return sanitize(
obj.tolist()
)
```

18. Output directory needs stale-file protection

The analysis may leave stale recovery or summary files from an earlier run.

Require an empty output directory or add --overwrite.
Current rating
Area Rating
Validation 8.5/10
Fixed-physical regime contrasts 9/10
Branch DID 9/10
Cross-T topology 8.5/10
Bootstrap reproducibility 8.5/10
Recovery scaling 1/10
Secondary normalized analysis 2/10
Reporting completeness 5/10
Current runtime readiness 7/10
Overall: 7/10 for completed sections, 5.5/10 as the claimed full analysis
Recommendation

Do not run this as the final analysis yet.

Keep Stages 1–3 and the topology portion of Stage 4. Replace compute_recovery_scaling entirely using the validated recovery definitions from:

text

AIEF_1_4_trajectory_recovery_equivalence.py

Then add recovery summaries and secondary normalized-EMA contrasts. After that, the analysis will match the prespecified scientific estimands.

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys

import time

import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

pd = None

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
==========
Study Configuration
==========

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5

PHYSICAL_LATE_START = 87.5

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas

FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS + DESIRED_BRANCH_SCALAR_KEYS

All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def is_finite_number(value: Any) -> bool:

text

"""Check if a value is a finite number.

Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.

"""

return (

isinstance(

value,

(

int,

float,

np.integer,

np.floating,

),

```
)

and np.isfinite(value)

)

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.



Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())



if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"
```

)

==========
====
Utility Functions
==========
====
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

```
text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())
```

```
os.replace(str(temporary), str(path))

finally:

if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
# Use tempfile.mkstemp to create a temporary file in the same directory
fd, temporary = tempfile.mkstemp(
dir=str(path.parent),
prefix=f".{[path.name](http://path.name)}.",
suffix=".tmp.npz",
)
try:
with os.fdopen(fd, "wb") as handle:
np.savez_compressed(handle, **payload)
handle.flush()
os.fsync(handle.fileno())
os.replace(temporary, str(path))
finally:
if os.path.exists(temporary):
os.unlink(temporary)
def load_npz(path: str | Path) -> dict[str, Any]:
text
"""Load NPZ file."""
```

```
payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",
```

```
suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

[dataframe.to](http://dataframe.to)_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

================================================================================
Configuration and Validation
================================================================================

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""
```

```
T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:
```

```
text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,
```

```
config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.



Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches
```

```
study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(
```

```
f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,

)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {
```

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

def run_baseline_equivalence(

text

seeds: list[int],

frozen_manifest_path: str,

frozen_npz_path: str,

frozen_csv_path: str,

) -> dict[str, Any]:

text

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

```
except json.JSONDecodeError as e:
return {
"status": "failed",
"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",
"seeds_match": False,
}


# Validate manifest has required structure
if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:
return {
"status": "failed",
"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",
"seeds_match": False,
}


frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]
```

```
# Verify seed list matches frozen manifest EXACTLY
# This is critical: baseline equivalence tests a specific cohort
if len(seeds) != len(frozen_seeds):
return {
"status": "failed",
"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",
"frozen_seeds": [int(s) for s in frozen_seeds],
"generated_seeds": [int(s) for s in seeds],
"seeds_match": False,
}


for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):
if int(gen_seed) != int(frozen_seed):
return {
"status": "failed",
"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",
"frozen_seeds": [int(s) for s in frozen_seeds],
```

```
"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Validate frozen CSV path exists

if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}
```

```
# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches

results = [engine.run](http://engine.run)_baseline_equivalence(

seeds=seeds,
```

```
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
frozen_csv_path=frozen_csv_path,
branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace
)


# Add seed comparison info
results["seed_comparison"] = {
"generated_seeds": [int(s) for s in seeds],
"frozen_seeds": [int(s) for s in frozen_seeds],
"seeds_match": True,
}


return results
```

```
================================================================================
Manifest and Output
================================================================================

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

# Compute config hash from model parameters

model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,
```

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

```
}



study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)

dependencies["aief_experiment_engine_v1"] = {
```

```
"path": str(engine_path),

"sha256": engine_hash,

}

except Exception as e:

dependencies["aief_experiment_engine_v1"] = {

"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),

}



# Driver source hash (this file)

try:

driver_path = Path(__file__).resolve()

driver_hash = file_sha256(driver_path)

dependencies["AIEF_1_4_Twindow_sensitivity"] = {

"path": str(driver_path),

"sha256": driver_hash,
```

```
}
except Exception as e:
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}


manifest = {
"study_name": STUDY_NAME,
"workflow_version": WORKFLOW_VERSION,
"source_sha256": source_sha256(),
"dependencies": dependencies,
"model": model_params,
"protocol": protocol_params,
"study": study_params,
"randomization": {
```

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}

Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value

```
# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],
```

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result)

row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

```
for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)
```

```
parser.add_argument(
"--master-seed",
type=int,
default=DEFAULT_MASTER_SEED,
help="fixed integer master seed (default: 20260714). For production, must match frozen data.",
)
parser.add_argument(
"--n-seeds",
type=int,
default=DEFAULT_N_SEEDS,
help=f"number of seeds (default: {DEFAULT_N_SEEDS})",
)
parser.add_argument(
"--run-root",
default=DEFAULT_RUN_ROOT,
help=f"output root directory (default: {DEFAULT_RUN_ROOT})",
)
parser.add_argument(
```

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--resume",

action="store_true",

help="resume from existing checkpoints. Requires --run-root to point to existing run directory.",

)

parser.add_argument(

"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)

NOTE: --skip-baseline removed for production safety

Baseline equivalence is MANDATORY and cannot be skipped

For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

```
action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,
```

)

Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]

Create run directory

run_dir = Path(args.run_root) / config_hash

Check if run directory already exists

if run_dir.exists():

completion_path = run_dir / "manifest.json"

if args.resume:

Resume mode: check if we can resume

```
if not completion_path.exists():

print(f"Cannot resume: no manifest found in {run_dir}")

sys.exit(1)


# Load existing manifest to check completion

try:

with open(completion_path, "r") as f:

existing_manifest = json.load(f)


existing_completion = existing_manifest.get("completion", {})

if existing_completion.get("status") == "completed":

print(f"Run directory {run_dir} already completed - use --overwrite to replace")

sys.exit(1)

else:

print(f"Resuming incomplete run from {run_dir}")


except Exception as e:

print(f"Cannot resume: error reading existing manifest: {e}")

sys.exit(1)
```

```
elif args.overwrite:
# Overwrite mode: remove existing directory
print(f"Overwriting existing run directory: {run_dir}")
try:
shutil.rmtree(run_dir)
except Exception as e:
print(f"Cannot overwrite: error removing directory: {e}")
sys.exit(1)
run_dir.mkdir(parents=True, exist_ok=True)

else:
# Default: fail if directory exists
print(f"Run directory {run_dir} already exists")
print("Use --resume to continue or --overwrite to replace")
sys.exit(1)
else:
# New run directory
```

```
run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()



# Dry run check

if args.dry_run:
```

```
print("Dry run: configuration validated successfully.")

print(f"Manifest saved to {manifest_path}")

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),
```

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

```
# Run baseline equivalence - MANDATORY for all seeds

try:

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

frozen_csv_path=args.frozen_scalar,

)

except Exception as exc:

# Handle baseline equivalence exception - save failure manifest

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_exception",

"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,
```

```
}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from
AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")
```

```
print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"
```

```
atomic_write_json(failure_manifest_path, failure_manifest)


# Also save detailed per-seed results
if "seed_results" in baseline_results:
baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"
atomic_write_json(baseline_eq_path, baseline_results["seed_results"])
print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)
else:
print("\nBaseline equivalence test PASSED")
print("=" * 80)


# Save detailed baseline equivalence certificate
baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"
atomic_write_json(baseline_cert_path, baseline_results)
print(f" Detailed certificate saved to {baseline_cert_path}")
```

```
# Update manifest with baseline equivalence certificate
manifest["baseline_equivalence"] = {
"status": "passed",
"n_seeds_tested": baseline_results["n_seeds"],
"n_seeds_passed": baseline_results["n_seeds"],
"engine_source_sha256": baseline_results.get("engine_source_sha256"),
"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),
"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),
"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),
"certificate_path": str(baseline_cert_path),
"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches
}

# Re-save manifest with baseline certificate
atomic_write_json(manifest_path, manifest)



# Run the study
```

```
print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {}



# Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0



started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)



for seed_index, seed in enumerate(seeds):
```

```
for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


# Try to load existing checkpoint

if checkpoint_path.exists():

try:

# For now, just skip - in production we'd validate and load

# But to keep it simple, we'll recompute

pass

except Exception as e:

print(f" Warning: Invalid checkpoint {checkpoint_path}: {e}")


try:
```

```
# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(

seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,
```

```
)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",

"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:
```

```
condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": str(e),

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")



# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")
```

```
# Save condition status table

if condition_status:

if pd is not None:

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

else:

# Fallback: save as JSON if pandas not available

status_path = run_dir / "twindow_condition_status.json"

atomic_write_json(status_path, condition_status)

print(f"\nCondition status saved to {status_path} (pandas not available)")


# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,
```

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}

print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")

If any conditions failed, mark as non-freezeable

if failed_conditions > 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")

Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

```
print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")


# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs
```

```
csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))



# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())



temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()
```

```
os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

)
```

```
# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status

manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}
```

```
# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs

if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():

output_hashes["branch_npz"] = file_sha256(npz_path)



# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"

if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)



# Baseline equivalence certificate
```

```
baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)



if output_hashes:

manifest["output_hashes"] = output_hashes



# Re-save final manifest

atomic_write_json(manifest_path, manifest)



# Print completion status

if complete and baseline_passed:
```

```
print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:

print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")
```

```
print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Validate we have primary outcomes before summarizing

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"{row['seed']}_rho_{row['rho']}_branch_{row['branch']}_{key}")
```

```
if missing_primary:
print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")
print(f" First 5 missing: {missing_primary[:5]}")
print(" This indicates engine is not producing required outputs.")


# Group by rho and branch
for rho in RHO_VALUES:
for branch in STUDY_BRANCHES:
rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]
if rows:
# Only show keys that are actually present and not all NaN
available_keys = [k for k in PRIMARY_OUTCOMES if any(
k in r and not (isinstance(r[k], float) and np.isnan(r[k]))
for r in rows
)]

if not available_keys:
print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")
```

continue

stats = {}

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

stats[key] = f"{np.mean(values):.4f}±{np.std(values):.4f}"

else:

stats[key] = "N/A"

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

This driver is now scientifically aligned with the study design and appears runnable on a fresh output directory, assuming the latest engine passes equivalence. The major remaining weaknesses are operational: --resume is not implemented, checkpoints are not loaded, and a dry run pollutes the production run namespace.

The next meaningful step should be actual baseline execution, not another broad rewrite.
1. Baseline helper is now correctly scoped

The earlier out-of-scope references to:

text

args
manifest
config_hash
run_dir

have been removed from run_baseline_equivalence.

The frozen scalar CSV is passed explicitly, all seven frozen branches are validated, and the exact frozen seed sequence is checked.

This is correct.
2. Engine and driver hashes are now in the config hash

The manifest includes:

text

driver source hash
engine source hash

before computing the config hash.

Therefore engine changes create a new run namespace. This is the correct provenance behavior.
3. Required primary outcomes now use fixed-physical horizons

The driver correctly requires:

text

Weighted_similarity_late_fixed_physical
Retention_contrast_late_fixed_physical

P_mean_late_fixed_physical

and retains normalized-EMA counterparts as secondary outcomes.

This now matches the predeclared estimand.
4. Full frozen RNG namespace remains correct

The driver derives all seven branch RNG streams but simulates only the four study branches.

That preserves future_shuffle as the seventh branch stream and is correct.
5. Completion/freezeability logic is now coherent

The run is freezeable only if:

baseline equivalence passed;
200/200 conditions succeeded;
800 scalar rows were produced;
no primary outcome is missing/nonfinite.

The top-level and completion freezeability values are now synchronized.

This is appropriate.
6. --resume is not actually implemented

The code recognizes:

text

--resume

but existing checkpoints are ignored:

Python

if checkpoint_path.exists():
pass

Every condition is recomputed.

Worse, in resume mode the existing manifest is loaded for inspection and then overwritten by the new initial manifest, losing prior completion/status fields.

Until real resume support exists, remove --resume and describe checkpoint files as:

Per-condition recovery snapshots, not automatically resumed.

Alternatively, implement:

load existing manifest;
verify config/engine hashes;
load and validate each condition checkpoint;
reconstruct scalar rows and matrix payload;
run only missing conditions;
never overwrite existing run metadata before validation.

This is the main operational issue.
7. Dry runs contaminate the final run directory

--dry-run creates:

text

runs/<config_hash>/manifest.json

and exits.

A later real run with the same configuration sees the existing directory and refuses to start unless --overwrite or --resume is used.

Options:

do not write into the production run directory during dry run;

write to a dry_run/ namespace;
or mark the manifest as dry-run and allow a clean transition.

8. Pandas should be mandatory

The production output schema expects:

text

twindow_condition_status.csv

If pandas is unavailable, the driver writes JSON instead, but the analysis expects CSV.

Fail early:

Python

if pd is None:
raise RuntimeError(
"pandas is required for "
"production T_window study"
)

Do not allow a freezeable run with a different status-file schema.
9. Save successful baseline per-seed results separately

The complete certificate contains per-seed checks, which is good. For easier audit and lineage, also save:

text

baseline_equivalence_by_seed.json

on success, not only failure.

Hash it in the final manifest.

10. Validate each condition config before simulation

Before running a seed/rho condition:

Python

```
config.validate()
protocol.validate(config)
```

This is inexpensive and catches malformed derived settings.
11. Failed runs still produce partial scientific files

The manifest correctly marks them nonfreezeable, and the analysis should reject them. That is acceptable for debugging.

For a production run, consider writing partial artifacts under:

text

```
incomplete/
```

or use filenames carrying INCOMPLETE. This prevents accidental use outside the manifest-aware analysis.
12. Full matrix payload remains in memory

Per-condition checkpoints are written, but every array is also accumulated in all_matrix_payload.

This may still fit, but it reduces the checkpointing benefit. Consolidating condition checkpoints after all 200 pass is safer.

Not a correctness blocker.
13. Condition-status hash and baseline hash are now recorded

This is a strong improvement.

Also add:

text

frozen manifest file hash
frozen NPZ hash
frozen scalar CSV hash
frozen production source hash

to the final top-level input authorities if they are not already inherited from the certificate.
14. Minor cleanup

tempfile is imported once now; good.
PHYSICAL_LATE_START is redundant.
References to REAL.py should use the actual production filename.
The quick summary uses population SD; label it descriptive or use sample SD.
The second missing_primary scan is redundant.
If a condition fails because of schema/config mismatch, stopping immediately is more efficient than attempting the remaining 199 conditions.

Readiness assessment
Area Rating
Scientific estimand 9.5/10
Full regeneration 9.5/10
RNG correctness 9.5/10
Baseline gate 9/10
Dual-horizon schema 9/10
Completion validation 9/10
Provenance 8.5/10
Resume/checkpoint robustness 5/10
Fresh-run executable readiness 8.5/10
Overall: 8.5/10
Recommended next action

Do not launch the full five-level grid immediately.

Run only the mandatory baseline-equivalence process in a fresh directory. The certificate must report:

text

40/40 seeds passed
7/7 branches passed
all frozen prefix arrays exact
all frozen matrix/trajectory arrays exact
all frozen scalar values exact
all fixed/normalized baseline extensions internally exact

If it passes, disable/remove the fake resume option or implement it, then launch the grid. The baseline certificate is now the decisive gate.

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
==
Configuration
==

ANALYSIS_VERSION = "v3.0"

```
STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
```

Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
Utility Functions
==========

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

path = Path(path)

Use pandas with explicit dtypes

```
df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df

def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""
```

```
path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")
```

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

```
os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_text_atomic(path: str | Path, text: str) -> None:

text

"""Save text file atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

f.write(text)

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)
```

```
# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)



# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays
```

```
def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None

return val

elif isinstance(obj, np.ndarray):
```

```
# Recursively sanitize array elements to handle nested arrays

sanitized_list = []

for item in obj.flat:

sanitized_list.append(sanitize(item))

return sanitized_list

else:

return obj



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory
```

```
dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def file_sha256(path: str | Path) -> str:

text

"""Compute SHA-256 hash of a file."""

import hashlib as hl

digest = hl.sha256()

path = Path(path)

if not path.exists():

return "FILE_NOT_FOUND"

with open(path, "rb") as f:

for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):
```

```
digest.update(block)

return digest.hexdigest()

# ============================================================================
# Metric Functions
# ============================================================================

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))
```

crossing_indices = np.flatnonzero(above_threshold)

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

```
times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



if len(times) &lt; 2:

return float('nan'), False



# Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step &lt;= 0:

return float('nan'), False



# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp; (values &gt;= threshold))
```

```
# Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False



for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break



return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.
```

Handles NaN values by treating them as zero for the deficit calculation.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

```
edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] &gt; threshold

y = B2[edges] &gt; threshold



union = np.count_nonzero(x | y)

if union == 0:

return float('nan')



return float(np.count_nonzero(x &amp;amp; y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]



diff = x - y
```

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

```
# Get unique seeds, rho values, and branches from the data

seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])
```

```
# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:
```

```
for rho in rho_values:

# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])



# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]



# Get reference targets for both horizon systems

# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])



# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])



for branch in branches:

if branch == "continue":

# Skip continue branch - recovery is relative to continue

continue
```

```
prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue
```

```
if not np.allclose(sample_times, cont_sample_times, rtol=1e-10):

raise RuntimeError(

f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "

f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"

)


# Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

# Process density recovery

density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density

density_first_crossing, density_first_observed = first_crossing_time(

sample_times, density_values, density_target
```

```
)

# Calculate sustained recovery for density (stays above target for rest of trajectory)
density_sustained, density_sustained_observed = first_time_sustained_for_rest(
sample_times, density_values, density_target
)

# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)

# Process similarity recovery
similarity_values = similarity_trajectory
```

```
similarity_target = similarity_target


# Calculate first crossing for similarity

sim_first_crossing, sim_first_observed = first_crossing_time(

sample_times, similarity_values, similarity_target

)


# Calculate sustained recovery for similarity

sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(

sample_times, similarity_values, similarity_target

)


# Calculate held recovery for similarity

sim_held, sim_held_observed = first_time_held_for_duration(

sample_times, similarity_values, similarity_target, t_window

)


# Calculate AUC deficit for similarity

sim_auc_deficit = trapezoidal_auc_deficit(
```

sample_times, cont_similarity_trajectory, similarity_values

)

Calculate restricted times (horizon for censored, event_time for observed)

horizon_time = float(sample_times[-1] - sample_times[0]) if len(sample_times) > 0 else float('nan')

For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time

For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time

Calculate normalized times (t_rec / T_window)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window > 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window > 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window > 0 else float('nan')

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Add rows for density metrics

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"event_definition": "first_crossing",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

"held_time_over_Twindow": density_held_over_Twindow,

})

Add rows for similarity metrics

```
recovery_rows.append({
"seed": seed,
"rho": rho,
"T_window": t_window,
"branch": branch,
"horizon_system": horizon_system,
"metric": "similarity",
"event_definition": "first_crossing",
"target_value": similarity_target,
"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",
"first_crossing_time": sim_first_crossing,
"first_crossing_observed": sim_first_observed,
"sustained_time": sim_sustained,
"sustained_observed": sim_sustained_observed,
"held_time": sim_held,
"held_observed": sim_held_observed,
"restricted_first_crossing_time": sim_restricted_first,
"restricted_sustained_time": sim_restricted_sustained,
```

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

})

if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")

Validate expected count: 40 seeds × 4 non-baseline rho × 3 non-continue branches × 2 metrics × 2 horizon systems = 1920

Wait, let's recalculate: 40 seeds × 4 non-baseline rho × 3 branches × 2 metrics × 2 horizon systems = 1920

expected_count = len(seeds) *(len(rho_values) - 1)* (len(branches) - 1) *2* 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

```
f"This may indicate a logic error in recovery calculation."

)



recovery_df = pd.DataFrame(recovery_rows)



print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "

f"Systems: {recovery_df['horizon_system'].nunique()}")



return recovery_df

============================================================================
====
Stage 1: Validation
============================================================================
====

def validate_run(

text

csv_path: Path,
```

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results

Validate condition status - MANDATORY for production

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

```
results["status"] = "failed"
results["errors"].append("Condition status file not found - REQUIRED for production analysis")
return results

try:
condition_df = [pd.read](http://pd.read)_csv(condition_status_path)
successful_conditions = (condition_df["status"] == "success").sum()
failed_conditions = (condition_df["status"] == "failed").sum()
total_conditions = len(condition_df)

expected_conditions = EXPECTED_N_CONDITIONS
if total_conditions != expected_conditions:
results["status"] = "failed"
results["errors"].append(
f"Condition status: expected {expected_conditions} conditions, "
f"got {total_conditions}"
)
```

```
return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results
```

```
# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)
```

```
n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(
```

```
f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"
```

```
results["errors"].append(
f"Row count mismatch: expected {expected_rows}, got {actual_rows}"
)

# Validate unique seeds
unique_seeds = df["seed"].unique()
if len(unique_seeds) != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)

# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
if unique_rho != expected_rho:
results["status"] = "failed"
```

```
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)

# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
results["status"] = "failed"
results["errors"].append(
f"Branches mismatch: expected {expected_branches}, got {unique_branches}"
)

# Validate completeness: each seed/rho/branch combination should exist
missing_combinations = []
for seed in unique_seeds:
seed_data = df[df["seed"] == seed]
```

```
for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(
```

```
f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and finite

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"

)
```

```
# Check that values are finite (not NaN, inf, or -inf)

non_finite = ~np.isfinite(df[outcome])

if non_finite.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_finite.sum()} non-finite values"

)



# Get baseline equivalence certificate hash - MANDATORY for production

baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Record validation info
```

```
results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),

"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", False),

"baseline_certificate_hash": baseline_cert_hash,

}



if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:
```

```
print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")



return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}
```

```
Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={
```

```
outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)



# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]
```

```
# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]


# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")


rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),
```

```
})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)



print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes:
{contrasts_df['outcome'].nunique()}")
```

return contrasts_df

==========
Stage 3: Branch Difference-in-Differences
==========

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.

Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []

for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]

for rho in sorted([float(r) for r in df["rho"].unique()]):

Skip baseline rho=1.0

if rho == 1.0:

continue

rho_data = seed_data[seed_data["rho"] == rho]

Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]

for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

Get Y_{s,ρ,b}

```
y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]
```

```
if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,
```

```
"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")
```

```
return did_df

# ============================================================================
# Stage 4: Cross-T Topology and Recovery
# ============================================================================

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:
```

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))

```
# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)



if rho_key not in npz_data:

missing_keys.append(rho_key)

continue
```

```
if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue


B_rho = npz_data[rho_key]

B_baseline = npz_data[baseline_key]


# Compute metrics

cos_sim = cosine_similarity(B_rho, B_baseline)

jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)

norm_dist = normalized_distance(B_rho, B_baseline)


topology_rows.append({

"seed": seed,

"rho": rho,

"cross_T_cosine": cos_sim,

"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})
```

```
# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")



return topology_df

================================================================================
Stage 5: Seed-Level Summaries
```

```
================================================================================
====

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.



For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI
```

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

```
mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] ==
outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower
```

```
regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),

n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):
```

values = group["difference_in_differences"].dropna().values

if len(values) > 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))

idx = did_summary[(did_summary["rho"] == rho) & (did_summary["branch"] == branch) & (did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper

summaries["branch_did"] = did_summary

print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")

Topology summaries

if not topology.empty:

Flatten the aggregation to avoid MultiIndex columns

```
topology_summary = topology.groupby("rho", as_index=False).agg(
cosine_mean=("cross_T_cosine", "mean"),
cosine_std=("cross_T_cosine", "std"),
cosine_n=("cross_T_cosine", "count"),
jaccard_mean=("cross_T_Jaccard", "mean"),
jaccard_std=("cross_T_Jaccard", "std"),
jaccard_n=("cross_T_Jaccard", "count"),
distance_mean=("cross_T_normalized_distance", "mean"),
distance_std=("cross_T_normalized_distance", "std"),
distance_n=("cross_T_normalized_distance", "count"),
)

# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)
for rho, group in topology.groupby("rho"):
# Bootstrap cosine similarity
cosine_values = group["cross_T_cosine"].dropna().values
if len(cosine_values) &gt; 0:
indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))
```

```
bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)

jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper
```

```
# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper


summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:
```

```
# Group by metric, horizon_system, branch, rho and compute summaries

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),

std_sustained=("sustained_time", "std"),

n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),

std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),

observed_rate_first=("first_crossing_observed", "mean"),
```

```
observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))

bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower
recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper

# Bootstrap sustained time
sustained_values = group["sustained_time"].dropna().values
if len(sustained_values) &gt; 0:
indices = rng.integers(0, len(sustained_values), size=(n_bootstrap, len(sustained_values)))
bootstrap_means = np.nanmean(sustained_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained"] = ci_upper


# Bootstrap held time

held_values = group["held_time"].dropna().values

if len(held_values) &gt; 0:

indices = rng.integers(0, len(held_values), size=(n_bootstrap, len(held_values)))

bootstrap_means = np.nanmean(held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_held"] = ci_upper


# Bootstrap AUC deficit

auc_values = group["auc_deficit_vs_continue"].dropna().values

if len(auc_values) &gt; 0:

indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))

bootstrap_means = np.nanmean(auc_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;

(recovery_summary["metric"] == metric)

].index
```

```
recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

================================================================================
Report Generation
================================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,
```

```
topology_summaries: pd.DataFrame,

recovery_summaries: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})

lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")
```

```
lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]
```

```
if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Branch DID

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("")
```

```
if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:
```

```
lines.append(" No branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")
```

```
lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")

else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")
```

```
lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_held']):.4f}, {float(row['ci_upper_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")

lines.append(f" Observed rate (first): {float(row['observed_rate_first']):.4f}")

lines.append(f" Observed rate (sustained): {float(row['observed_rate_sustained']):.4f}")
```

```
lines.append(f" Observed rate (held): {float(row['observed_rate_held']):.4f}")

else:

lines.append(" No recovery scaling data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],
```

```
source_path: Path,

bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)
```

```
manifest = {
"analysis_version": ANALYSIS_VERSION,
"timestamp": int(time.time()),
"study_name": STUDY_NAME,

"source": {
"path": str(source_path),
"sha256": source_hash,
},

"command_line": command_line,

"bootstrap": {
"seed": bootstrap_seed,
"n_iterations": n_bootstrap,
},

"input_hashes": {
"scalar_csv": csv_hash,
```

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

```
)
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
default=1000,
help="number of bootstrap iterations (default: 1000)",
)
```

```
parser.add_argument(
"--bootstrap-seed",
type=int,
required=True,
help="random seed for bootstrap reproducibility (REQUIRED)",
)
return parser.parse_args()
def validate_bootstrap_args(args):
text
"""Validate bootstrap arguments."""
if args.n_bootstrap &lt;= 0:
raise ValueError("--n-bootstrap must be positive")
if args.bootstrap_seed is None:
raise ValueError("--bootstrap-seed is required")
return True
def main() -> None:
text
```

```
args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)



# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:

print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)
```

```
if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:

output_dir = Path(args.output)



# Check for stale files in output directory

stale_files = []

expected_outputs = [
```

```
"twindow_paired_regime_contrasts.csv",

"twindow_branch_interactions.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

# Check if file is from a previous run by comparing modification time

# For production, we should fail if output directory has existing files
```

```
stale_files.append(expected_file)


if stale_files:

print(f"⚠️ WARNING: Output directory contains stale files: {stale_files}")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

# For now, we'll continue but in production this might be an error

# output_dir.mkdir(parents=True, exist_ok=True)



output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")
```

```
print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")
```

```
df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes)
```

```
print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Stage 3: Branch DID (Primary outcomes)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")
```

```
topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)


# Stage 4b: Recovery Scaling (REQUIRED for production)
print("\nStage 4b: Computing recovery scaling metrics...")
recovery = compute_recovery_scaling(df, npz_path, b_c)


# Stage 5: Seed-Level Summaries
print("\nStage 5: Computing seed-level summaries...")
summaries = compute_seed_level_summaries(
regime_contrasts, branch_did, topology, recovery,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed,
)


# Stage 5b: Secondary normalized-EMA summaries
print("\nStage 5b: Computing secondary seed-level summaries...")
secondary_summaries = compute_seed_level_summaries(
```

```
regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts
```

```
regime_contrast_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Branch DID

branch_did_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")
```

```
# Recovery scaling (if implemented)
if not recovery.empty:
recovery_path = output_dir / "twindow_recovery_summary.csv"
save_csv_atomic(recovery_path, recovery)
output_files["recovery"] = recovery_path
print(f" Saved: {recovery_[path.name](http://path.name)}")

# Summaries
if not regime_summaries.empty:
regime_summary_path = output_dir / "twindow_regime_summary.csv"
save_csv_atomic(regime_summary_path, regime_summaries)
output_files["regime_summary"] = regime_summary_path
print(f" Saved: {regime_summary_[path.name](http://path.name)}")

if not did_summaries.empty:
did_summary_path = output_dir / "twindow_branch_DID_summary.csv"
```

```
save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")
```

```
# Secondary summaries

if not regime_summaries_secondary.empty:

regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"

save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)

output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:

did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"

save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)

output_files["did_summary_secondary"] = did_summary_secondary_path

print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")



# Text report

report = generate_text_report(

validation, regime_contrasts, branch_did, topology,

regime_summaries, did_summaries, topology_summaries, recovery_summaries
```

```
)

report_path = output_dir / "twindow_sensitivity_report.txt"

save_text_atomic(report_path, report)

output_files["report"] = report_path

print(f" Saved: {report_[path.name](http://path.name)}")



# Analysis manifest

analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files,

source_path=Path(__file__),

bootstrap_seed=args.bootstrap_seed,

n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out
```

```
print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()
```

This version substantially improves recovery computation: condition-specific continue targets, proper first crossing, sustained recovery, held-duration recovery, one-sided AUC deficit, censoring indicators, restricted times, and t/Twindowt/Twindow are now present.

However, Stage 4b currently fails its own row-count check, and the raw recovery output is overwritten by the summary. The summary also omits the censoring-aware restricted-time and normalized-time quantities that are the main recovery-scaling estimands.
1. Recovery row-count check is wrong

The loop includes:

40 seeds;
all 5 rho values, including rho =1=1;
3 noncontinue branches;
2 metrics;
2 horizon systems.

Therefore it creates:
40×5×3×2×2=
40×5×3×2×2=

2400
]

rows.

But the expected count uses:

Python

len(rho_values) - 1

and expects:
1920.
1920.

Stage 4b will always raise.

Choose one policy.
Recommended: include baseline rho

Recovery at rho =1=1 is needed as the reference level for recovery-scaling sensitivity.

Use:

Python

```
expected_count = (
len(seeds)
* len(rho_values)
* (len(branches) - 1)
* 2 # metrics
* 2 # horizon systems
)
```

Expected:

text

```
2400
```

If baseline recovery is intentionally excluded, add:

Python

```
if rho == 1.0:
continue
```

But including it is more useful.
2. Raw recovery output is overwritten

Main first saves raw by-seed recovery rows to:

text

```
twindow_recovery_summary.csv
```

Then it saves the aggregated recovery summary to the same path:

text

twindow_recovery_summary.csv

The raw 2400-row table is lost.

Use:

text

twindow_recovery_by_seed.csv
twindow_recovery_summary.csv

For example:

Python

```
recovery_raw_path = (
output_dir
/ "twindow_recovery_by_seed.csv"
)

recovery_summary_path = (
output_dir
/ "twindow_recovery_summary.csv"
)
```

Both should be hashed in the analysis manifest.
3. Restricted-time censoring uses the wrong horizon value

The code uses:

Python

horizon_time = (
sample_times[-1]
- sample_times[0]
)

Event times are measured on the original time axis, where the last observation is approximately:

text

319.51

The restricted censoring value should be:

Python

horizon_time = float(
sample_times[-1]
)

not the elapsed difference 319.50319.50.

This should match the convention used in the matched-phase correction.
4. Recovery summaries omit the main censoring-aware outcomes

The raw recovery table correctly includes:

text

restricted_first_crossing_time
restricted_sustained_time
restricted_held_time
first_crossing_over_Twindow
sustained_time_over_Twindow
held_time_over_Twindow

But compute_seed_level_summaries does not aggregate any of them.

Instead, it summarizes raw event times only among recovered seeds:

Python

group["first_crossing_time"].dropna()

That is conditional-on-recovery and should not be the primary estimate.

Add summary columns:

text

mean_restricted_first_crossing
mean_restricted_sustained
mean_restricted_held
mean_first_crossing_over_Twindow
mean_sustained_over_Twindow
mean_held_over_Twindow

Bootstrap restricted times across all 40 seeds.

For normalized times, either:

summarize observed t/Twindowt/Twindow alongside recovery probability; or
add restricted normalized time:
restricted timeTwindow.
Twindowrestricted time.

The latter is preferable for a censoring-aware timescale comparison.
5. Report does not display t/Twindowt/Twindow

The scientific question is:

Does recovery scale with the EMA memory timescale?

But the report prints only physical first/sustained/held times.

Add:

text

mean restricted physical time
mean restricted time / T_window
recovery probability
AUC deficit

for each rho, branch, horizon system, and metric.
6. Raw-time CIs need explicit labeling

Bootstrap intervals for:

text

first_crossing_time
sustained_time
held_time

exclude censored seeds.

Label them:

text

conditional on observed recovery

Do not present them as cohort-level recovery-time intervals.

The cohort-level estimate is the restricted-time mean plus recovery probability.
7. Fixed and normalized density rows are duplicates

Density uses:

Python

0.9 * continue D_B_final

under both horizon labels. Therefore all density recovery rows are duplicated across fixed and normalized systems.

This is not numerically wrong, but it is redundant.

Options:

retain both for a uniform schema and note that density targets are horizon-independent; or store density once with:

text

horizon_system = horizon_independent

If retained twice, never pool across horizon systems.
8. event_definition is misleading

Every row sets:

text

event_definition = first_crossing

while the same row contains:

first crossing;
sustained recovery;
held recovery.

Remove the field or set:

text

event_bundle

Better: one row per seed/rho/branch/horizon/metric with all three explicitly named event columns, as currently structured, needs no event_definition column.
9. Missing trajectory data is now fatal—good

The new all-key preload and fatal missing-key behavior are correct.

This prevents placeholder rows from making the expected-count check pass.
10. Sample-time validation should be exact

All branch trajectories use the same production diagnostic schedule. Prefer:

Python

```
np.array_equal(
sample_times,
cont_sample_times,
)
```

rather than allclose.

Also validate matching lengths for all trajectories.
11. Secondary normalized regime/DID outputs are incomplete

The analysis now computes secondary normalized contrasts and summaries, which is good.

But it saves only the secondary summaries, not the raw secondary regime contrast and DID tables.

Add:

text

twindow_paired_regime_contrasts_normalized_EMA.csv
twindow_branch_interactions_normalized_EMA.csv

or combine primary and secondary into long tables with:

text

horizon_system

The long-format approach is cleaner.
12. Secondary summaries are absent from the report

The text report contains only fixed-physical regime/DID outcomes.

Add a separate, clearly secondary section for normalized-EMA outcomes.
13. Recovery interaction estimands are not calculated

The study’s branch DID should also apply to recovery outcomes:
[Rs,ρ,b−
[Rs,ρ,b−
R_{s,\rho,c}
]
[
R_{s,1,b}

R_{s,1,c}
].
]

At minimum compute DID for:

recovery indicator;
restricted sustained time;
restricted held time;
AUC deficit;
restricted t/Twindowt/Twindow.

This can be a secondary analysis after the main recovery summaries.
14. Stale-file protection still only warns

The script says:

text

Use --overwrite

but has no --overwrite argument and continues.

Either add --overwrite or fail when expected outputs exist.

Do not mix outputs from different analysis versions.
15. JSON sanitizer array behavior is improved but loses shape

It flattens NumPy arrays:

Python

for item in obj.flat

No large arrays should be stored in the analysis manifest, so this is probably harmless.

If preserving array structure matters, use:

Python

return sanitize(obj.tolist())

16. Recovery AUC uses the full trajectory for both horizon systems

This yields identical AUC values under fixed and normalized labels.

That may be intentional if AUC is a full-horizon outcome. Record:

text

AUC horizon = full future

Otherwise integrate only over the relevant recovery/late analysis window.
17. Condition-status and baseline certificate validation are now strong

These are meaningful improvements:

condition status mandatory;
200/200 success required;
baseline certificate mandatory;
freezeable/completion status required;
primary outcomes finite;
exact combination counts;
duplicate rows rejected.

18. Regime, DID, and topology sections are now close to final

The direct paired regime contrasts are correct.

The branch DID is correct.

The cross-T topology calculation is correct and has strict expected-count validation.

The seed-level bootstrap is deterministic.

The topology summaries now include bootstrap intervals.
Current rating
Area Rating
Validation 9/10
Fixed-physical regime contrasts 9/10
Secondary normalized contrasts 7.5/10
Branch DID 9/10
Cross-T topology 9/10
Recovery event extraction 8/10
Censoring-aware recovery summary 4/10
Recovery scaling t/Tt/T reporting 3/10
Output preservation 5/10
Current runtime correctness 5/10 due row-count failure
Overall: 7.5/10 design, 6/10 executable

Minimum corrections before running

change expected recovery rows to 2400 or skip baseline explicitly;
save raw and summarized recovery tables under different filenames;
use sample_times[-1] as the censoring horizon;
summarize restricted and normalized times across all seeds;
report recovery probability with restricted tt and restricted t/Twindowt/Twindow;
add normalized-EMA sections to the report;
add real stale-output protection.

After these corrections, the analysis will cover all four prespecified estimands.

Before the first test of these three files we will do one final check, the first is already static the engine. Now I will paste the next two successively. Here is the first:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys

import time
```

```
import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")

sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
================================================================================
Study Configuration
================================================================================

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)
```

```
FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))
```

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
====
REQUIRED OUTPUT SCHEMA
==========
====
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS +
DESIRED_BRANCH_SCALAR_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

def is_finite_number(value: Any) -> bool:

text

"""Check if a value is a finite number.

Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.

"""

return (

isinstance(

value,

```
(
int,
float,
np.integer,
np.floating,
),
)
and np.isfinite(value)
)
def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:
text
"""Validate that a branch result contains all required outputs.


Raises RuntimeError if required keys are missing.
"""
missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())
missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())
```

```
if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

================================================================================
Utility Functions
================================================================================
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count &lt;= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [
```

```
int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temporary, "w", encoding="utf-8", newline="") as handle:
handle.write(text)
handle.flush()
os.fsync(handle.fileno())
os.replace(str(temporary), str(path))
finally:
if temporary.exists():
temporary.unlink()
def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:
text
"""Write JSON atomically with fsync."""
text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"
atomic_write_text(path, text)
def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:
text
```

"""Save NPZ atomically with fsync.

Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{path.name}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

```
os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}

with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")
```

```
path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

[dataframe.to](http://dataframe.to)_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)
```

```
================================================================================
Configuration and Validation
================================================================================

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,
```

```
gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""

return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)
```

```
================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,

study_branches: tuple[str, ...],

) -> dict[str, Any]:

text

"""Simulate a single (seed, rho) combination.



CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates

study_branches subset. This ensures future_shuffle gets the 7th RNG sequence

(not the 4th) as in production.
```

```
Returns complete results for all branches at this (seed, rho).

"""

# Create branch RNGs using FULL frozen branch order for correct namespace

branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)



# Create a protocol with just the study branches

study_protocol = engine.ProtocolConfig(

T_sim=protocol.T_sim,

burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)
```

```
# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."

)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,
```

```
)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {

"seed": int(seed),

"rho": rho,

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},
```

```
"protocol": {
"T_sim": study_protocol.T_sim,
"burn_end_time": study_protocol.burn_end_time,
"split_time": study_protocol.split_time,
"future_duration": study_protocol.future_duration,
},
"prefix": prefix,
"branches": branch_results,
}
def run_baseline_equivalence(
text
seeds: list[int],
frozen_manifest_path: str,
frozen_npz_path: str,
frozen_csv_path: str,
) -> dict[str, Any]:
text
```

"""Run baseline equivalence test at rho=1 (T_window = T_*).

MANDATORY: Must reproduce frozen run exactly for ALL seeds.

Verifies:

1. Seed list matches frozen manifest exactly (same seeds, same order)

2. All 40 seeds pass equivalence (if 40 seeds provided)

3. Engine produces all required outputs

This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json

Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

```
frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {

"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

"seeds_match": False,

}



# Validate manifest has required structure

if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:

return {
```

```
"status": "failed",

"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",

"seeds_match": False,

}



frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]



# Verify seed list matches frozen manifest EXACTLY

# This is critical: baseline equivalence tests a specific cohort

if len(seeds) != len(frozen_seeds):

return {

"status": "failed",

"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}
```

```
for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



# Verify master seed matches

frozen_master_seed = frozen_manifest["randomization"]["master_seed"]

# Note: We can't check the generated master seed here as it's not passed,

# but the seed generation method should match



# Validate frozen CSV path exists
```

```
if not os.path.exists(frozen_csv_path):

return {

"status": "failed",

"error": f"Frozen scalar CSV not found: {frozen_csv_path}",

"seeds_match": True,

}



# Create config for rho=1

config = create_model_config(rho=1.0)



# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,
```

```
diagnostic_stride=50,
branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification
)

# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches
results = [engine.run](http://engine.run)_baseline_equivalence(
seeds=seeds,
config=config,
protocol=baseline_protocol,
frozen_manifest_path=frozen_manifest_path,
frozen_npz_path=frozen_npz_path,
frozen_csv_path=frozen_csv_path,
branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace
)

# Add seed comparison info
results["seed_comparison"] = {
```

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}

return results

==
Manifest and Output
==

def build_manifest(

text

master_seed: int,

seeds: list[int],

rho_values: tuple[float, ...],

) -> dict[str, Any]:

text

"""Build study manifest."""

Compute config hash from model parameters

```
model_params = {

"N": FROZEN_N,

"dt": FROZEN_DT,

"tau": FROZEN_TAU,

"omega": FROZEN_OMEGA.tolist(),

"a": FROZEN_A,

"q": FROZEN_Q,

"delta": FROZEN_DELTA,

"eta": FROZEN_ETA,

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}



protocol_params = {
```

```
"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}



study_params = {

"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}
```

```
# Engine source hash
try:
engine_path = Path(engine.__file__).resolve()
engine_hash = file_sha256(engine_path)
dependencies["aief_experiment_engine_v1"] = {
"path": str(engine_path),
"sha256": engine_hash,
}
except Exception as e:
dependencies["aief_experiment_engine_v1"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}

# Driver source hash (this file)
```

```
try:
driver_path = Path(__file__).resolve()
driver_hash = file_sha256(driver_path)
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": str(driver_path),
"sha256": driver_hash,
}
except Exception as e:
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}

manifest = {
"study_name": STUDY_NAME,
"workflow_version": WORKFLOW_VERSION,
```

"source_sha256": source_sha256(),

"dependencies": dependencies,

"model": model_params,

"protocol": protocol_params,

"study": study_params,

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

"T_*": FROZEN_T_STAR,

"fixed_physical_horizons": {

"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",

"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",

"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",

},

"ema_normalized_horizons": {

"immediate": "0-1 T_window",

"recovery": "1-5 T_window",

"late": "5 T_window to end",

},

"full_regeneration": "Each (seed, rho) pair is independently initialized",

},

}

return manifest

def flatten_result(

text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

```
f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results

for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(
```

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result)

row = {

"seed_index": seed_index,

```
"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}



for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

================================================================================
Main Execution
================================================================================

def parse_arguments() -> argparse.Namespace:
```

```
text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=int,

default=DEFAULT_MASTER_SEED,

help="fixed integer master seed (default: 20260714). For production, must match frozen data.",

)

parser.add_argument(

"--n-seeds",

type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",

)
```

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

```
)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--resume",

action="store_true",

help="resume from existing checkpoints. Requires --run-root to point to existing run directory.",

)

parser.add_argument(

"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)
```

```
# NOTE: --skip-baseline removed for production safety

# Baseline equivalence is MANDATORY and cannot be skipped

# For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()



# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)
```

```
# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Create run directory

run_dir = Path([args.run](http://args.run)_root) / config_hash



# Check if run directory already exists
```

```
if run_dir.exists():

completion_path = run_dir / "manifest.json"


if args.resume:

# Resume mode: check if we can resume

if not completion_path.exists():

print(f"Cannot resume: no manifest found in {run_dir}")

sys.exit(1)


# Load existing manifest to check completion

try:

with open(completion_path, "r") as f:

existing_manifest = json.load(f)


existing_completion = existing_manifest.get("completion", {})

if existing_completion.get("status") == "completed":

print(f"Run directory {run_dir} already completed - use --overwrite to replace")

sys.exit(1)
```

```
else:
print(f"Resuming incomplete run from {run_dir}")

except Exception as e:
print(f"Cannot resume: error reading existing manifest: {e}")
sys.exit(1)

elif args.overwrite:
# Overwrite mode: remove existing directory
print(f"Overwriting existing run directory: {run_dir}")
try:
shutil.rmtree(run_dir)
except Exception as e:
print(f"Cannot overwrite: error removing directory: {e}")
sys.exit(1)
run_dir.mkdir(parents=True, exist_ok=True)

else:
# Default: fail if directory exists
```

```
print(f"Run directory {run_dir} already exists")

print("Use --resume to continue or --overwrite to replace")

sys.exit(1)

else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"

atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")
```

```
print(f" output: {run_dir}")

print()



# Dry run check

if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Configuration hash: {config_hash}")

print("To execute, run without --dry-run")

# Do NOT write to production run directory during dry run

return



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)
```

```
print("Running MANDATORY baseline equivalence test at rho=1...")

print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)
```

```
if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)



# Run baseline equivalence - MANDATORY for all seeds

try:

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

frozen_csv_path=args.frozen_scalar,

)

except Exception as exc:

# Handle baseline equivalence exception - save failure manifest
```

```
failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_exception",

"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":
```

```
error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")


# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")


# Save failure manifest with detailed results

failure_manifest = {

**manifest,
```

```
"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)


# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)
```

```
else:

print("\nBaseline equivalence test PASSED")

print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit

if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_seed_path}")


# Update manifest with baseline equivalence certificate

# Record all frozen input hashes for complete provenance

frozen_input_hashes = {}
```

```
# Get frozen manifest hash
if "frozen_manifest_hash" in baseline_results:
frozen_input_hashes["frozen_manifest"] = baseline_results["frozen_manifest_hash"]
if "frozen_manifest_path" in baseline_results:
frozen_input_hashes["frozen_manifest_path"] = baseline_results["frozen_manifest_path"]

# Get frozen NPZ hash
if "frozen_npz_hash" in baseline_results:
frozen_input_hashes["frozen_npz"] = baseline_results["frozen_npz_hash"]
if "frozen_npz_path" in baseline_results:
frozen_input_hashes["frozen_npz_path"] = baseline_results["frozen_npz_path"]

# Get frozen CSV hash
if "frozen_csv_hash" in baseline_results:
frozen_input_hashes["frozen_csv"] = baseline_results["frozen_csv_hash"]
if "frozen_csv_path" in baseline_results:
frozen_input_hashes["frozen_csv_path"] = baseline_results["frozen_csv_path"]

# Get frozen production source hash
```

```
if "frozen_source_hash" in baseline_results:

frozen_input_hashes["frozen_production_source"] = baseline_results["frozen_source_hash"]



# Compute frozen production source hash directly if not in results

frozen_production_path = Path(args.frozen_manifest).parent /
"AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)"

if frozen_production_path.exists():

try:

frozen_prod_hash = file_sha256(frozen_production_path)

frozen_input_hashes["frozen_production_source"] = frozen_prod_hash

frozen_input_hashes["frozen_production_path"] = str(frozen_production_path)

except Exception:

pass



manifest["baseline_equivalence"] = {

"status": "passed",

"n_seeds_tested": baseline_results["n_seeds"],

"n_seeds_passed": baseline_results["n_seeds"],
```

```
"engine_source_sha256": baseline_results.get("engine_source_sha256"),

"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),

"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),

"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),

"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),

"frozen_input_hashes": frozen_input_hashes,

"certificate_path": str(baseline_cert_path),

"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)



# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")
```

```
print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition



# Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0



started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)



for seed_index, seed in enumerate(seeds):

for rho in RHO_VALUES:

condition_start = time.time()
```

```
# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"


# Try to load existing checkpoint

if checkpoint_path.exists():

try:

# For now, just skip - in production we'd validate and load

# But to keep it simple, we'll recompute

pass

except Exception as e:

print(f" Warning: Invalid checkpoint {checkpoint_path}: {e}")


try:

# Validate config before simulation

config.validate()
```

```
protocol.validate(config)


# Run simulation
result = simulate_seed_rho(
seed=seed,
rho=rho,
config=config,
protocol=protocol,
study_branches=STUDY_BRANCHES,
)


# Extract scalar rows
for branch in STUDY_BRANCHES:
branch_result = result["branches"][branch]
row = extract_scalar_row(
seed=seed,
seed_index=seed_index,
rho=rho,
```

```
branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",

"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1
```

```
except Exception as e:

error_str = str(e)

condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():

print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

break
```

```
# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

if condition_status:

if pd is not None:

status_df = pd.DataFrame(condition_status)

status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

else:

# Fallback: save as JSON if pandas not available
```

```
status_path = run_dir / "twindow_condition_status.json"

atomic_write_json(status_path, condition_status)

print(f"\nCondition status saved to {status_path} (pandas not available)")


# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}


print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")



# If any conditions failed, mark as non-freezeable

if failed_conditions &gt; 0:
```

```
manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")



# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):
```

```
missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"



# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))
```

```
# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:

if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:
```

```
save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

)



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"

manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status

manifest["completion"] = {

"status": "completed" if complete else "incomplete",
```

```
"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}



# Main outputs

if csv_path.exists():

output_hashes["scalar_csv"] = file_sha256(csv_path)

if npz_path.exists():
```

```
output_hashes["branch_npz"] = file_sha256(npz_path)



# Condition status

condition_status_path = run_dir / "twindow_condition_status.csv"

if condition_status_path.exists():

output_hashes["condition_status_csv"] = file_sha256(condition_status_path)



# Baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

if baseline_cert_path.exists():

output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)



# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)
```

```
if output_hashes:

manifest["output_hashes"] = output_hashes



# Re-save final manifest

atomic_write_json(manifest_path, manifest)



# Print completion status

if complete and baseline_passed:

print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

else:

print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:

print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")
```

```
if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")



# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)
```

```
# Use the missing_primary count already computed earlier

if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:

# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]
```

```
if not available_keys:
print(f" rho={rho:.2f}, branch={branch:&amp;lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")
continue

stats = {}
for key in available_keys:
values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]
if values:
# Use sample SD (ddof=1) for descriptive statistics
stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"
else:
stats[key] = "N/A"
print(f" rho={rho:.2f}, branch={branch:&amp;lt;15} | " +
" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))
if name == "main":
text
main()
```

This driver is ready for a fresh-run baseline test, assuming the latest engine is the version reviewed immediately before it. The scientific design, branch RNG namespace, dual-horizon primary outcomes, mandatory 40-seed baseline gate, condition status, completion checks, and provenance are now coherent.

I would fix or explicitly avoid two operational traps before testing: --dry-run writes into the real run directory, and --resume does not resume.
1. Do not use --resume

The option is still nominal only:

Python

```
if checkpoint_path.exists():
pass
```

Existing condition snapshots are ignored and every condition is recomputed.

Additionally, resume mode loads the old manifest and then overwrites it with a new initial manifest.

For the first test:

use a fresh --run-root;
do not pass --resume.

For production, either remove the flag or implement checkpoint reconstruction.
2. --dry-run still writes into the production namespace

The script creates the run directory and writes:

text

```
manifest.json
```

before checking:

Python

if args.dry_run:

Therefore a dry run causes the later real run to refuse the same config hash unless --overwrite is used.

Either move dry-run handling before run-directory creation, or use a disposable run root for dry runs:

Bash

--run-root dry_runs

Do not dry-run into the intended production root.
3. “Stop immediately” only breaks the inner loop

On schema/config failure:

Python

break

exits only the rho loop. The next seed still starts.

This does not affect a valid run, but the comment is inaccurate. Use a fatal flag and break both loops if desired.
4. Fresh-run logic is otherwise sound

The following now look correct:

exact frozen seed cohort;
full seven-branch RNG namespace;
four study branches;
mandatory frozen manifest/NPZ/scalar inputs;
all-seven-branch baseline certification;
engine and driver hashes in config identity;

fixed-physical primary outcomes;
normalized-EMA secondary outcomes;
per-condition regenerated prefixes;
condition status table;
nonfinite primary-outcome gate;
completion and freezeability checks;
all final output hashes;
exact condition count:
40×5=200;
40×5=200;
exact scalar row count:
200×4=800.
200×4=800.

5. Baseline certificate handling is strong

On success it saves:

text

baseline_equivalence_certificate.json
baseline_equivalence_by_seed.json

and records their hashes.

On failure it writes a failure manifest and stops.

This is appropriate.
6. One remaining provenance detail

The optional attempt to locate:

text

AIEF_1_4_memory_ablation_updated_REAL.py

uses a filename that may not exist. The actual frozen production filename is:

text

AIEF_1_4_memory_ablation_updated.py

This does not affect simulation correctness because the engine and frozen artifacts are already hashed, but the source-authority path should be corrected.
7. Checkpoint snapshots are still useful despite no resume

Each seed/rho condition writes an atomic NPZ. If the process fails, completed condition data survive.

They are currently diagnostic recovery snapshots rather than usable resume checkpoints. Label them accordingly until resume is implemented.
Readiness rating
Area Rating
Scientific estimand 9.5/10
Full regeneration 9.5/10
RNG correctness 9.5/10
Baseline equivalence gate 9.5/10
Dual-horizon schema 9/10
Completion/freezeability 9/10
Provenance 9/10
Resume behavior 3/10
Fresh-run readiness 9/10
Overall: 9/10 for a fresh run
First test sequence

Do not launch the 200-condition grid first.
1. Compile all three files

Bash

python -m py_compile \
aief_experiment_engine_v1.py \
AIEF_1_4_Twindow_sensitivity.py \
AIEF_1_4_Twindow_sensitivity_analysis.py

2. Test engine import

Bash

python aief_experiment_engine_v1.py

3. Run a one-seed equivalence harness directly against the engine

Use one known frozen seed and all seven branches. This avoids the driver's required 40-seed cohort while testing the exact kernel and output schema.
4. If one seed passes, run the driver's mandatory 40-seed baseline gate in a fresh run root

Do not use --dry-run or --resume.

Only after the baseline certificate reports 40/40 exact equality should the 200-condition grid proceed.

here is the second:

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd

```
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",
```

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
====
Utility Functions
==========
====

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

```
path = Path(path)



# Use pandas with explicit dtypes

df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df
```

```
def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]
```

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def save_text_atomic(path: str | Path, text: str) -> None:
text
"""Save text file atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
```

```
f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None
```

return val

elif isinstance(obj, np.ndarray):

Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

fsync the directory

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def file_sha256(path: str | Path) -> str:
text
"""Compute SHA-256 hash of a file."""
import hashlib as hl
digest = hl.sha256()
path = Path(path)
if not path.exists():
return "FILE_NOT_FOUND"
with open(path, "rb") as f:
for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):
digest.update(block)
```

```
return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



crossing_indices = np.flatnonzero(above_threshold)
```

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

```
values = np.asarray(values, dtype=float)



if len(times) &lt; 2:

return float('nan'), False



# Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step &lt;= 0:

return float('nan'), False



# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp; (values &gt;= threshold))
```

Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False

for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break

return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Get unique seeds, rho values, and branches from the data

```
seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])
```

```
# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:

for rho in rho_values:
```

```
# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])


# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]


# Get reference targets for both horizon systems

# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])


# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

if branch == "continue":

# Skip continue branch - recovery is relative to continue

continue
```

```
prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue

# Use exact equality since all trajectories use the same diagnostic schedule
```

```
if not np.array_equal(sample_times, cont_sample_times):
raise RuntimeError(
f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"
)

# Also validate that trajectory lengths match
if len(density_trajectory) != len(cont_density_trajectory):
raise RuntimeError(
f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"
)
if len(similarity_trajectory) != len(cont_similarity_trajectory):
raise RuntimeError(
f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"
)
```

```
# Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

# Process density recovery

density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density

density_first_crossing, density_first_observed = first_crossing_time(

sample_times, density_values, density_target

)


# Calculate sustained recovery for density (stays above target for rest of trajectory)

density_sustained, density_sustained_observed = first_time_sustained_for_rest(

sample_times, density_values, density_target

)
```

```
# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)

# Process similarity recovery
similarity_values = similarity_trajectory
similarity_target = similarity_target

# Calculate first crossing for similarity
sim_first_crossing, sim_first_observed = first_crossing_time(
sample_times, similarity_values, similarity_target
)
```

```
# Calculate sustained recovery for similarity
sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(
sample_times, similarity_values, similarity_target
)

# Calculate held recovery for similarity
sim_held, sim_held_observed = first_time_held_for_duration(
sample_times, similarity_values, similarity_target, t_window
)

# Calculate AUC deficit for similarity
sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

# Calculate restricted times (horizon for censored, event_time for observed)
# Use the last sample time as the horizon (not the elapsed difference)
horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')
```

For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time

For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time

Calculate normalized times (t_rec / T_window)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window > 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window > 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window > 0 else float('nan')

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Add rows for density metrics

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,

```
"held_time": density_held,

"held_observed": density_held_observed,

"restricted_first_crossing_time": density_restricted_first,

"restricted_sustained_time": density_restricted_sustained,

"restricted_held_time": density_restricted_held,

"auc_deficit_vs_continue": density_auc_deficit,

"first_crossing_over_Twindow": density_first_over_Twindow,

"sustained_time_over_Twindow": density_sustained_over_Twindow,

"held_time_over_Twindow": density_held_over_Twindow,

})


# Add rows for similarity metrics

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,
```

"metric": "similarity",

"target_value": similarity_target,

"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

})

```
if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")



# Validate expected count: 40 seeds × 5 rho × 3 non-continue branches × 2 metrics × 2 horizon systems = 2400

expected_count = len(seeds) *len(rho_values)* (len(branches) - 1) *2* 2

if len(recovery_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "

f"This may indicate a logic error in recovery calculation."

)



recovery_df = pd.DataFrame(recovery_rows)



print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")

print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "

f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "
```

f"Systems: {recovery_df['horizon_system'].nunique()}")

return recovery_df

==========
Stage 1: Validation
==========

def validate_run(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results

Validate condition status - MANDATORY for production

condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results

try:

condition_df = pd.read_csv(condition_status_path)

successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)

```
expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "

f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results
```

```
# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")

return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results
```

```
# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status

completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results
```

```
if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:

results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"
```

```
results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)

if actual_rows != expected_rows:

results["status"] = "failed"

results["errors"].append(

f"Row count mismatch: expected {expected_rows}, got {actual_rows}"

)



# Validate unique seeds

unique_seeds = df["seed"].unique()

if len(unique_seeds) != EXPECTED_N_SEEDS:

results["status"] = "failed"

results["errors"].append(
```

```
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)

# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)

# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
results["status"] = "failed"
```

```
results["errors"].append(
f"Branches mismatch: expected {expected_branches}, got {unique_branches}"
)

# Validate completeness: each seed/rho/branch combination should exist
missing_combinations = []
for seed in unique_seeds:
seed_data = df[df["seed"] == seed]
for rho in [float(r) for r in df["rho"].unique()]:
rho_data = seed_data[seed_data["rho"] == rho]
for branch in unique_branches:
if not ((rho_data["branch"] == branch).any()):
missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")

if missing_combinations:
results["status"] = "failed"
results["errors"].append(
```

```
f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():

results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and finite

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")

else:
```

```
# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"

)


# Check that values are finite (not NaN, inf, or -inf)

non_finite = ~np.isfinite(df[outcome])

if non_finite.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_finite.sum()} non-finite values"

)



# Get baseline equivalence certificate hash - MANDATORY for production
```

```
baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"

if not baseline_cert_path.exists():

results["status"] = "failed"

results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")

return results

baseline_cert_hash = file_sha256(baseline_cert_path)



# Record validation info

results["validation"] = {

"n_rows": actual_rows,

"n_seeds": len(unique_seeds),

"n_rho": len(unique_rho),

"n_branches": len(unique_branches),

"n_duplicates": int(duplicate_rows.sum()),

"n_missing_combinations": len(missing_combinations),

"baseline_equivalence": baseline_eq.get("status"),

"freezeable": manifest.get("freezeable", False),
```

"baseline_certificate_hash": baseline_cert_hash,

}

if results["status"] == "passed":

print("✅ Stage 1: Validation PASSED")

print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho, {len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")

return results

==========
Stage 2: Regime Contrasts
==========

```
def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],
```

```
].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]

.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,
```

```
on="seed",

how="inner",

validate="many_to_one",

)



# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")
```

```
rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,

"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(
```

```
f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)



print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")



return contrasts_df

================================================================================
Stage 3: Branch Difference-in-Differences
================================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:
```

```
text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} -
Y_{s,1,continue}]



For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.



Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]



for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue
```

```
rho_data = seed_data[seed_data["rho"] == rho]

# Get baseline (rho=1.0) data for this seed
baseline_data = seed_data[seed_data["rho"] == 1.0]

for branch in NON_CONTINUE_BRANCHES:
for outcome in outcomes:
# Get Y_{s,ρ,b}
y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]
if len(y_rho_b_match) == 0:
raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")
y_rho_b = float(y_rho_b_match.iloc[0])

# Get Y_{s,ρ,continue}
y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]
if len(y_rho_c_match) == 0:
raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")
y_rho_c = float(y_rho_c_match.iloc[0])
```

```
# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")

y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({
```

```
"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,

"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(
```

```
f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)



print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches:
{did_df['branch'].nunique()}")



return did_df

================================================================================
Stage 4: Cross-T Topology and Recovery
================================================================================

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(
```

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))

Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)

topology_rows = []

missing_keys = []

for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline

```
# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)


if rho_key not in npz_data:

missing_keys.append(rho_key)

continue

if baseline_key not in npz_data:

missing_keys.append(baseline_key)

continue


B_rho = npz_data[rho_key]

B_baseline = npz_data[baseline_key]


# Compute metrics

cos_sim = cosine_similarity(B_rho, B_baseline)

jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)

norm_dist = normalized_distance(B_rho, B_baseline)
```

```
topology_rows.append({

"seed": seed,

"rho": rho,

"cross_T_cosine": cos_sim,

"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)
```

print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")

return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

if recovery.empty:

return pd.DataFrame()

Create seeded RNG for reproducibility

if bootstrap_seed is None:

raise ValueError("bootstrap_seed is required")

rng = np.random.default_rng(bootstrap_seed)

did_rows = []

seeds = sorted(recovery["seed"].unique())

rho_values = sorted([float(r) for r in recovery["rho"].unique()])

```
branches = sorted(recovery["branch"].unique())

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())



# Recovery outcomes to compute DID for

recovery_outcomes = [

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]



# Also compute DID for recovery indicators

indicator_outcomes = [
```

```
"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]


for rho in rho_values:

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in branches:
```

```
if branch == "continue":

continue


for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;
```

```
(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,1,b} (baseline rho=1.0, same branch)

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]
```

```
if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or
len(r_base_c_match) == 0:

# Skip if any data is missing

continue



r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])



# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline



did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,
```

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &

(rho_data["horizon_system"] == horizon_system) &

```
(rho_data["metric"] == metric)

][outcome]


r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)
```

][outcome]

if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or len(r_base_c_match) == 0:

continue

r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])

For binary outcomes, use difference in proportions

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline

did_rows.append({

"seed": seed,

"rho": rho,

```
"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})



if not did_rows:

return pd.DataFrame()



did_df = pd.DataFrame(did_rows)
```

```
print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "

f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.
```

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

```
# Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)
```

```
ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),
```

```
n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary
```

```
print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)



# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)
```

```
for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)
```

```
jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper
```

```
summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

# Raw event times (conditional on observed recovery)

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),

std_sustained=("sustained_time", "std"),
```

```
n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),

std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),


# Censoring-aware restricted times (cohort-level estimates)

mean_restricted_first=("restricted_first_crossing_time", "mean"),

std_restricted_first=("restricted_first_crossing_time", "std"),

n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),
```

```
n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window)

mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),

std_first_over_Twindow=("first_crossing_over_Twindow", "std"),

n_first_over_Twindow=("first_crossing_over_Twindow", "count"),

mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),

std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),

n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),

mean_held_over_Twindow=("held_time_over_Twindow", "mean"),

std_held_over_Twindow=("held_time_over_Twindow", "std"),

n_held_over_Twindow=("held_time_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),

observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)
```

```
# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))

bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;

(recovery_summary["branch"] == branch) &amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower
```

```
recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper


# Bootstrap restricted first crossing time (cohort-level, includes censored)

restricted_first_values = group["restricted_first_crossing_time"].dropna().values

if len(restricted_first_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))

bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper
```

```
# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))



idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper
```

```
# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper


# Bootstrap normalized first crossing time over T_window
```

```
first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values
```

```
if len(sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:
```

```
indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper


# Bootstrap AUC deficit

auc_values = group["auc_deficit_vs_continue"].dropna().values

if len(auc_values) &gt; 0:

indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))
```

```
bootstrap_means = np.nanmean(auc_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

================================================================================
```

Report Generation

==========

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,

recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

```
lines = []
lines.append("=" * 80)
lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")
lines.append("=" * 80)
lines.append("")


# Validation summary
lines.append("VALIDATION SUMMARY")
lines.append("-" * 40)
val = validation.get("validation", {})
lines.append(f" Status: {validation.get('status', 'unknown')}")
lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")
lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")
lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")
lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")
lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")
lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")
```

```
lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:

for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])
```

```
n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)

lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:
```

```
out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")
```

```
lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"
```

```
lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):

rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])
```

```
std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")



# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])
```

```
lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")

else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")
```

```
lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")



# Cohort-level estimates (includes censored)

lines.append(f" Cohort-level estimates (all seeds):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")
```

```
lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" Mean restricted t/T_window (first): {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted t/T_window (sustained):
{float(row['mean_sustained_over_Twindow']):.4f} ± {float(row['std_sustained_over_Twindow']):.4f}
(n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")
```

```
lines.append(f" Mean restricted t/T_window (held): {float(row['mean_held_over_Twindow']):.4f} ± {float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_held_over_Twindow']):.4f}, {float(row['ci_upper_held_over_Twindow']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f} (n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f}, {float(row['ci_upper_auc_deficit']):.4f}]")



# Recovery probabilities

lines.append(f" Recovery probabilities:")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")



# Conditional-on-recovery estimates

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")

lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ± {float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")
```

```
if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)
```

```
lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")
```

```
else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,
```

```
bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {
```

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,

"source": {

"path": str(source_path),

"sha256": source_hash,

},

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

```
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
default=1000,
help="number of bootstrap iterations (default: 1000)",
)
parser.add_argument(
```

```
"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")
```

```
if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)



# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:
```

```
print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:
```

```
output_dir = Path(args.output)



# Check for stale files in output directory

expected_outputs = [

"twindow_paired_regime_contrasts.csv",

"twindow_paired_regime_contrasts_fixed_physical.csv",

"twindow_paired_regime_contrasts_normalized_EMA.csv",

"twindow_branch_interactions.csv",

"twindow_branch_interactions_fixed_physical.csv",

"twindow_branch_interactions_normalized_EMA.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_by_seed.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",
```

```
"twindow_branch_DID_secondary_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)



if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)
```

```
output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")

print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)
```

```
if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")

sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]
```

```
rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")

print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)
```

```
print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"
```

```
# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"



# Combine primary and secondary branch DID into long format

branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)



# Stage 4: Cross-T Topology

print("\nStage 4: Computing cross-T topology metrics...")

topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)



# Stage 4b: Recovery Scaling (REQUIRED for production)

print("\nStage 4b: Computing recovery scaling metrics...")

recovery = compute_recovery_scaling(df, npz_path, b_c)
```

```
# Stage 4c: Recovery Interaction DID

print("\nStage 4c: Computing recovery interaction DID...")

recovery_did = compute_recovery_did(recovery,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed)



# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology, recovery,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")
```

```
secondary_summaries = compute_seed_level_summaries(
regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed,
)


# Stage 5c: Recovery DID summaries
print("\nStage 5c: Computing recovery DID summaries...")
recovery_did_summaries = {}
if not recovery_did.empty:
recovery_did_summary = recovery_did.groupby(
["rho", "branch", "horizon_system", "metric", "outcome"],
as_index=False,
).agg(
mean=("recovery_difference_in_differences", "mean"),
std=("recovery_difference_in_differences", "std"),
n=("recovery_difference_in_differences", "count"),
```

```
)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):

values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;

(recovery_did_summary["branch"] == branch) &amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;
```

```
(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary

print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# Save outputs - construct output_files incrementally
```

```
print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)

output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)
```

```
regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"

save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)

branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")
```

```
# Branch DID (secondary only - normalized_EMA)

branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology

if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)
```

```
output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)

output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:
```

```
topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:

recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)

if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path
```

```
print(f" Saved: {recovery_did_[path.name](http://path.name)}")



# Recovery DID summaries

if not recovery_did_summary.empty:

recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path

print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")



# Secondary summaries

if not regime_summaries_secondary.empty:

regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"

save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)

output_files["regime_summary_secondary"] = regime_summary_secondary_path

print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")



if not did_summaries_secondary.empty:
```

```
did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"
save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)
output_files["did_summary_secondary"] = did_summary_secondary_path
print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")


# Text report
report = generate_text_report(
validation, regime_contrasts, branch_did, topology,
regime_summaries, did_summaries, topology_summaries,
recovery_summaries, regime_summaries_secondary, did_summaries_secondary,
recovery_did_summary
)
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")
```

```
# Analysis manifest

analysis_manifest = generate_analysis_manifest(

csv_path, npz_path, manifest_path, validation, output_files,

source_path=Path(__file__),

bootstrap_seed=args.bootstrap_seed,

n_bootstrap=args.n_bootstrap,

command_line=command_line,

)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)
```

print(report)

print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text

main()

This is now close to a complete production analysis. The fixed and normalized regime contrasts, branch DID, topology comparison, recovery event extraction, censoring-aware restricted times, deterministic bootstrap, status validation, and output separation are substantially improved.

Two scientific logic errors remain in the recovery layer:

recovery DID cannot be computed because continue rows are absent;
the report labels observed-only t/Twindowt/Twindow means as censoring-aware restricted means.

These should be fixed before the first analysis test.
1. Recovery DID is guaranteed to be empty

compute_recovery_scaling explicitly skips continue:

Python

if branch == "continue":
continue

Therefore the recovery table contains only:

text

reset_ema
reset_B
future_shuffle

But compute_recovery_did searches for:

Python

recovery["branch"] == "continue"

for every seed/rho/horizon/metric.

Those rows do not exist, so every DID comparison is skipped and:

Python

recovery_did

is empty.
Recommended correction: include continue rows

Do not skip continue in compute_recovery_scaling.

For continue:

compare the trajectory with itself;
AUC deficit is zero;
calculate first/sustained/held times against the same condition-specific target;
preserve the same schema.

Then expected recovery row count becomes:

40×5×4×2×2=
40×5×4×2×2=

]

Change:

Python

expected_count = (
len(seeds)
* len(rho_values)
* len(branches)
* 2
* 2
)

With continue rows present, the predeclared recovery DID becomes well-defined:
[Rs,ρ,b−Rs,ρ,c]−
[Rs,ρ,b−Rs,ρ,c]−

[
R_{s,1,b}-R_{s,1,c}
].
]
Alternative

If continue rows are intentionally excluded, do not call the result DID. Define:
Rs,ρ,b−Rs,1,b
Rs,ρ,b−Rs,1,b

as a branch-specific recovery sensitivity contrast.

The first option is more consistent with the rest of the study.
2. Restricted normalized times are not calculated

The raw table contains:

text

restricted_first_crossing_time
restricted_sustained_time
restricted_held_time

and separately:

text

first_crossing_over_Twindow
sustained_time_over_Twindow
held_time_over_Twindow

But the normalized fields use the uncensored event times. Unrecovered seeds receive NaN.

The recovery report labels these:

text

Mean restricted t/T_window

which is incorrect. Those means are conditional on observed recovery.

Add:

Python

```
restricted_first_over_Twindow = (
restricted_first_crossing_time
/ T_window
)

restricted_sustained_over_Twindow = (
restricted_sustained_time
/ T_window
)
```

restricted_held_over_Twindow = (
restricted_held_time
/ T_window
)

Store and summarize them across all 40 seeds.

Then distinguish report labels:
Cohort-level censoring-aware

text

mean restricted time
mean restricted time / T_window
recovery probability

Conditional on observed recovery

text

mean observed event time
mean observed event time / T_window

Do not call the latter restricted.
3. Fixed-physical held duration should be explicitly defined

Both horizon systems currently use:

Python

duration = T_window

for held recovery.

That is a valid normalized-memory outcome, but it means the fixed-physical row is not entirely fixed-physical.

Choose and document one of:
Option A

Fixed-physical held duration:
T∗=17.5
T∗=17.5

Normalized held duration:
Twindow.
Twindow.
Option B

Both systems use one current TwindowTwindow, while differing only in their late-similarity target.

If using option B, say so explicitly in the output:

text

held_duration_definition =
one_current_T_window

Option A gives a cleaner fixed-versus-normalized comparison.
4. Density rows are duplicated across horizon systems

Density target is:
0.9DB,finalcontinue,
0.9DB,finalcontinue,

which is horizon-independent. Therefore fixed and normalized density rows are identical, except possibly if held-duration definitions differ.

This is acceptable for a uniform schema, but add:

text

target_horizon_dependence =
horizon_independent

and never pool the two rows as separate observations.
5. Recovery summaries should bootstrap recovery probabilities

observed_rate_* is reported as a mean of Boolean indicators, but no interval is calculated.

At the seed level, bootstrap:

text

first crossing probability
sustained recovery probability
held recovery probability

for each rho/branch/horizon/metric.

Alternatively use Wilson intervals as descriptive marginal intervals, while retaining paired bootstrap for DID.
6. Recovery DID summaries should use restricted outcomes as primary

The DID currently includes:

restricted times;
raw observed-only normalized times;
AUC;
indicators.

After adding restricted normalized times, prioritize:

text

restricted_first_crossing_time
restricted_sustained_time
restricted_held_time

restricted_first_over_Twindow
restricted_sustained_over_Twindow
restricted_held_over_Twindow
auc_deficit_vs_continue
first_crossing_observed
sustained_observed
held_observed

Raw observed-only time DIDs should be clearly labeled secondary/conditional.
7. Secondary normalized analyses are now present

The script now:

computes fixed-physical regime/DID;
computes normalized-EMA regime/DID;
saves combined and separate raw tables;
saves separate summaries;
includes both in the report.

This is correct.
8. Recovery raw and summary files are now separated

Using:

text

twindow_recovery_by_seed.csv
twindow_recovery_summary.csv

fixes the previous overwrite problem.

This is correct and important for reinterpretability.
9. Stale-output handling still leaves stale files under --overwrite

The script permits overwrite but does not remove all expected files before analysis. If a new analysis fails before writing every output, old files remain.

When --overwrite is passed:

Python

for filename in expected_outputs:
path = output_dir / filename
if path.exists():
path.unlink()

Without --overwrite, retain the current fail-fast behavior.
10. CSV atomic writer still does not fsync the file itself

It writes through pandas to a path, then replaces and fsyncs the directory. Prefer opening the temporary file handle, flushing/fsyncing, then replacing.

This is not a scientific blocker.
11. Recovery AUC is duplicated across horizon systems

AUC is integrated over the entire future trajectory in both horizon rows, so values are identical.

Record:

text

auc_horizon = full_future

or integrate over distinct horizon intervals if horizon-specific AUC is intended.
12. Recovery row schema no longer needs event_definition

The previous misleading field has been removed in this version, which is good.
13. Validation and core contrasts are strong

The following are now close to final:

mandatory condition-status validation;
baseline certificate requirement;
freezeable/completion gate;
finite primary outcomes;

direct fixed-physical regime contrasts;
normalized secondary contrasts;
fixed and normalized DIDs;
cross-T topology;
expected-count validation;
deterministic bootstrap;
topology intervals;
stale-file protection;
analysis provenance.

Current rating
Area Rating
Validation 9/10
Fixed-physical regime contrasts 9.5/10
Normalized secondary contrasts 9/10
Branch DID 9.5/10
Cross-T topology 9/10
Recovery event extraction 8.5/10
Censoring-aware physical times 8/10
Censoring-aware normalized times 4/10
Recovery DID 2/10 because continue is absent
Output preservation 8.5/10
Current executable readiness 7.5/10
Overall: 8/10
Minimum final corrections

include continue rows in recovery output and change expected count to 3200;
add restricted-time divided by TwindowTwindow;
correct report labels for observed-only versus restricted normalized time;
define fixed versus normalized held-duration conventions;
add intervals for recovery probabilities;
delete stale outputs on --overwrite.

After those changes, this analysis will cover all four prespecified estimands and will be ready for actual testing.

Evaluate before our first rub:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Driver.

Parameterized study varying T_window = rho T_ where T_* = 25 / mean(OMEGA) = 17.5

and rho ∈ {0.5, 0.75, 1, 1.5, 2}.

Study Design:

For each seed s ∈ seeds (40 seeds)

For each rho ∈ {0.5, 0.75, 1, 1.5, 2}

Set T_window = rho * 17.5

Set alpha = exp(-dt / T_window)

Full regeneration for each (s, rho) - NO split state reuse

Branches: continue, reset_ema, reset_B, future_shuffle

Fixed physical horizons: immediate=0-17.5, recovery=17.5-87.5, late=87.5-320

EMA-normalized horizons: immediate=0-1 T_window, recovery=1-5 T_window

Output:

Scalars in CSV: per (seed, rho, branch) scalar outcomes

Arrays in NPZ: per (seed, rho, branch) trajectory and matrix outcomes

MANDATORY: Baseline equivalence test at rho=1 must pass against frozen production.

"""

from future import annotations

```
import argparse

import csv

import hashlib

import json

import os

import platform

import shutil

import sys

import time

import tempfile

from pathlib import Path

from typing import Any

try:

text

import pandas as pd

except ImportError:

text
```

print("ERROR: pandas is required for production T_window study")

print("Install with: pip install pandas")

sys.exit(1)

import numpy as np
Import the experiment engine

import aief_experiment_engine_v1 as engine
==========
Study Configuration
==========

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"

WORKFLOW_VERSION = "v1.0"
Frozen production values (from AIEF_1_4_memory_ablation_updated_REAL.py)

FROZEN_N = 12

FROZEN_DT = 0.01

FROZEN_TAU = 0.5

FROZEN_OMEGA = np.array([

text

1.00, 1.02, 1.50, 2.00, 2.01, 1.01,

1.03, 1.51, 2.02, 2.00, 1.00, 1.04

], dtype=float)

```
FROZEN_A = 1.0

FROZEN_Q = 1.5

FROZEN_DELTA = 0.05

FROZEN_ETA = 0.04

FROZEN_GAMMA = 0.015

FROZEN_MU = 0.03

FROZEN_B_C = 0.12

FROZEN_B_INIT_SCALE = 0.0005
Derived from frozen values

FROZEN_MEAN_OMEGA = float(np.mean(FROZEN_OMEGA))

FROZEN_T_STAR = 25.0 / FROZEN_MEAN_OMEGA # 17.5
Protocol from frozen run

FROZEN_T_SIM = 800.0

FROZEN_N_STEPS = int(round(FROZEN_T_SIM / FROZEN_DT)) # 80000

FROZEN_BURN_END = int(0.20 * FROZEN_N_STEPS) # 16000

FROZEN_HIST_END = int(0.60 * FROZEN_N_STEPS) # 48000

FROZEN_FUTURE_STEPS = FROZEN_N_STEPS - FROZEN_HIST_END # 32000

FROZEN_DELAY_STEPS = int(round(FROZEN_TAU / FROZEN_DT)) # 50
```

FROZEN_WINDOW_STEPS = int(round(FROZEN_T_STAR / FROZEN_DT)) # 1750

FROZEN_RECOVERY_END = min(5 * FROZEN_WINDOW_STEPS, FROZEN_FUTURE_STEPS)
8750
Rho values for T_window sensitivity

RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)
Frozen branch order from AIEF_1_4_memory_ablation_updated_REAL.py

FROZEN_BRANCH_ORDER = (

text

"continue",

"reset_ema",

"reset_B",

"reset_B_ema",

"B_carrier_only",

"full_reset",

"future_shuffle",

)
Study subset of branches

STUDY_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Fixed physical horizon times (in time units, not steps)

PHYSICAL_IMMEDIATE_END = 17.5

PHYSICAL_RECOVERY_END = 87.5 # Also PHYSICAL_LATE_START

PHYSICAL_FUTURE_END = 320.0
Defaults

DEFAULT_MASTER_SEED = 20260714

DEFAULT_N_SEEDS = 40

DEFAULT_RUN_ROOT = "runs"
==========
REQUIRED OUTPUT SCHEMA
==========
These are the MINIMUM required outputs that must be present in engine results
If any are missing, the study should fail with a clear error

REQUIRED_PREFIX_KEYS = (

text

"B_at_split",

"R_physical_past",

"theta_at_split",

"coherence_acc_at_split",

"history_at_split",

"selected_at_split",

)

REQUIRED_BRANCH_MATRIX_KEYS = (

text

"B_final",

"B_mean_late",

"P_full",

"P_immediate",

"P_recovery",

"P_late",

"R_physical_future",

"R_driver_future",

"sample_times",

"density_trajectory",

"similarity_trajectory",

)
Primary outcomes (must be present)
NOW USING FIXED-PHYSICAL HORIZONS as the primary estimands

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes for comparison (normalized EMA)

SECONDARY_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Backward compatible names (map to normalized EMA for legacy compatibility)

LEGACY_OUTCOMES = (

text

"Weighted_similarity_late",

"Retention_contrast_late",

"P_mean_late",

)
Separate FROZEN vs EXTENDED schemas
FROZEN keys must exist in historical frozen data

FROZEN_BRANCH_SCALAR_KEYS = (

text

"D_B_final",

"P_mean_full",

"P_mean_immediate",

"P_mean_recovery",

"P_mean_late",

"Weighted_similarity_final",

"Weighted_similarity_late",

"Jaccard_final",

"Retention_selected_late",

"Emergence_unselected_late",

"Retention_contrast_late",

"Corr_Rpast_Pfull",

"Corr_Rpast_Plate",

"Total_edge_weight",

"Mean_row_sum",

"Max_row_sum",

"Nonzero_edge_fraction",

"Effective_edges",

"Weight_gini",

"Weight_entropy",

"r_mean_future",

"r_mean_late",

"Minimum_similarity",

Projection diagnostics (in frozen data)

"G_min_future",

"G_min_pre_future",

"Pre_gap_violation_fraction",

"Projection_active_fraction",

"Projection_active_given_violation",

"Projection_norm_mean_active",

"Projection_norm_max",

"Projection_relative_state_mean_active",

"Projection_relative_state_max",

"Projection_relative_update_mean_active",

"Projection_relative_update_max",

Metadata

"T_window",

"rho",

)
EXTENDED keys are new in v1 (dual-horizon)

EXTENDED_BRANCH_SCALAR_KEYS = (

text

Fixed physical horizon scalars (PRIMARY ESTIMANDS)

"P_mean_late_fixed_physical",

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"Corr_Rpast_Plate_fixed_physical",

"r_mean_late_fixed_physical",

Normalized EMA horizon scalars (explicit)

"P_mean_late_normalized_EMA",

"Weighted_similarity_late_normalized_EMA",

"Retention_contrast_late_normalized_EMA",

"Corr_Rpast_Plate_normalized_EMA",

"r_mean_late_normalized_EMA",

)
All required scalar keys (frozen + extended)

REQUIRED_BRANCH_SCALAR_KEYS = FROZEN_BRANCH_SCALAR_KEYS + EXTENDED_BRANCH_SCALAR_KEYS
Desired scalar keys (not yet produced by engine, but included in CSV for future)

DESIRED_BRANCH_SCALAR_KEYS = (

text

Time-based metrics (not yet in engine)

"Time_to_similarity_below_0_9",

"Time_to_similarity_below_0_5",

Ratio metrics (not yet in engine)

"Late_persistence_ratio_to_continue",

"Late_similarity_ratio_to_continue",

)
All scalar keys (required + desired) for CSV output

BRANCH_SCALAR_KEYS = REQUIRED_BRANCH_SCALAR_KEYS + DESIRED_BRANCH_SCALAR_KEYS
All matrix keys

BRANCH_MATRIX_KEYS = REQUIRED_BRANCH_MATRIX_KEYS

```
def is_finite_number(value: Any) -> bool:
text
"""Check if a value is a finite number.


Handles int, float, numpy scalar types, and excludes NaN, inf, -inf, None, strings.
"""
return (
isinstance(
value,
(
int,
float,
np.integer,
np.floating,
),
)
and np.isfinite(value)
```

)

def validate_branch_outputs(branch: str, branch_result: dict[str, Any]) -> None:

text

"""Validate that a branch result contains all required outputs.

Raises RuntimeError if required keys are missing.

"""

missing_scalars = set(REQUIRED_BRANCH_SCALAR_KEYS) - set(branch_result.keys())

missing_matrices = set(REQUIRED_BRANCH_MATRIX_KEYS) - set(branch_result.keys())

if missing_scalars or missing_matrices:

missing = list(missing_scalars) + list(missing_matrices)

raise RuntimeError(

f"Branch '{branch}' missing required outputs: {missing[:10]}... "

f"({len(missing)} total missing)"

)

==========

Utility Functions
==========
Note: master-seed is now type=int, no longer supports 'random' for production

def generate_seed_set(master_seed: int, count: int) -> list[int]:

text

"""Generate deterministic seed set from master seed."""

if count <= 0:

raise ValueError("seed count must be positive")

children = np.random.SeedSequence(master_seed).spawn(count)

seeds = [

int(child.generate_state(1, dtype=np.uint64)[0])

for child in children

]

if len(set(seeds)) != len(seeds):

raise RuntimeError("duplicate generated seeds")

return seeds

def source_sha256() -> str | None:

text

```
"""Compute SHA-256 of this source file."""

try:

return hashlib.sha256(Path(__file__).read_bytes()).hexdigest()

except (NameError, OSError):

return None

def atomic_write_text(path: str | Path, text: str) -> None:

text

"""Write text atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temporary = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temporary, "w", encoding="utf-8", newline="") as handle:

handle.write(text)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary), str(path))

finally:
```

```
if temporary.exists():

temporary.unlink()

def atomic_write_json(path: str | Path, data: dict[str, Any]) -> None:

text

"""Write JSON atomically with fsync."""

text = json.dumps(data, indent=2, sort_keys=True, allow_nan=False) + "\n"

atomic_write_text(path, text)

def save_npz_atomic(path: str | Path, payload: dict[str, Any]) -> None:

text

"""Save NPZ atomically with fsync.



Uses a temporary file with .npz suffix so np.savez_compressed doesn't add extra suffix.

"""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

# Use tempfile.mkstemp to create a temporary file in the same directory

fd, temporary = tempfile.mkstemp(
```

```
dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp.npz",

)

try:

with os.fdopen(fd, "wb") as handle:

np.savez_compressed(handle, **payload)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

def load_npz(path: str | Path) -> dict[str, Any]:

text

"""Load NPZ file."""

payload: dict[str, Any] = {}
```

```
with np.load(str(path), allow_pickle=False) as data:

for key in data.files:

value = data[key]

payload[key] = value.item() if value.ndim == 0 else value.copy()

return payload

def save_csv_atomic(path: str | Path, dataframe) -> None:

text

"""Save CSV atomically with fsync."""

if pd is None:

raise RuntimeError("pandas is required for CSV output but is not installed")



path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

fd, temporary = tempfile.mkstemp(

dir=str(path.parent),

prefix=f".{[path.name](http://path.name)}.",

suffix=".tmp",
```

)

try:

with os.fdopen(fd, "w", encoding="utf-8", newline="") as handle:

dataframe.to_csv(handle, index=False)

handle.flush()

os.fsync(handle.fileno())

os.replace(temporary, str(path))

finally:

if os.path.exists(temporary):

os.unlink(temporary)

```
================================================================================
Configuration and Validation
================================================================================
```

def create_model_config(rho: float) -> engine.ModelConfig:

text

"""Create ModelConfig for a given rho value."""

T_window = rho * FROZEN_T_STAR

return engine.ModelConfig(

```
N=FROZEN_N,

dt=FROZEN_DT,

tau=FROZEN_TAU,

omega=tuple(FROZEN_OMEGA.tolist()),

a=FROZEN_A,

q=FROZEN_Q,

delta=FROZEN_DELTA,

eta=FROZEN_ETA,

mu=FROZEN_MU,

gamma=FROZEN_GAMMA,

T_window=T_window,

B_c=FROZEN_B_C,

B_init_scale=FROZEN_B_INIT_SCALE,

scale_epsilon=1e-12,

)

def create_protocol_config() -> engine.ProtocolConfig:

text

"""Create ProtocolConfig matching frozen production."""
```

```
return engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0, # 0.20 * 800

split_time=480.0, # 0.60 * 800

future_duration=320.0, # 800 - 480

diagnostic_stride=50,

branches=STUDY_BRANCHES,

)

================================================================================
Simulation Functions
================================================================================

def simulate_seed_rho(

text

seed: int,

rho: float,

config: engine.ModelConfig,

protocol: engine.ProtocolConfig,
```

```
study_branches: tuple[str, ...],
) -> dict[str, Any]:
text
"""Simulate a single (seed, rho) combination.

CRITICAL: Uses FROZEN_BRANCH_ORDER for RNG namespace, but only simulates
study_branches subset. This ensures future_shuffle gets the 7th RNG sequence
(not the 4th) as in production.

Returns complete results for all branches at this (seed, rho).
"""
# Create branch RNGs using FULL frozen branch order for correct namespace
branch_rngs = engine.make_branch_rngs(seed, FROZEN_BRANCH_ORDER)

# Create a protocol with just the study branches
study_protocol = engine.ProtocolConfig(
T_sim=protocol.T_sim,
```

```
burn_end_time=protocol.burn_end_time,

split_time=protocol.split_time,

future_duration=protocol.future_duration,

diagnostic_stride=protocol.diagnostic_stride,

branches=study_branches,

)



# Simulate prefix

split_state, prefix = engine.simulate_prefix(seed, config, study_protocol)



# Simulate only study branches, using RNGs from frozen namespace

branch_results = {}

for branch in study_branches:

if branch not in branch_rngs:

raise RuntimeError(

f"Branch '{branch}' not in FROZEN_BRANCH_ORDER. "

f"Cannot map to production RNG namespace."
```

```
)

rng = branch_rngs[branch]

branch_results[branch] = engine.simulate_future(

split_state.clone(),

prefix,

branch,

config,

study_protocol,

rng,

)

# Add metadata

branch_results[branch]["T_window"] = config.T_window

branch_results[branch]["rho"] = rho



return {

"seed": int(seed),

"rho": rho,
```

"config": {

"N": config.N,

"dt": config.dt,

"tau": config.tau,

"T_window": config.T_window,

"alpha": config.alpha,

"mean_omega": config.mean_omega,

"T_star": config.T_star,

},

"protocol": {

"T_sim": study_protocol.T_sim,

"burn_end_time": study_protocol.burn_end_time,

"split_time": study_protocol.split_time,

"future_duration": study_protocol.future_duration,

},

"prefix": prefix,

"branches": branch_results,

}

```
def run_baseline_equivalence(
text
seeds: list[int],
frozen_manifest_path: str,
frozen_npz_path: str,
frozen_csv_path: str,
) -> dict[str, Any]:
text
"""Run baseline equivalence test at rho=1 (T_window = T_*).


MANDATORY: Must reproduce frozen run exactly for ALL seeds.


Verifies:
1. Seed list matches frozen manifest exactly (same seeds, same order)
2. All 40 seeds pass equivalence (if 40 seeds provided)
3. Engine produces all required outputs
```

```
This helper ONLY performs equivalence and returns a result.

Main handles manifests and output paths.

"""

import json



# Load frozen manifest to compare seeds

try:

with open(frozen_manifest_path, 'r', encoding='utf-8') as f:

frozen_manifest = json.load(f)

except FileNotFoundError:

return {

"status": "failed",

"error": f"Frozen manifest not found: {frozen_manifest_path}",

"seeds_match": False,

}

except json.JSONDecodeError as e:

return {
```

```
"status": "failed",

"error": f"Frozen manifest is not valid JSON: {frozen_manifest_path} ({e})",

"seeds_match": False,

}



# Validate manifest has required structure

if "randomization" not in frozen_manifest or "paired_seeds" not in frozen_manifest["randomization"]:

return {

"status": "failed",

"error": f"Frozen manifest missing required structure: {frozen_manifest_path}",

"seeds_match": False,

}



frozen_seeds = frozen_manifest["randomization"]["paired_seeds"]



# Verify seed list matches frozen manifest EXACTLY

# This is critical: baseline equivalence tests a specific cohort
```

```
if len(seeds) != len(frozen_seeds):

return {

"status": "failed",

"error": f"Seed count mismatch: generated={len(seeds)}, frozen={len(frozen_seeds)}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,

}



for i, (gen_seed, frozen_seed) in enumerate(zip(seeds, frozen_seeds)):

if int(gen_seed) != int(frozen_seed):

return {

"status": "failed",

"error": f"Seed mismatch at index {i}: generated={gen_seed}, frozen={frozen_seed}",

"frozen_seeds": [int(s) for s in frozen_seeds],

"generated_seeds": [int(s) for s in seeds],

"seeds_match": False,
```

```
}


# Verify master seed matches
frozen_master_seed = frozen_manifest["randomization"]["master_seed"]
# Note: We can't check the generated master seed here as it's not passed,
# but the seed generation method should match


# Validate frozen CSV path exists
if not os.path.exists(frozen_csv_path):
return {
"status": "failed",
"error": f"Frozen scalar CSV not found: {frozen_csv_path}",
"seeds_match": True,
}


# Create config for rho=1
config = create_model_config(rho=1.0)
```

```
# For baseline equivalence, we need to validate ALL 7 branches

# to certify the engine as reusable infrastructure

baseline_protocol = engine.ProtocolConfig(

T_sim=FROZEN_T_SIM,

burn_end_time=160.0,

split_time=480.0,

future_duration=320.0,

diagnostic_stride=50,

branches=FROZEN_BRANCH_ORDER, # All 7 branches for engine certification

)



# Run the engine's baseline equivalence test with ALL seeds and ALL 7 branches

results = [engine.run](http://engine.run)_baseline_equivalence(

seeds=seeds,

config=config,

protocol=baseline_protocol,
```

```
frozen_manifest_path=frozen_manifest_path,

frozen_npz_path=frozen_npz_path,

frozen_csv_path=frozen_csv_path,

branch_order=FROZEN_BRANCH_ORDER, # Use FULL branch order for RNG namespace

)



# Add seed comparison info

results["seed_comparison"] = {

"generated_seeds": [int(s) for s in seeds],

"frozen_seeds": [int(s) for s in frozen_seeds],

"seeds_match": True,

}



return results

================================================================================
Manifest and Output
================================================================================
```

```
def build_manifest(
text
master_seed: int,
seeds: list[int],
rho_values: tuple[float, ...],
) -> dict[str, Any]:
text
"""Build study manifest."""
# Compute config hash from model parameters
model_params = {
"N": FROZEN_N,
"dt": FROZEN_DT,
"tau": FROZEN_TAU,
"omega": FROZEN_OMEGA.tolist(),
"a": FROZEN_A,
"q": FROZEN_Q,
"delta": FROZEN_DELTA,
"eta": FROZEN_ETA,
```

"gamma": FROZEN_GAMMA,

"mu": FROZEN_MU,

"B_c": FROZEN_B_C,

"B_init_scale": FROZEN_B_INIT_SCALE,

"T_star": FROZEN_T_STAR,

}

protocol_params = {

"T_sim": FROZEN_T_SIM,

"burn_end_time": 160.0,

"split_time": 480.0,

"future_duration": 320.0,

"diagnostic_stride": 50,

"branches": list(STUDY_BRANCHES),

}

study_params = {

```
"rho_values": list(rho_values),

"T_window_values": [rho * FROZEN_T_STAR for rho in rho_values],

"n_seeds": len(seeds),

"statistical_unit": "40 seed/graphs",

}



# Add dependencies - engine and driver source hashes

dependencies = {}



# Engine source hash

try:

engine_path = Path(engine.__file__).resolve()

engine_hash = file_sha256(engine_path)

dependencies["aief_experiment_engine_v1"] = {

"path": str(engine_path),

"sha256": engine_hash,

}
```

```
except Exception as e:
dependencies["aief_experiment_engine_v1"] = {
"path": "NOT_FOUND",
"sha256": "NOT_AVAILABLE",
"error": str(e),
}


# Driver source hash (this file)
try:
driver_path = Path(__file__).resolve()
driver_hash = file_sha256(driver_path)
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
"path": str(driver_path),
"sha256": driver_hash,
}
except Exception as e:
dependencies["AIEF_1_4_Twindow_sensitivity"] = {
```

```
"path": "NOT_FOUND",

"sha256": "NOT_AVAILABLE",

"error": str(e),

}



manifest = {

"study_name": STUDY_NAME,

"workflow_version": WORKFLOW_VERSION,

"source_sha256": source_sha256(),

"dependencies": dependencies,

"model": model_params,

"protocol": protocol_params,

"study": study_params,

"randomization": {

"master_seed": int(master_seed),

"paired_seeds": [int(seed) for seed in seeds],

"seed_generation": "SeedSequence(master_seed).spawn(n_seeds)",
```

"branch_generation": "second per-seed child sequence spawns named uint64 branch seeds",

"bit_generator": np.random.default_rng(0).bit_generator.__class__.__name__,

},

"environment": {

"python": platform.python_version(),

"numpy": np.__version__,

"platform": platform.platform(),

},

"schema": {

"prefix_keys": list(REQUIRED_PREFIX_KEYS),

"branch_scalar_keys": list(BRANCH_SCALAR_KEYS),

"branch_matrix_keys": list(BRANCH_MATRIX_KEYS),

"required_branch_scalar_keys": list(REQUIRED_BRANCH_SCALAR_KEYS),

"required_branch_matrix_keys": list(REQUIRED_BRANCH_MATRIX_KEYS),

},

"notes": {

"purpose": "T_window sensitivity study - varying T_window = rho *T_*",

```
"T_*": FROZEN_T_STAR,
"fixed_physical_horizons": {
"immediate": f"0-{PHYSICAL_IMMEDIATE_END} time units",
"recovery": f"{PHYSICAL_IMMEDIATE_END}-{PHYSICAL_RECOVERY_END} time units",
"late": f"{PHYSICAL_RECOVERY_END}-{PHYSICAL_FUTURE_END} time units",
},
"ema_normalized_horizons": {
"immediate": "0-1 T_window",
"recovery": "1-5 T_window",
"late": "5 T_window to end",
},
"full_regeneration": "Each (seed, rho) pair is independently initialized",
},
}



return manifest
def flatten_result(
```

```
text

seed: int,

rho: float,

result: dict[str, Any],

) -> dict[str, Any]:

text

"""Flatten a (seed, rho) result into a flat dict for NPZ storage."""

payload: dict[str, Any] = {

f"seed_{seed}_rho_{rho}_config_T_window": result["config"]["T_window"],

f"seed_{seed}_rho_{rho}_config_alpha": result["config"]["alpha"],

}



# Add prefix keys

for key, value in result["prefix"].items():

payload[f"prefix_seed_{seed}_rho_{rho}_{key}"] = value



# Add branch results
```

```
for branch, branch_result in result["branches"].items():

for key, value in branch_result.items():

if isinstance(value, np.ndarray):

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value

else:

payload[f"{branch}_seed_{seed}_rho_{rho}_{key}"] = value



return payload

def extract_scalar_row(

text

seed: int,

seed_index: int,

rho: float,

branch: str,

branch_result: dict[str, Any],

) -> dict[str, Any]:

text
```

"""Extract a scalar row for CSV output.

Validates that all required outputs are present before extraction.

"""

Validate required outputs

validate_branch_outputs(branch, branch_result)

row = {

"seed_index": seed_index,

"seed": seed,

"rho": rho,

"T_window": branch_result.get("T_window", float('nan')),

"branch": branch,

}

for key in BRANCH_SCALAR_KEYS:

if key in branch_result:

```
row[key] = branch_result[key]

else:

row[key] = float('nan')



return row

# ============================================================================
# Main Execution
# ============================================================================

def parse_arguments() -> argparse.Namespace:

text

"""Parse command line arguments."""

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

parser.add_argument(

"--master-seed",

type=int,
```

```
default=DEFAULT_MASTER_SEED,

help="fixed integer master seed (default: 20260714). For production, must match frozen data.",

)

parser.add_argument(

"--n-seeds",

type=int,

default=DEFAULT_N_SEEDS,

help=f"number of seeds (default: {DEFAULT_N_SEEDS})",

)

parser.add_argument(

"--run-root",

default=DEFAULT_RUN_ROOT,

help=f"output root directory (default: {DEFAULT_RUN_ROOT})",

)

parser.add_argument(

"--frozen-manifest",

default=None,
```

help="path to frozen manifest for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-npz",

default=None,

help="path to frozen NPZ for baseline equivalence test. "

"Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

"--frozen-scalar",

default=None,

help="path to frozen scalar CSV for baseline equivalence test. "

"MANDATORY for scalar validation. Generate with: python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40",

)

parser.add_argument(

```
"--overwrite",

action="store_true",

help="overwrite existing run directory. Use with caution - will delete existing data.",

)

# NOTE: --skip-baseline removed for production safety

# Baseline equivalence is MANDATORY and cannot be skipped

# For debugging, use a separate nonproduction namespace

parser.add_argument(

"--dry-run",

action="store_true",

help="validate configuration and exit without running",

)

return parser.parse_args()

def main() -> None:

text

"""Main entry point."""

args = parse_arguments()
```

```
# Generate seeds

seeds = generate_seed_set(int(args.master_seed), args.n_seeds)



# Build manifest

manifest = build_manifest(

master_seed=int(args.master_seed),

seeds=seeds,

rho_values=RHO_VALUES,

)



# Compute config hash

manifest_text = json.dumps(

manifest, sort_keys=True, separators=(",", ":"), allow_nan=False

)

config_hash = hashlib.sha256(manifest_text.encode()).hexdigest()[:12]



# Check dry-run mode FIRST to avoid contaminating production directory
```

```
if args.dry_run:

print("Dry run: configuration validated successfully.")

print(f"Configuration hash: {config_hash}")

print("To execute, run without --dry-run")

print(f"Expected run directory: {Path([args.run](http://args.run)_root) / config_hash}")

return



# Create run directory

run_dir = Path([args.run](http://args.run)_root) / config_hash



# Check if run directory already exists

if run_dir.exists():

if args.overwrite:

# Overwrite mode: remove existing directory

print(f"Overwriting existing run directory: {run_dir}")

try:

shutil.rmtree(run_dir)
```

```
except Exception as e:

print(f"Cannot overwrite: error removing directory: {e}")

sys.exit(1)

run_dir.mkdir(parents=True, exist_ok=True)


else:

# Default: fail if directory exists

print(f"Run directory {run_dir} already exists")

print("Use --overwrite to replace")

sys.exit(1)

else:

# New run directory

run_dir.mkdir(parents=True, exist_ok=True)



# Save manifest

manifest["config_hash"] = config_hash

manifest_path = run_dir / "manifest.json"
```

```
atomic_write_json(manifest_path, manifest)



print(f"{STUDY_NAME}")

print(f" master seed: {args.master_seed}")

print(f" n_seeds: {args.n_seeds}")

print(f" rho values: {RHO_VALUES}")

print(f" configuration: {config_hash}")

print(f" output: {run_dir}")

print()



# Create protocol config (same for all rho)

protocol = create_protocol_config()



# MANDATORY: Run baseline equivalence test at rho=1

# This cannot be skipped - baseline equivalence is required for freezeability

print("=" * 80)

print("Running MANDATORY baseline equivalence test at rho=1...")
```

```
print("=" * 80)



# Validate all frozen paths are provided (MANDATORY)

for path_name, path_value in [

("frozen-manifest", args.frozen_manifest),

("frozen-npz", args.frozen_npz),

("frozen-scalar", args.frozen_scalar),

]:

if path_value is None:

print(f"\n{path_name} path not provided!")

print("All frozen baseline artifacts are required for production runs.")

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then provide all paths with --frozen-manifest, --frozen-npz, --frozen-scalar")

sys.exit(1)

if not os.path.isfile(path_value):

print(f"\n{path_name} file not found: {path_value}")
```

print("To generate frozen data, run:")

print(" python3 AIEF_1_4_memory_ablation_updated_REAL.py --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path.")

sys.exit(1)

Run baseline equivalence - MANDATORY for all seeds

try:

baseline_results = run_baseline_equivalence(

seeds=seeds,

frozen_manifest_path=args.frozen_manifest,

frozen_npz_path=args.frozen_npz,

frozen_csv_path=args.frozen_scalar,

)

except Exception as exc:

Handle baseline equivalence exception - save failure manifest

failure_manifest = {

**manifest,

```
"config_hash": config_hash,

"status": "FAILED",

"failure_reason": "baseline_equivalence_exception",

"exception_type": type(exc).__name__,

"exception_message": str(exc),

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)

print(f"\nBaseline equivalence exception occurred!")

print(f"Failure manifest saved to {failure_manifest_path}")

print(f"Exception: {type(exc).__name__}: {exc}")

sys.exit(1)



if baseline_results["status"] != "passed":

error_msg = baseline_results.get('error', 'unknown')

print(f"\nBaseline equivalence test FAILED: {error_msg}")
```

```
# Check if it's a configuration mismatch

if "mismatch" in error_msg.lower():

print("\nConfiguration mismatch detected!")

print("The frozen data does not match the expected production configuration.")

print("Expected configuration (from AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)):")

print(f" N={FROZEN_N}, DT={FROZEN_DT}, TAU={FROZEN_TAU}")

print(f" OMEGA={FROZEN_OMEGA.tolist()}")

print(f" T_WINDOW={FROZEN_T_STAR:.6f}")

print("\nTo generate correct frozen data:")

print(" python3 AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py) --master-seed 20260714 --n-seeds 40")

print("Then use the generated run directory path with --frozen-manifest and --frozen-npz")



# Save failure manifest with detailed results

failure_manifest = {

**manifest,

"config_hash": config_hash,

"status": "FAILED",
```

```
"failure_reason": "baseline_equivalence_failed",

"failure_details": baseline_results,

"freezeable": False,

}

failure_manifest_path = run_dir / "FAILED_manifest.json"

atomic_write_json(failure_manifest_path, failure_manifest)


# Also save detailed per-seed results

if "seed_results" in baseline_results:

baseline_eq_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_eq_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_eq_path}")


print(f"\nFailure manifest saved to {failure_manifest_path}")


sys.exit(1)

else:

print("\nBaseline equivalence test PASSED")
```

```
print("=" * 80)


# Save detailed baseline equivalence certificate

baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"

atomic_write_json(baseline_cert_path, baseline_results)

print(f" Detailed certificate saved to {baseline_cert_path}")


# Save per-seed baseline results separately for easier audit

if "seed_results" in baseline_results:

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

atomic_write_json(baseline_seed_path, baseline_results["seed_results"])

print(f" Per-seed results saved to {baseline_seed_path}")


# Update manifest with baseline equivalence certificate

# Record all frozen input hashes for complete provenance

frozen_input_hashes = {}


# Get frozen manifest hash

if "frozen_manifest_hash" in baseline_results:
```

```
frozen_input_hashes["frozen_manifest"] = baseline_results["frozen_manifest_hash"]
if "frozen_manifest_path" in baseline_results:
frozen_input_hashes["frozen_manifest_path"] = baseline_results["frozen_manifest_path"]

# Get frozen NPZ hash
if "frozen_npz_hash" in baseline_results:
frozen_input_hashes["frozen_npz"] = baseline_results["frozen_npz_hash"]
if "frozen_npz_path" in baseline_results:
frozen_input_hashes["frozen_npz_path"] = baseline_results["frozen_npz_path"]

# Get frozen CSV hash
if "frozen_csv_hash" in baseline_results:
frozen_input_hashes["frozen_csv"] = baseline_results["frozen_csv_hash"]
if "frozen_csv_path" in baseline_results:
frozen_input_hashes["frozen_csv_path"] = baseline_results["frozen_csv_path"]

# Get frozen production source hash
if "frozen_source_hash" in baseline_results:
frozen_input_hashes["frozen_production_source"] = baseline_results["frozen_source_hash"]
```

```
# Compute frozen production source hash directly if not in results

# Check both possible filenames: with and without _REAL suffix

frozen_production_paths = [

Path(args.frozen_manifest).parent /
"AIEF_1_4_memory_ablation_updated_[REAL.py](http://REAL.py)",

Path(args.frozen_manifest).parent / "AIEF_1_4_memory_ablation_[updated.py](http://updated.py)",

]

for candidate_path in frozen_production_paths:

if candidate_path.exists():

try:

frozen_prod_hash = file_sha256(candidate_path)

frozen_input_hashes["frozen_production_source"] = frozen_prod_hash

frozen_input_hashes["frozen_production_path"] = str(candidate_path)

break

except Exception:

pass
```

```
manifest["baseline_equivalence"] = {

"status": "passed",

"n_seeds_tested": baseline_results["n_seeds"],

"n_seeds_passed": baseline_results["n_seeds"],

"engine_source_sha256": baseline_results.get("engine_source_sha256"),

"frozen_manifest_hash": baseline_results.get("frozen_manifest_hash"),

"frozen_npz_hash": baseline_results.get("frozen_npz_hash"),

"frozen_csv_hash": baseline_results.get("frozen_csv_hash", "N/A"),

"frozen_production_source_hash": frozen_input_hashes.get("frozen_production_source", "N/A"),

"frozen_input_hashes": frozen_input_hashes,

"certificate_path": str(baseline_cert_path),

"seed_results_path": str(baseline_seed_path) if "seed_results" in baseline_results else "N/A",

"branches_validated": list(FROZEN_BRANCH_ORDER), # All 7 branches

}


# Re-save manifest with baseline certificate

atomic_write_json(manifest_path, manifest)
```

```
# Run the study

print("\nRunning T_window sensitivity study...")

print(f" rho values: {RHO_VALUES}")

print(f" branches: {STUDY_BRANCHES}")

print(f" seeds: {len(seeds)}")



all_scalar_rows: list[dict[str, Any]] = []

all_matrix_payload: dict[str, Any] = {} # Full payload in memory; checkpoints also written per-condition



# Track condition status

condition_status = []

successful_conditions = 0

failed_conditions = 0



started = time.time()

total_combinations = len(seeds) * len(RHO_VALUES)
```

```
# Fatal error flag for breaking outer loop on schema/config mismatch

fatal_error = False



for seed_index, seed in enumerate(seeds):

# Check for fatal error from previous seed

if fatal_error:

print(f" Skipping seed {seed} due to fatal schema/config error")

break


for rho in RHO_VALUES:

condition_start = time.time()


# Create config for this rho

config = create_model_config(rho)


# Checkpoint path for this condition (saved for diagnostic recovery, not resume)

checkpoint_path = run_dir / f"condition_seed_{seed}_rho_{rho:.2f}.npz"
```

```
try:

# Validate config before simulation

config.validate()

protocol.validate(config)


# Run simulation

result = simulate_seed_rho(

seed=seed,

rho=rho,

config=config,

protocol=protocol,

study_branches=STUDY_BRANCHES,

)


# Extract scalar rows

for branch in STUDY_BRANCHES:

branch_result = result["branches"][branch]

row = extract_scalar_row(
```

```
seed=seed,

seed_index=seed_index,

rho=rho,

branch=branch,

branch_result=branch_result,

)

all_scalar_rows.append(row)


# Extract matrix payload

matrix_payload = flatten_result(seed, rho, result)

all_matrix_payload.update(matrix_payload)


# Save checkpoint atomically

save_npz_atomic(checkpoint_path, matrix_payload)


condition_status.append({

"seed": seed,

"rho": rho,

"status": "success",
```

```
"elapsed_seconds": time.time() - condition_start,

})

successful_conditions += 1


except Exception as e:

error_str = str(e)

condition_status.append({

"seed": seed,

"rho": rho,

"status": "failed",

"error": error_str,

"elapsed_seconds": time.time() - condition_start,

})

failed_conditions += 1

print(f" ERROR: seed={seed}, rho={rho:.2f}: {e}")


# Stop immediately on schema/config mismatch - use fatal flag to break outer loop

if "schema" in error_str.lower() or "config" in error_str.lower() or "missing" in error_str.lower():
```

```
print(f"\n FATAL: Schema/config mismatch detected - stopping immediately")

print(f" This error will affect all remaining conditions")

# Set fatal error flag to break outer loop after inner loop

fatal_error = True

break



# Status

elapsed = time.time() - started

current_combination = seed_index * len(RHO_VALUES) + list(RHO_VALUES).index(rho) + 1

status_str = f"[{current_combination}/{total_combinations}] seed={seed}, rho={rho:.2f}
({elapsed:.1f}s)"

if successful_conditions + failed_conditions == current_combination:

print(f" {status_str}")



# Save condition status table

if condition_status:

if pd is not None:

status_df = pd.DataFrame(condition_status)
```

```
status_path = run_dir / "twindow_condition_status.csv"

save_csv_atomic(status_path, status_df)

print(f"\nCondition status saved to {status_path}")

else:

# Fallback: save as JSON if pandas not available

status_path = run_dir / "twindow_condition_status.json"

atomic_write_json(status_path, condition_status)

print(f"\nCondition status saved to {status_path} (pandas not available)")


# Record condition status in manifest

manifest["condition_status"] = {

"path": str(status_path),

"sha256": file_sha256(status_path),

"expected_rows": total_combinations,

"successful_rows": successful_conditions,

"failed_rows": failed_conditions,

}
```

```
print(f" Successful: {successful_conditions}/{total_combinations}")

print(f" Failed: {failed_conditions}/{total_combinations}")



# If any conditions failed, mark as non-freezeable

if failed_conditions &amp;gt; 0:

manifest["freezeable"] = False

manifest["failed_conditions"] = failed_conditions

atomic_write_json(manifest_path, manifest)

print(f"\nWARNING: {failed_conditions} conditions failed - run is NOT freezeable")



# Validate we have the expected number of scalar rows

expected_rows = total_combinations * len(STUDY_BRANCHES)

actual_rows = len(all_scalar_rows)

if actual_rows != expected_rows:

print(f"\nERROR: Expected {expected_rows} scalar rows, got {actual_rows}")

print("Run is INCOMPLETE and NOT freezeable")
```

```
# Validate primary outcomes are present and finite

missing_primary = []

for row in all_scalar_rows:

for key in PRIMARY_OUTCOMES:

if key not in row or not is_finite_number(row[key]):

missing_primary.append(f"seed={row['seed']}, rho={row['rho']}, branch={row['branch']},
key={key}")



if missing_primary:

print(f"\nERROR: {len(missing_primary)} primary outcomes are missing or non-finite!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")

print("Run is INCOMPLETE and NOT freezeable")



# Save outputs

csv_path = run_dir / f"{STUDY_NAME}_{config_hash}_scalar_summary.csv"

npz_path = run_dir / f"{STUDY_NAME}_{config_hash}_branch_diagnostics.npz"
```

```
# Write CSV

if all_scalar_rows:

# Sort by seed_index, rho, branch for consistent ordering

all_scalar_rows.sort(key=lambda r: (r["seed_index"], r["rho"], r["branch"]))


# Get all fieldnames

fieldnames = list(all_scalar_rows[0].keys())


temporary_csv = csv_path.with_suffix(csv_path.suffix + ".tmp")

try:

with open(temporary_csv, "w", newline="", encoding="utf-8") as handle:

writer = csv.DictWriter(handle, fieldnames=fieldnames)

writer.writeheader()

writer.writerows(all_scalar_rows)

handle.flush()

os.fsync(handle.fileno())

os.replace(str(temporary_csv), str(csv_path))

finally:
```

```
if temporary_csv.exists():

temporary_csv.unlink()



# Write NPZ

if all_matrix_payload:

save_npz_atomic(npz_path, all_matrix_payload)



# Determine completion status

complete = (

failed_conditions == 0

and successful_conditions == total_combinations

and len(all_scalar_rows) == expected_rows

and len(missing_primary) == 0

)



# Top-level freezeable flag

baseline_passed = baseline_results.get("status") == "passed"
```

```
manifest["freezeable"] = complete and baseline_passed



# Final manifest update with completion status

manifest["completion"] = {

"status": "completed" if complete else "incomplete",

"n_conditions_requested": total_combinations,

"n_conditions_successful": successful_conditions,

"n_conditions_failed": failed_conditions,

"n_scalar_rows": len(all_scalar_rows),

"n_matrix_entries": len(all_matrix_payload),

"n_missing_primary_outcomes": len(missing_primary),

"elapsed_seconds": time.time() - started,

"freezeable": manifest["freezeable"],

}



# Add output hashes - hash ALL final outputs for complete provenance

output_hashes = {}
```

```
# Main outputs
if csv_path.exists():
output_hashes["scalar_csv"] = file_sha256(csv_path)
if npz_path.exists():
output_hashes["branch_npz"] = file_sha256(npz_path)


# Condition status
condition_status_path = run_dir / "twindow_condition_status.csv"
if condition_status_path.exists():
output_hashes["condition_status_csv"] = file_sha256(condition_status_path)


# Baseline equivalence certificate
baseline_cert_path = run_dir / "baseline_equivalence_certificate.json"
if baseline_cert_path.exists():
output_hashes["baseline_certificate"] = file_sha256(baseline_cert_path)
```

```
# Baseline per-seed results (if saved separately)

baseline_seed_path = run_dir / "baseline_equivalence_by_seed.json"

if baseline_seed_path.exists():

output_hashes["baseline_seed_results"] = file_sha256(baseline_seed_path)



if output_hashes:

manifest["output_hashes"] = output_hashes



# Re-save final manifest

atomic_write_json(manifest_path, manifest)



# Print completion status

if complete and baseline_passed:

print(f"\nRun COMPLETED successfully and is FREEZEABLE")

elif complete and not baseline_passed:

print(f"\nRun COMPLETED but baseline equivalence FAILED - NOT freezeable")

else:
```

```
print(f"\nRun INCOMPLETE - NOT freezeable")

if failed_conditions &gt; 0:

print(f" Failed conditions: {failed_conditions}/{total_combinations}")

if len(all_scalar_rows) != expected_rows:

print(f" Missing scalar rows: {expected_rows - len(all_scalar_rows)}")

if len(missing_primary) &gt; 0:

print(f" Missing primary outcomes: {len(missing_primary)}")



# Print summary

elapsed = time.time() - started

print(f"\nCompleted in {elapsed:.1f}s")

print(f"Saved {csv_path}")

print(f"Saved {npz_path}")

print(f"Manifest: {manifest_path}")

# Print top-level freezeable flag

freezeable_status = manifest.get("freezeable", False)

print(f"Freezeable: {freezeable_status}")
```

```
# Quick summary statistics

print("\n" + "=" * 80)

print("Summary Statistics by (rho, branch)")

print("=" * 80)



# Use the missing_primary count already computed earlier

if missing_primary:

print(f"WARNING: {len(missing_primary)} primary outcomes are non-finite or missing!")

print(f" First 5 missing: {missing_primary[:5]}")

print(" This indicates engine is not producing required outputs.")



# Group by rho and branch

for rho in RHO_VALUES:

for branch in STUDY_BRANCHES:

rows = [r for r in all_scalar_rows if r["rho"] == rho and r["branch"] == branch]

if rows:
```

```
# Only show keys that are actually present and not all NaN

available_keys = [k for k in PRIMARY_OUTCOMES if any(

k in r and not (isinstance(r[k], float) and np.isnan(r[k]))

for r in rows

)]


if not available_keys:

print(f" rho={rho:.2f}, branch={branch:&lt;15} | ALL PRIMARY OUTCOMES MISSING/NaN!")

continue


stats = {}

for key in available_keys:

values = [r[key] for r in rows if key in r and isinstance(r[key], (int, float)) and not np.isnan(r[key])]

if values:

# Use sample SD (ddof=1) for descriptive statistics

stats[key] = f"{np.mean(values):.4f}±{np.std(values, ddof=1):.4f}"

else:

stats[key] = "N/A"
```

print(f" rho={rho:.2f}, branch={branch:<15} | " +

" | ".join(f"{k}={stats.get(k, 'N/A')}" for k in available_keys))

if name == "main":

text

main()

Verdict: GO for baseline testing, not yet for the full grid

This driver is now scientifically coherent and operationally much improved. I see no remaining error that obviously invalidates the intended TwindowTwindow experiment.

The one important operational omission is that there is no way to stop after baseline equivalence. If the 40-seed equivalence gate passes, the script immediately launches all 200 regenerated conditions. Add a baseline-only mode before the first run.
1. Add --baseline-only

CLI:

Python

```
parser.add_argument(
"--baseline-only",
action="store_true",
help=(
"run the mandatory baseline "
"equivalence test, save the "
"certificate, and exit"
),
)
```

After saving the successful certificate and updated manifest:

Python

```
if args.baseline_only:
manifest["run_scope"] = (
"baseline_equivalence_only"
)
manifest["freezeable"] = False

atomic_write_json(
manifest_path,
manifest,
)

print(
"Baseline-only run completed. "
"Sensitivity grid was not started."
)
return
```

This allows the required sequence:

exact baseline certification;
inspect certificate;
only then launch the expensive grid.

Without this flag, the first successful test automatically becomes the production run.
2. Validate secondary outcomes as finite

The driver requires the secondary normalized fields to exist, but only checks primary fixed-physical fields for finiteness.

Since the analysis uses the secondary fields, validate:

Python

```
missing_secondary = []

for row in all_scalar_rows:
```

```
for key in SECONDARY_OUTCOMES:
if (
key not in row
or not is_finite_number(
row[key]
)
):
missing_secondary.append(
f"seed={row['seed']}, "
f"rho={row['rho']}, "
f"branch={row['branch']}, "
f"key={key}"
)
```

Include this in complete and the final manifest.
3. Correct the frozen production source filename

The source lookup uses filenames containing REAL.py or malformed link text. The actual source is:

text

AIEF_1_4_memory_ablation_updated.py

This affects provenance only, not simulation.
4. Condition snapshots are accurately described now

The code no longer exposes a fake resume CLI and describes the condition files as diagnostic recovery snapshots. That is acceptable.

The complete matrix payload still remains in memory, but this is a performance risk rather than a correctness error.
5. Failure handling is adequate

A schema/config failure now sets a fatal flag and exits both loops. Other condition-specific failures are recorded, and any failure makes the run incomplete/nonfreezeable.

Partial debug outputs remain possible but the downstream analysis rejects them.

This is appropriate.
6. Baseline gate is strong

The baseline stage now requires:

frozen manifest;
frozen NPZ;
frozen scalar CSV;
exact seed list;
all 40 seeds;
all seven branches;
exact frozen prefix arrays;
exact matrix/trajectory arrays;
exact scalar outcomes;
internal fixed/normalized baseline equivalence;
saved certificate and per-seed checks.

That is the correct gate.
7. Provenance is now strong

The config hash includes:

driver source hash;
engine source hash;
model;
protocol;
rho grid;
seeds;
branch design.

The final manifest includes:

baseline certificate;
per-seed equivalence;
condition status;
scalar and NPZ hashes;
completion status;
freezeability.

This is sufficient for the first production version.
Final readiness rating
Area Rating
Scientific estimand 9.5/10
Baseline equivalence 9.5/10
RNG pairing 9.5/10
Full regeneration 9.5/10
Dual-horizon outputs 9/10
Condition validation 9/10
Provenance 9/10
Large-run recovery 7/10
Current baseline-test readiness 9.5/10
Overall: 9/10
First execution sequence

After adding --baseline-only:

Bash

```
python -m py_compile \
aief_experiment_engine_v1.py \
AIEF_1_4_Twindow_sensitivity.py \
AIEF_1_4_Twindow_sensitivity_analysis.py
```

Then:

Bash

```
python AIEF_1_4_Twindow_sensitivity.py \
--frozen-manifest runs/5a9f093a2fe0/manifest.json \
--frozen-npz runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_branch_diagnostics.npz \
--frozen-scalar runs/5a9f093a2fe0/AIEF_1_4_5a9f093a2fe0_scalar_summary.csv \
--baseline-only \
--run-root twindow_baseline_tests
```

Inspect:

text

baseline_equivalence_certificate.json
baseline_equivalence_by_seed.json

Required outcome:

text

40/40 seeds passed
7/7 branches passed
no nonzero maximum differences
all extended baseline horizon checks passed

Only after that should the same driver be run without --baseline-only in a fresh production run root.

Now this one:

#!/usr/bin/env python3

"""AIEF-1.4 T_window Sensitivity Study Analysis Driver v3.

Correct implementation of the prespecified estimands:

Regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}

Branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]

Cross-T topology: S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Recovery scaling: t_rec / T_window for both horizon systems

Output:

twindow_paired_regime_contrasts.csv

twindow_branch_interactions.csv

twindow_cross_condition_topology.csv

twindow_recovery_summary.csv

twindow_sensitivity_report.txt

analysis_manifest.json

This script implements the 5-stage analysis structure as specified.

"""

```
from future import annotations

import argparse

import csv

import hashlib

import json

import os

import sys

import time

from pathlib import Path

from typing import Any

import numpy as np

import pandas as pd
```

```
================================================================================
Configuration
================================================================================

ANALYSIS_VERSION = "v3.0"

STUDY_NAME = "AIEF_1_4_Twindow_sensitivity"
Expected values (for validation)

EXPECTED_N_SEEDS = 40

EXPECTED_RHO_VALUES = (0.5, 0.75, 1.0, 1.5, 2.0)

EXPECTED_BRANCHES = ("continue", "reset_ema", "reset_B", "future_shuffle")
Primary outcomes - FIXED-PHYSICAL HORIZONS (corrected from legacy)

PRIMARY_OUTCOMES = (

text

"Weighted_similarity_late_fixed_physical",

"Retention_contrast_late_fixed_physical",

"P_mean_late_fixed_physical",

)
Secondary outcomes - NORMALIZED EMA HORIZONS

SECONDARY_NORMALIZED_OUTCOMES = (

text

"Weighted_similarity_late_normalized_EMA",
```

"Retention_contrast_late_normalized_EMA",

"P_mean_late_normalized_EMA",

)
Expected counts

EXPECTED_N_CONDITIONS = EXPECTED_N_SEEDS * len(EXPECTED_RHO_VALUES)

EXPECTED_N_ROWS = EXPECTED_N_CONDITIONS * len(EXPECTED_BRANCHES)

EXPECTED_N_REGIME_CONTRASTS = EXPECTED_N_SEEDS
(len(EXPECTED_RHO_VALUES) - 1) len(PRIMARY_OUTCOMES)

EXPECTED_N_DID = EXPECTED_N_SEEDS (len(EXPECTED_RHO_VALUES) - 1)
(len(EXPECTED_BRANCHES) - 1) * len(PRIMARY_OUTCOMES)

EXPECTED_N_TOPOLOGY = EXPECTED_N_SEEDS * (len(EXPECTED_RHO_VALUES) - 1)
Non-continue branches for DID

NON_CONTINUE_BRANCHES = ("reset_ema", "reset_B", "future_shuffle")
Matrix keys needed for cross-T topology

MATRIX_KEYS = ("B_final",)
==========
====
Utility Functions
==========
====

def load_csv_atomic(path: str | Path) -> pd.DataFrame:

text

"""Load CSV file atomically with validation."""

```
path = Path(path)



# Use pandas with explicit dtypes

df = [pd.read](http://pd.read)_csv(

path,

dtype={"seed": str, "branch": str}, # Keep seeds as strings

)



# Convert numeric columns, coercing errors to NaN

for col in df.columns:

if col not in ("seed", "seed_index", "branch", "rho"):

try:

df[col] = [pd.to](http://pd.to)_numeric(df[col], errors="coerce")

except Exception:

pass



return df
```

```
def load_npz_selective(path: str | Path, keys: list[str] | None = None) -> dict[str, Any]:

text

"""Load only specified keys from NPZ file to save memory."""

path = Path(path)

payload: dict[str, Any] = {}



if not path.exists():

print(f"Warning: NPZ file not found: {path}")

return payload



try:

with np.load(str(path), allow_pickle=False) as data:

if keys is None:

keys = list(data.files)

for key in keys:

if key in data:

value = data[key]
```

payload[key] = value.item() if value.ndim == 0 else value.copy()

except Exception as e:

print(f"Warning: Could not load NPZ {path}: {e}")

return payload

def save_csv_atomic(path: str | Path, df: pd.DataFrame) -> None:

text

"""Save CSV atomically with fsync."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

df.to_csv(temp_path, index=False)

temp_path.replace(path)

fsync the directory to ensure the replace is visible

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def save_text_atomic(path: str | Path, text: str) -> None:
text
"""Save text file atomically with fsync."""
path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)



temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")
try:
with open(temp_path, "w", encoding="utf-8") as f:
f.write(text)
```

```
f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

# fsync the directory

dir_fd = [os.open](http://os.open)(str(path.parent), os.O_RDONLY)

try:

os.fsync(dir_fd)

finally:

os.close(dir_fd)

finally:

if temp_path.exists():

temp_path.unlink()

def save_json_atomic(path: str | Path, data: dict[str, Any]) -> None:

text

"""Save JSON atomically with fsync, handling NaN values."""

path = Path(path)

path.parent.mkdir(parents=True, exist_ok=True)
```

```
# Convert NaN and infinity to None for JSON serialization

# Recursively handles nested structures including numpy arrays

def sanitize(obj: Any) -&gt; Any:

if isinstance(obj, dict):

return {k: sanitize(v) for k, v in obj.items()}

elif isinstance(obj, list):

return [sanitize(v) for v in obj]

elif isinstance(obj, tuple):

return tuple(sanitize(v) for v in obj)

elif isinstance(obj, float):

if np.isnan(obj) or np.isinf(obj):

return None

return obj

elif isinstance(obj, (np.integer, np.floating)):

val = obj.item()

if isinstance(val, float) and (np.isnan(val) or np.isinf(val)):

return None
```

return val

elif isinstance(obj, np.ndarray):

Convert to list and recursively sanitize to preserve structure

return sanitize(obj.tolist())

else:

return obj

temp_path = path.with_suffix(path.suffix + f".tmp-{os.getpid()}-{time.time_ns()}")

try:

with open(temp_path, "w", encoding="utf-8") as f:

json.dump(sanitize(data), f, indent=2, sort_keys=True, allow_nan=False)

f.write("\n")

f.flush()

os.fsync(f.fileno())

temp_path.replace(path)

fsync the directory

dir_fd = os.open(str(path.parent), os.O_RDONLY)

```
try:
os.fsync(dir_fd)
finally:
os.close(dir_fd)
finally:
if temp_path.exists():
temp_path.unlink()
def file_sha256(path: str | Path) -> str:
text
"""Compute SHA-256 hash of a file."""
import hashlib as hl
digest = hl.sha256()
path = Path(path)
if not path.exists():
return "FILE_NOT_FOUND"
with open(path, "rb") as f:
for block in iter(lambda: [f.read](http://f.read)(1024 * 1024), b""):
digest.update(block)
```

```
return digest.hexdigest()

============================================================================
====
Metric Functions
============================================================================
====

def first_crossing_time(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find first time when values cross above threshold.



Returns (first_crossing_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)



# Handle NaN values by treating them as below threshold

above_threshold = (np.isfinite(values) &amp;amp; (values &amp;gt;= threshold))



crossing_indices = np.flatnonzero(above_threshold)
```

if len(crossing_indices) > 0:

return float(times[crossing_indices[0]]), True

else:

return float('nan'), False

def first_time_sustained_for_rest(times: np.ndarray, values: np.ndarray, threshold: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for rest of trajectory.

Returns (sustained_time, observed).

"""

times = np.asarray(times, dtype=float)

values = np.asarray(values, dtype=float)

Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) & (values >= threshold))

Check from the end backwards: find first index where all subsequent values are above threshold

suffix_all = np.logical_and.accumulate(condition[::-1])[::-1]

sustained_indices = np.flatnonzero(suffix_all)

if len(sustained_indices) > 0:

return float(times[sustained_indices[0]]), True

else:

return float('nan'), False

def first_time_held_for_duration(times: np.ndarray, values: np.ndarray, threshold: float, duration: float) -> tuple[float, bool]:

text

"""Find earliest time after which values stay above threshold for at least the specified duration.

Returns (held_time, observed).

"""

times = np.asarray(times, dtype=float)

```
values = np.asarray(values, dtype=float)



if len(times) &lt; 2:

return float('nan'), False



# Calculate sample step

sample_step = float(times[1] - times[0])

if sample_step &lt;= 0:

return float('nan'), False



# Required number of consecutive samples

required_samples = max(1, int(np.ceil(duration / sample_step)))



# Handle NaN values by treating them as below threshold

condition = (np.isfinite(values) &amp; (values &gt;= threshold))
```

Use sliding window to find runs of sufficient length

held_time = float('nan')

observed = False

for i in range(len(condition) - required_samples + 1):

window = condition[i:i + required_samples]

if np.all(window):

held_time = float(times[i])

observed = True

break

return held_time, observed

def trapezoidal_auc_deficit(times: np.ndarray, reference: np.ndarray, target: np.ndarray) -> float:

text

"""Calculate one-sided AUC deficit: integral of max(0, reference - target) using trapezoidal rule.

Handles NaN values by treating them as zero for the deficit calculation.

"""

times = np.asarray(times, dtype=float)

reference = np.asarray(reference, dtype=float)

target = np.asarray(target, dtype=float)

if len(times) < 2:

return float('nan')

Calculate one-sided deficit: max(0, reference - target)

Handle NaN by replacing with 0

ref_safe = np.where(np.isfinite(reference), reference, 0.0)

tgt_safe = np.where(np.isfinite(target), target, 0.0)

deficit = np.maximum(0.0, ref_safe - tgt_safe)

Trapezoidal integration

return float(np.trapz(deficit, x=times))

def cosine_similarity(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute cosine similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

norm_x = np.linalg.norm(x)

norm_y = np.linalg.norm(y)

if norm_x <= 1e-12 or norm_y <= 1e-12:

return float('nan')

return float(np.dot(x, y) / (norm_x * norm_y))

def jaccard_similarity(B1: np.ndarray, B2: np.ndarray, threshold: float) -> float:

text

"""Compute Jaccard similarity between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges] > threshold

y = B2[edges] > threshold

union = np.count_nonzero(x | y)

if union == 0:

return float('nan')

return float(np.count_nonzero(x & y) / union)

def normalized_distance(B1: np.ndarray, B2: np.ndarray) -> float:

text

"""Compute normalized L2 distance between two B matrices over upper triangle edges."""

edges = np.triu_indices(B1.shape[0], k=1)

x = B1[edges]

y = B2[edges]

diff = x - y

norm_diff = np.linalg.norm(diff)

norm_y = np.linalg.norm(y)

if norm_y <= 1e-12:

return float('nan')

return float(norm_diff / norm_y)

def compute_recovery_scaling(

text

df: pd.DataFrame,

npz_path: Path,

b_c: float,

) -> pd.DataFrame:

text

"""Compute recovery scaling metrics for both horizon systems.

For each seed, rho, branch, and horizon system:

1. Load: sample_times, density_trajectory, similarity_trajectory

2. Use the same seed/rho continue branch as reference

3. Calculate condition-specific targets:

- Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

- Similarity target: 0.9 * S_late_continue (for appropriate horizon system)

4. Calculate:

- first_crossing_time: first time metric > target

- sustained_time: earliest time after which metric stays above target for rest of trajectory

- held_time: earliest time after which metric stays above target for at least T_window duration

- censoring indicators for each event type

- restricted_time: horizon for censored events, event_time for observed events

- auc_deficit: one-sided integral of (continue_trajectory - branch_trajectory) using trapezoidal rule

- t_rec / T_window: normalized recovery times

5. Output separate rows for fixed_physical and normalized_EMA horizon systems

Returns DataFrame with complete recovery metrics.

"""

Get unique seeds, rho values, and branches from the data

```
seeds = sorted(df["seed"].unique())

rho_values = sorted([float(r) for r in df["rho"].unique()])

branches = sorted(df["branch"].unique())



# Build complete list of required keys for all conditions

# We need: sample_times, density_trajectory, similarity_trajectory for all seed/rho/branch

all_keys = set()

for seed in seeds:

for rho in rho_values:

for branch in branches:

prefix = f"{branch}_seed_{seed}_rho_{rho}"

all_keys.update([

f"{prefix}_sample_times",

f"{prefix}_density_trajectory",

f"{prefix}_similarity_trajectory",

])
```

```
# Load all required data in one operation

print(f" Loading {len(all_keys)} trajectory arrays from NPZ...")

npz_data = load_npz_selective(npz_path, keys=list(all_keys))



# Check for missing keys - this should be fatal for production

missing_keys = [k for k in all_keys if k not in npz_data]

if missing_keys:

raise RuntimeError(

f"Missing trajectory keys in NPZ: {missing_keys[:10]}... "

f"({len(missing_keys)} total missing). "

f"This indicates schema mismatch or incomplete simulation."

)



recovery_rows = []



for seed in seeds:

for rho in rho_values:
```

```
# Get T_window for this rho from the data

t_window = float(df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho)]["T_window"].iloc[0])


# Get continue branch reference values for this seed/rho

cont_row = df[(df["seed"] == seed) &amp;amp; (df["rho"] == rho) &amp;amp; (df["branch"] ==
"continue")].iloc[0]


# Get reference targets for both horizon systems

# Density target: 0.9 * D_B_final_continue (fraction of edges above B_c)

density_target_continue = 0.9 * float(cont_row["D_B_final"])


# Similarity targets for both horizon systems

similarity_target_fixed = 0.9 * float(cont_row["Weighted_similarity_late_fixed_physical"])

similarity_target_normalized = 0.9 * float(cont_row["Weighted_similarity_late_normalized_EMA"])


for branch in branches:

# INCLUDE continue branch for DID calculation

# For continue: compare trajectory with itself, AUC deficit = 0

# This makes recovery DID well-defined: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]
```

```
prefix = f"{branch}_seed_{seed}_rho_{rho}"

cont_prefix = f"continue_seed_{seed}_rho_{rho}"


# Load trajectory data

try:

sample_times = np.asarray(npz_data[f"{prefix}_sample_times"], dtype=float)

density_trajectory = np.asarray(npz_data[f"{prefix}_density_trajectory"], dtype=float)

similarity_trajectory = np.asarray(npz_data[f"{prefix}_similarity_trajectory"], dtype=float)


# Load continue (reference) trajectory data

cont_sample_times = np.asarray(npz_data[f"{cont_prefix}_sample_times"], dtype=float)

cont_density_trajectory = np.asarray(npz_data[f"{cont_prefix}_density_trajectory"], dtype=float)

cont_similarity_trajectory = np.asarray(npz_data[f"{cont_prefix}_similarity_trajectory"], dtype=float)


except Exception as e:

raise RuntimeError(f"Error loading trajectory data for seed={seed}, rho={rho}, branch={branch}:
{e}")


# Validate that sample times match between branch and continue

# Use exact equality since all trajectories use the same diagnostic schedule
```

```
if not np.array_equal(sample_times, cont_sample_times):
raise RuntimeError(
f"Sample times mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(sample_times)} times, continue has {len(cont_sample_times)}"
)

# Also validate that trajectory lengths match
if len(density_trajectory) != len(cont_density_trajectory):
raise RuntimeError(
f"Density trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(density_trajectory)} points, continue has {len(cont_density_trajectory)}"
)
if len(similarity_trajectory) != len(cont_similarity_trajectory):
raise RuntimeError(
f"Similarity trajectory length mismatch for seed={seed}, rho={rho}, branch={branch}: "
f"branch has {len(similarity_trajectory)} points, continue has {len(cont_similarity_trajectory)}"
)
```

```
# Process each horizon system

for horizon_system, similarity_target in [

("fixed_physical", similarity_target_fixed),

("normalized_EMA", similarity_target_normalized),

]:

# Process density recovery

density_values = density_trajectory

density_target = density_target_continue


# Calculate first crossing for density

density_first_crossing, density_first_observed = first_crossing_time(

sample_times, density_values, density_target

)


# Calculate sustained recovery for density (stays above target for rest of trajectory)

density_sustained, density_sustained_observed = first_time_sustained_for_rest(

sample_times, density_values, density_target

)
```

```
# Calculate held recovery for density (stays above target for T_window duration)
density_held, density_held_observed = first_time_held_for_duration(
sample_times, density_values, density_target, t_window
)

# Calculate AUC deficit for density (one-sided: continue - branch)
density_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_density_trajectory, density_values
)

# Process similarity recovery
similarity_values = similarity_trajectory
similarity_target = similarity_target

# Calculate first crossing for similarity
sim_first_crossing, sim_first_observed = first_crossing_time(
sample_times, similarity_values, similarity_target
)
```

```
# Calculate sustained recovery for similarity
sim_sustained, sim_sustained_observed = first_time_sustained_for_rest(
sample_times, similarity_values, similarity_target
)

# Calculate held recovery for similarity
sim_held, sim_held_observed = first_time_held_for_duration(
sample_times, similarity_values, similarity_target, t_window
)

# Calculate AUC deficit for similarity
sim_auc_deficit = trapezoidal_auc_deficit(
sample_times, cont_similarity_trajectory, similarity_values
)

# Calculate restricted times (horizon for censored, event_time for observed)
# Use the last sample time as the horizon (not the elapsed difference)
horizon_time = float(sample_times[-1]) if len(sample_times) &gt; 0 else float('nan')
```

```
# For density metrics

density_restricted_first = density_first_crossing if density_first_observed else horizon_time

density_restricted_sustained = density_sustained if density_sustained_observed else horizon_time

density_restricted_held = density_held if density_held_observed else horizon_time


# For similarity metrics

sim_restricted_first = sim_first_crossing if sim_first_observed else horizon_time

sim_restricted_sustained = sim_sustained if sim_sustained_observed else horizon_time

sim_restricted_held = sim_held if sim_held_observed else horizon_time


# Calculate normalized times (t_rec / T_window)

# Raw observed times (conditional on recovery)

density_first_over_Twindow = density_first_crossing / t_window if np.isfinite(density_first_crossing) and t_window &gt; 0 else float('nan')

density_sustained_over_Twindow = density_sustained / t_window if np.isfinite(density_sustained) and t_window &gt; 0 else float('nan')

density_held_over_Twindow = density_held / t_window if np.isfinite(density_held) and t_window &gt; 0 else float('nan')
```

sim_first_over_Twindow = sim_first_crossing / t_window if np.isfinite(sim_first_crossing) and t_window > 0 else float('nan')

sim_sustained_over_Twindow = sim_sustained / t_window if np.isfinite(sim_sustained) and t_window > 0 else float('nan')

sim_held_over_Twindow = sim_held / t_window if np.isfinite(sim_held) and t_window > 0 else float('nan')

Censoring-aware restricted normalized times (cohort-level estimates)

These use restricted times (horizon for censored, event_time for observed) divided by T_window

density_restricted_first_over_Twindow = density_restricted_first / t_window if np.isfinite(density_restricted_first) and t_window > 0 else float('nan')

density_restricted_sustained_over_Twindow = density_restricted_sustained / t_window if np.isfinite(density_restricted_sustained) and t_window > 0 else float('nan')

density_restricted_held_over_Twindow = density_restricted_held / t_window if np.isfinite(density_restricted_held) and t_window > 0 else float('nan')

sim_restricted_first_over_Twindow = sim_restricted_first / t_window if np.isfinite(sim_restricted_first) and t_window > 0 else float('nan')

sim_restricted_sustained_over_Twindow = sim_restricted_sustained / t_window if np.isfinite(sim_restricted_sustained) and t_window > 0 else float('nan')

sim_restricted_held_over_Twindow = sim_restricted_held / t_window if np.isfinite(sim_restricted_held) and t_window > 0 else float('nan')

Add rows for density metrics

```
# Note: For continue branch, AUC deficit = 0 (comparing trajectory with itself)

# Held duration uses T_window for both horizon systems (Option B: documented below)

recovery_rows.append({

"seed": seed,

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "density",

"target_value": density_target,

"target_source": "0.9 * D_B_final_continue",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": density_first_crossing,

"first_crossing_observed": density_first_observed,

"sustained_time": density_sustained,

"sustained_observed": density_sustained_observed,
```

```
"held_time": density_held,
"held_observed": density_held_observed,
"restricted_first_crossing_time": density_restricted_first,
"restricted_sustained_time": density_restricted_sustained,
"restricted_held_time": density_restricted_held,
"auc_deficit_vs_continue": density_auc_deficit,
"first_crossing_over_Twindow": density_first_over_Twindow,
"sustained_time_over_Twindow": density_sustained_over_Twindow,
"held_time_over_Twindow": density_held_over_Twindow,
"restricted_first_over_Twindow": density_restricted_first_over_Twindow,
"restricted_sustained_over_Twindow": density_restricted_sustained_over_Twindow,
"restricted_held_over_Twindow": density_restricted_held_over_Twindow,
})

# Add rows for similarity metrics
recovery_rows.append({
"seed": seed,
```

"rho": rho,

"T_window": t_window,

"branch": branch,

"horizon_system": horizon_system,

"metric": "similarity",

"target_value": similarity_target,

"target_source": f"0.9 * Weighted_similarity_late_{horizon_system}",

"target_horizon_dependence": "horizon_independent",

"held_duration_definition": "one_current_T_window",

"auc_horizon": "full_future",

"first_crossing_time": sim_first_crossing,

"first_crossing_observed": sim_first_observed,

"sustained_time": sim_sustained,

"sustained_observed": sim_sustained_observed,

"held_time": sim_held,

"held_observed": sim_held_observed,

"restricted_first_crossing_time": sim_restricted_first,

"restricted_sustained_time": sim_restricted_sustained,

"restricted_held_time": sim_restricted_held,

"auc_deficit_vs_continue": sim_auc_deficit,

"first_crossing_over_Twindow": sim_first_over_Twindow,

"sustained_time_over_Twindow": sim_sustained_over_Twindow,

"held_time_over_Twindow": sim_held_over_Twindow,

"restricted_first_over_Twindow": sim_restricted_first_over_Twindow,

"restricted_sustained_over_Twindow": sim_restricted_sustained_over_Twindow,

"restricted_held_over_Twindow": sim_restricted_held_over_Twindow,

})

if not recovery_rows:

raise RuntimeError("No recovery scaling data computed - check NPZ contents and data structure")

Validate expected count: 40 seeds × 5 rho × 4 branches × 2 metrics × 2 horizon systems = 3200

Now includes continue branch for DID calculation

expected_count = len(seeds) *len(rho_values)* len(branches) *2* 2

if len(recovery_rows) != expected_count:

```
raise RuntimeError(
f"Expected {expected_count} recovery rows, got {len(recovery_rows)}. "
f"This may indicate a logic error in recovery calculation."
)


recovery_df = pd.DataFrame(recovery_rows)


print(f"✅ Stage 4b: Recovery Scaling - {len(recovery_rows)} recovery metrics computed")
print(f" Seeds: {recovery_df['seed'].nunique()}, Rho: {recovery_df['rho'].nunique()}, "
f"Branches: {recovery_df['branch'].nunique()}, Metrics: {recovery_df['metric'].nunique()}, "
f"Systems: {recovery_df['horizon_system'].nunique()}")


return recovery_df

================================================================================
Stage 1: Validation
================================================================================

def validate_run(
```

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

) -> dict[str, Any]:

text

"""Validate the run before analysis.

Checks:

- 800 scalar rows (40 seeds × 5 rho × 4 branches)

- 40 unique seeds

- 5 rho values per seed

- 4 branches per seed/rho

- baseline equivalence passed

- run is freezeable

- no duplicate condition keys

- all primary outcomes are present and numeric

"""

results = {

"status": "passed",

"errors": [],

"warnings": [],

}

Load manifest

try:

with open(manifest_path, "r") as f:

manifest = json.load(f)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load manifest: {e}")

return results

Validate condition status - MANDATORY for production

```
condition_status_path = Path(csv_path.parent) / "twindow_condition_status.csv"

if not condition_status_path.exists():

results["status"] = "failed"

results["errors"].append("Condition status file not found - REQUIRED for production analysis")

return results



try:

condition_df = [pd.read](http://pd.read)_csv(condition_status_path)

successful_conditions = (condition_df["status"] == "success").sum()

failed_conditions = (condition_df["status"] == "failed").sum()

total_conditions = len(condition_df)


expected_conditions = EXPECTED_N_CONDITIONS

if total_conditions != expected_conditions:

results["status"] = "failed"

results["errors"].append(

f"Condition status: expected {expected_conditions} conditions, "
```

```
f"got {total_conditions}"

)

return results


if failed_conditions &gt; 0:

results["status"] = "failed"

results["errors"].append(

f"Condition status: {failed_conditions} conditions failed, "

f"{successful_conditions} succeeded"

)

return results


# Hash the condition status file

condition_status_hash = file_sha256(condition_status_path)

results["condition_status_hash"] = condition_status_hash


except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Could not validate condition status: {e}")
```

```
return results



# Check baseline equivalence

baseline_eq = manifest.get("baseline_equivalence", {})

if baseline_eq.get("status") != "passed":

results["status"] = "failed"

results["errors"].append(

f"Baseline equivalence not passed: {baseline_eq.get('status', 'unknown')}"

)

return results



# Check freezeable status - FAIL if not freezeable

# Check both top-level and completion.freezeable, default to False

completion = manifest.get("completion", {})

freezeable = manifest.get("freezeable", completion.get("freezeable", False))



# Also check completion status
```

```
completion_status = completion.get("status", "")

n_successful = completion.get("n_conditions_successful", 0)

n_failed = completion.get("n_conditions_failed", 0)

n_expected = EXPECTED_N_CONDITIONS



# Must have all conditions successful and explicit freezeable=true

if not freezeable:

results["status"] = "failed"

results["errors"].append("Run is marked as non-freezeable")

return results



if completion_status != "completed":

results["status"] = "failed"

results["errors"].append(f"Run not completed: status={completion_status}")

return results



if n_successful != n_expected or n_failed != 0:
```

```
results["status"] = "failed"

results["errors"].append(

f"Incomplete run: {n_successful}/{n_expected} successful, {n_failed} failed"

)

return results



# Load CSV

try:

df = load_csv_atomic(csv_path)

except Exception as e:

results["status"] = "failed"

results["errors"].append(f"Cannot load CSV: {e}")

return results



# Validate row count

expected_rows = EXPECTED_N_ROWS

actual_rows = len(df)
```

```
if actual_rows != expected_rows:
results["status"] = "failed"
results["errors"].append(
f"Row count mismatch: expected {expected_rows}, got {actual_rows}"
)


# Validate unique seeds
unique_seeds = df["seed"].unique()
if len(unique_seeds) != EXPECTED_N_SEEDS:
results["status"] = "failed"
results["errors"].append(
f"Seed count mismatch: expected {EXPECTED_N_SEEDS}, got {len(unique_seeds)}"
)


# Validate rho values
unique_rho = sorted([float(r) for r in df["rho"].unique()])
expected_rho = sorted([float(r) for r in EXPECTED_RHO_VALUES])
```

```
if unique_rho != expected_rho:
results["status"] = "failed"
results["errors"].append(
f"Rho values mismatch: expected {expected_rho}, got {unique_rho}"
)


# Validate branches
unique_branches = sorted(df["branch"].unique())
expected_branches = sorted(EXPECTED_BRANCHES)
if unique_branches != expected_branches:
results["status"] = "failed"
results["errors"].append(
f"Branches mismatch: expected {expected_branches}, got {unique_branches}"
)


# Validate completeness: each seed/rho/branch combination should exist
missing_combinations = []
```

```
for seed in unique_seeds:

seed_data = df[df["seed"] == seed]

for rho in [float(r) for r in df["rho"].unique()]:

rho_data = seed_data[seed_data["rho"] == rho]

for branch in unique_branches:

if not ((rho_data["branch"] == branch).any()):

missing_combinations.append(f"seed={seed}, rho={rho}, branch={branch}")



if missing_combinations:

results["status"] = "failed"

results["errors"].append(

f"Missing {len(missing_combinations)} combinations: {missing_combinations[:5]}"

)



# Check for duplicates

duplicate_rows = df.duplicated(subset=["seed", "rho", "branch"])

if duplicate_rows.any():
```

```
results["status"] = "failed"

results["errors"].append(

f"Duplicate rows found: {duplicate_rows.sum()} duplicates"

)



# Validate primary outcomes are present and finite

for outcome in PRIMARY_OUTCOMES:

if outcome not in df.columns:

results["status"] = "failed"

results["errors"].append(f"Primary outcome '{outcome}' missing from CSV")

else:

# Check that values are numeric

non_numeric = df[outcome].apply(lambda x: not isinstance(x, (int, float, np.number)))

if non_numeric.any():

results["status"] = "failed"

results["errors"].append(

f"Primary outcome '{outcome}' has {non_numeric.sum()} non-numeric values"
```

```
)

# Check that values are finite (not NaN, inf, or -inf)
non_finite = ~np.isfinite(df[outcome])
if non_finite.any():
results["status"] = "failed"
results["errors"].append(
f"Primary outcome '{outcome}' has {non_finite.sum()} non-finite values"
)

# Get baseline equivalence certificate hash - MANDATORY for production
baseline_cert_path = Path(csv_path.parent) / "baseline_equivalence_certificate.json"
if not baseline_cert_path.exists():
results["status"] = "failed"
results["errors"].append("Baseline equivalence certificate not found - REQUIRED for production analysis")
return results
baseline_cert_hash = file_sha256(baseline_cert_path)
```

```
# Record validation info
results["validation"] = {
"n_rows": actual_rows,
"n_seeds": len(unique_seeds),
"n_rho": len(unique_rho),
"n_branches": len(unique_branches),
"n_duplicates": int(duplicate_rows.sum()),
"n_missing_combinations": len(missing_combinations),
"baseline_equivalence": baseline_eq.get("status"),
"freezeable": manifest.get("freezeable", False),
"baseline_certificate_hash": baseline_cert_hash,
}

if results["status"] == "passed":
print("✅ Stage 1: Validation PASSED")
```

```
print(f" {actual_rows} rows, {len(unique_seeds)} seeds, {len(unique_rho)} rho,
{len(unique_branches)} branches")

else:

print("❌ Stage 1: Validation FAILED")

for error in results["errors"]:

print(f" {error}")

for warning in results["warnings"]:

print(f" WARNING: {warning}")




return results

================================================================================
Stage 2: Regime Contrasts
================================================================================

def compute_regime_contrasts(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:
```

```
text

"""Compute regime contrasts: Δ_{s,ρ}^regime(Y) = Y_{s,ρ,continue} - Y_{s,1,continue}



Uses direct merge instead of pivot to avoid MultiIndex column issues.



For each seed and non-baseline rho, compute the difference between

the outcome at that rho and the outcome at rho=1 (baseline).

"""

# Filter to continue branch only

cont = df.loc[

df["branch"] == "continue",

["seed", "rho", *outcomes],

].copy()



# Get baseline (rho=1.0) values

baseline = (

cont.loc[cont["rho"] == 1.0]
```

```
.drop(columns="rho")

.rename(

columns={

outcome: f"{outcome}_baseline"

for outcome in outcomes

}

)

)



# Merge with all rho values

merged = cont.merge(

baseline,

on="seed",

how="inner",

validate="many_to_one",

)
```

```
# Filter to non-baseline rho

merged = merged.loc[merged["rho"] != 1.0]



# Compute contrasts

rows = []

for _, row in merged.iterrows():

for outcome in outcomes:

value = row[outcome]

reference = row[f"{outcome}_baseline"]



# Validate that both values are numeric

if not isinstance(value, (int, float, np.number)) or not isinstance(reference, (int, float, np.number)):

raise RuntimeError(f"Non-numeric value for {outcome} at seed={row['seed']}, rho={row['rho']}")



rows.append({

"seed": row["seed"],

"rho": row["rho"],

"outcome": outcome,
```

```
"Y_rho_continue": float(value),

"Y_baseline_continue": float(reference),

"paired_regime_contrast": float(value - reference),

})



if not rows:

raise RuntimeError("No regime contrasts computed - check input data")



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)* len(outcomes)

if len(rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} regime contrasts, got {len(rows)}"

)



contrasts_df = pd.DataFrame(rows)
```

```
print(f"✅ Stage 2: Regime Contrasts - {len(rows)} contrasts computed")

print(f" Seeds: {contrasts_df['seed'].nunique()}, Rho values: {contrasts_df['rho'].nunique()}, Outcomes: {contrasts_df['outcome'].nunique()}")




return contrasts_df

# ============================================================================
# Stage 3: Branch Difference-in-Differences
# ============================================================================

def compute_branch_did(

text

df: pd.DataFrame,

outcomes: tuple[str, ...] = PRIMARY_OUTCOMES,

) -> pd.DataFrame:

text

"""Compute branch DID: I_{s,ρ,b}(Y) = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]




For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences relative to continue branch.
```

```
Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

did_rows = []



for seed in df["seed"].unique():

seed_data = df[df["seed"] == seed]


for rho in sorted([float(r) for r in df["rho"].unique()]):

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]
```

```
for branch in NON_CONTINUE_BRANCHES:

for outcome in outcomes:

# Get Y_{s,ρ,b}

y_rho_b_match = rho_data[rho_data["branch"] == branch][outcome]

if len(y_rho_b_match) == 0:

raise RuntimeError(f"Missing data: seed={seed}, rho={rho}, branch={branch}, outcome={outcome}")

y_rho_b = float(y_rho_b_match.iloc[0])


# Get Y_{s,ρ,continue}

y_rho_c_match = rho_data[rho_data["branch"] == "continue"][outcome]

if len(y_rho_c_match) == 0:

raise RuntimeError(f"Missing continue data: seed={seed}, rho={rho}, outcome={outcome}")

y_rho_c = float(y_rho_c_match.iloc[0])


# Get Y_{s,1,b}

y_base_b_match = baseline_data[baseline_data["branch"] == branch][outcome]

if len(y_base_b_match) == 0:

raise RuntimeError(f"Missing baseline data: seed={seed}, branch={branch}, outcome={outcome}")
```

```
y_base_b = float(y_base_b_match.iloc[0])


# Get Y_{s,1,continue}

y_base_c_match = baseline_data[baseline_data["branch"] == "continue"][outcome]

if len(y_base_c_match) == 0:

raise RuntimeError(f"Missing baseline continue data: seed={seed}, outcome={outcome}")

y_base_c = float(y_base_c_match.iloc[0])


# Compute DID

branch_effect_rho = y_rho_b - y_rho_c

branch_effect_baseline = y_base_b - y_base_c

did = branch_effect_rho - branch_effect_baseline


did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,

"outcome": outcome,

"Y_rho_branch": y_rho_b,
```

```
"Y_rho_continue": y_rho_c,

"branch_effect_rho": branch_effect_rho,

"Y_baseline_branch": y_base_b,

"Y_baseline_continue": y_base_c,

"branch_effect_baseline": branch_effect_baseline,

"difference_in_differences": did,

})



# Validate expected count

expected_count = EXPECTED_N_SEEDS *(len(EXPECTED_RHO_VALUES) - 1)*
len(NON_CONTINUE_BRANCHES) * len(outcomes)

if len(did_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} DID values, got {len(did_rows)}"

)



did_df = pd.DataFrame(did_rows)
```

print(f"✅ Stage 3: Branch DID - {len(did_rows)} DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, Branches: {did_df['branch'].nunique()}")

return did_df

==
Stage 4: Cross-T Topology and Recovery
==

def build_continue_b_final_key(seed: str, rho: float) -> str:

text

"""Build canonical key for B_final matrix."""

return f"continue_seed_{seed}_rho_{rho}_B_final"

def compute_cross_t_topology(

text

npz_path: Path,

df: pd.DataFrame,

seeds: list[str],

rho_values: list[float],

b_c: float,

) -> pd.DataFrame:

text

"""Compute cross-T topology metrics.

For each seed and non-baseline rho, compute:

- S_{s,ρ} = cos(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- J_{s,ρ} = Jaccard(B_final_{s,ρ,continue}, B_final_{s,1,continue})

- D_{s,ρ} = normalized_L2_distance(B_final_{s,ρ,continue}, B_final_{s,1,continue})

Requires exactly EXPECTED_N_TOPOLOGY rows (40 seeds × 4 non-baseline rho = 160).

Any missing matrix is a schema failure.

"""

Build list of exact keys to load

keys_to_load = []

for seed in seeds:

```
for rho in rho_values:

keys_to_load.append(build_continue_b_final_key(seed, rho))



# Load only the required B_final matrices

npz_data = load_npz_selective(npz_path, keys=keys_to_load)



topology_rows = []

missing_keys = []



for seed in seeds:

for rho in rho_values:

if rho == 1.0:

continue # Skip baseline



# Get B_final for this (seed, rho, continue)

rho_key = build_continue_b_final_key(seed, rho)

baseline_key = build_continue_b_final_key(seed, 1.0)
```

```
if rho_key not in npz_data:
missing_keys.append(rho_key)
continue
if baseline_key not in npz_data:
missing_keys.append(baseline_key)
continue

B_rho = npz_data[rho_key]
B_baseline = npz_data[baseline_key]

# Compute metrics
cos_sim = cosine_similarity(B_rho, B_baseline)
jaccard = jaccard_similarity(B_rho, B_baseline, threshold=b_c)
norm_dist = normalized_distance(B_rho, B_baseline)

topology_rows.append({
"seed": seed,
"rho": rho,
"cross_T_cosine": cos_sim,
```

```
"cross_T_Jaccard": jaccard,

"cross_T_normalized_distance": norm_dist,

})



# Validate we have all expected comparisons

expected_count = EXPECTED_N_TOPOLOGY

if len(topology_rows) != expected_count:

raise RuntimeError(

f"Expected {expected_count} topology comparisons, got {len(topology_rows)}. "

f"Missing keys: {missing_keys[:10]}"

)



topology_df = pd.DataFrame(topology_rows)



print(f"✅ Stage 4: Cross-T Topology - {len(topology_rows)} topology comparisons computed")

print(f" Seeds: {topology_df['seed'].nunique()}, Rho: {topology_df['rho'].nunique()}")
```

return topology_df

def compute_recovery_did(

text

recovery: pd.DataFrame,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> pd.DataFrame:

text

"""Compute recovery interaction DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

For each seed, non-baseline rho, and non-continue branch, compute the

difference-in-differences for recovery outcomes relative to continue branch.

Uses restricted times (censoring-aware) and normalized t/T_window as primary outcomes.

Skips rho=1.0 (baseline) as DID should be zero by construction.

"""

```
if recovery.empty:

return pd.DataFrame()



# Create seeded RNG for reproducibility

if bootstrap_seed is None:

raise ValueError("bootstrap_seed is required")

rng = np.random.default_rng(bootstrap_seed)



did_rows = []



seeds = sorted(recovery["seed"].unique())

rho_values = sorted([float(r) for r in recovery["rho"].unique()])

branches = sorted(recovery["branch"].unique())

horizon_systems = sorted(recovery["horizon_system"].unique())

metrics = sorted(recovery["metric"].unique())
```

```
# Recovery outcomes to compute DID for

# Prioritize censoring-aware restricted outcomes as primary

recovery_outcomes = [

"restricted_first_crossing_time",

"restricted_sustained_time",

"restricted_held_time",

"restricted_first_over_Twindow",

"restricted_sustained_over_Twindow",

"restricted_held_over_Twindow",

"auc_deficit_vs_continue",

"first_crossing_over_Twindow",

"sustained_time_over_Twindow",

"held_time_over_Twindow",

]



# Also compute DID for recovery indicators

indicator_outcomes = [
```

```
"first_crossing_observed",

"sustained_observed",

"held_observed",

]



for seed in seeds:

seed_data = recovery[recovery["seed"] == seed]


for rho in rho_values:

# Skip baseline rho=1.0

if rho == 1.0:

continue


rho_data = seed_data[seed_data["rho"] == rho]


# Get baseline (rho=1.0) data for this seed

baseline_data = seed_data[seed_data["rho"] == 1.0]


for branch in branches:
```

```
if branch == "continue":

continue


for horizon_system in horizon_systems:

for metric in metrics:

# Compute DID for each recovery outcome

for outcome in recovery_outcomes:

# Get values: R_{s,ρ,b}, R_{s,ρ,c}, R_{s,1,b}, R_{s,1,c}


# R_{s,ρ,b}

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,ρ,c} (continue branch at same rho)

r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;
```

```
(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


# R_{s,1,b} (baseline rho=1.0, same branch)

r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


# R_{s,1,c} (baseline rho=1.0, continue)

r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]
```

```
if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or
len(r_base_c_match) == 0:

# Skip if any data is missing

continue



r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])



# Compute DID: [R_{s,ρ,b} - R_{s,ρ,c}] - [R_{s,1,b} - R_{s,1,c}]

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline



did_rows.append({

"seed": seed,

"rho": rho,

"branch": branch,
```

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})

Compute DID for recovery indicators (binary outcomes)

for outcome in indicator_outcomes:

Get values

r_rho_b_match = rho_data[

(rho_data["branch"] == branch) &

(rho_data["horizon_system"] == horizon_system) &

```
(rho_data["metric"] == metric)

][outcome]


r_rho_c_match = rho_data[

(rho_data["branch"] == "continue") &amp;amp;

(rho_data["horizon_system"] == horizon_system) &amp;amp;

(rho_data["metric"] == metric)

][outcome]


r_base_b_match = baseline_data[

(baseline_data["branch"] == branch) &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)

][outcome]


r_base_c_match = baseline_data[

(baseline_data["branch"] == "continue") &amp;amp;

(baseline_data["horizon_system"] == horizon_system) &amp;amp;

(baseline_data["metric"] == metric)
```

][outcome]

if len(r_rho_b_match) == 0 or len(r_rho_c_match) == 0 or len(r_base_b_match) == 0 or len(r_base_c_match) == 0:

continue

r_rho_b = float(r_rho_b_match.iloc[0])

r_rho_c = float(r_rho_c_match.iloc[0])

r_base_b = float(r_base_b_match.iloc[0])

r_base_c = float(r_base_c_match.iloc[0])

For binary outcomes, use difference in proportions

branch_effect_rho = r_rho_b - r_rho_c

branch_effect_baseline = r_base_b - r_base_c

did = branch_effect_rho - branch_effect_baseline

did_rows.append({

"seed": seed,

"rho": rho,

```
"branch": branch,

"horizon_system": horizon_system,

"metric": metric,

"outcome": outcome,

"R_rho_branch": r_rho_b,

"R_rho_continue": r_rho_c,

"branch_effect_rho": branch_effect_rho,

"R_baseline_branch": r_base_b,

"R_baseline_continue": r_base_c,

"branch_effect_baseline": branch_effect_baseline,

"recovery_difference_in_differences": did,

})



if not did_rows:

return pd.DataFrame()



did_df = pd.DataFrame(did_rows)
```

```
print(f"✅ Stage 4c: Recovery DID - {len(did_rows)} recovery DID values computed")

print(f" Seeds: {did_df['seed'].nunique()}, Rho: {did_df['rho'].nunique()}, "

f"Branches: {did_df['branch'].nunique()}, Systems: {did_df['horizon_system'].nunique()}")



return did_df

def compute_seed_level_summaries(

text

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

recovery: pd.DataFrame | None = None,

n_bootstrap: int = 1000,

bootstrap_seed: int | None = None,

) -> dict[str, pd.DataFrame]:

text

"""Compute seed-level summaries with bootstrap confidence intervals.
```

For each outcome and rho, compute:

- Mean contrast

- Std error

- 95% bootstrap CI

Uses seeded bootstrap for reproducibility.

"""

summaries = {}

Create seeded RNG for bootstrap

if bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required for reproducible analysis")

rng = np.random.default_rng(bootstrap_seed)

Regime contrast summaries

if not regime_contrasts.empty:

```
# Use named aggregation to avoid MultiIndex columns

regime_summary = regime_contrasts.groupby(

["rho", "outcome"],

as_index=False,

).agg(

mean=("paired_regime_contrast", "mean"),

std=("paired_regime_contrast", "std"),

n=("paired_regime_contrast", "count"),

)


# Add bootstrap CI

for (rho, outcome), group in regime_contrasts.groupby(["rho", "outcome"]):

values = group["paired_regime_contrast"].dropna().values

if len(values) &gt; 0:

# Seeded bootstrap

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)
```

```
ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = regime_summary[(regime_summary["rho"] == rho) &amp;amp; (regime_summary["outcome"] == outcome)].index

regime_summary.loc[idx, "ci_lower"] = ci_lower

regime_summary.loc[idx, "ci_upper"] = ci_upper


summaries["regime_contrasts"] = regime_summary

print(f"✅ Stage 5: Regime Contrast Summaries - {len(regime_summary)} groups")



# Branch DID summaries

if not branch_did.empty:

did_summary = branch_did.groupby(

["rho", "branch", "outcome"],

as_index=False,

).agg(

mean=("difference_in_differences", "mean"),

std=("difference_in_differences", "std"),
```

```
n=("difference_in_differences", "count"),

)


# Add bootstrap CI

for (rho, branch, outcome), group in branch_did.groupby(["rho", "branch", "outcome"]):

values = group["difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng.integers(0, len(values), size=(n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)


ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = did_summary[(did_summary["rho"] == rho) &amp;amp; (did_summary["branch"] == branch) &amp;amp;
(did_summary["outcome"] == outcome)].index

did_summary.loc[idx, "ci_lower"] = ci_lower

did_summary.loc[idx, "ci_upper"] = ci_upper


summaries["branch_did"] = did_summary
```

```
print(f"✅ Stage 5: Branch DID Summaries - {len(did_summary)} groups")



# Topology summaries

if not topology.empty:

# Flatten the aggregation to avoid MultiIndex columns

topology_summary = topology.groupby("rho", as_index=False).agg(

cosine_mean=("cross_T_cosine", "mean"),

cosine_std=("cross_T_cosine", "std"),

cosine_n=("cross_T_cosine", "count"),

jaccard_mean=("cross_T_Jaccard", "mean"),

jaccard_std=("cross_T_Jaccard", "std"),

jaccard_n=("cross_T_Jaccard", "count"),

distance_mean=("cross_T_normalized_distance", "mean"),

distance_std=("cross_T_normalized_distance", "std"),

distance_n=("cross_T_normalized_distance", "count"),

)



# Add bootstrap CI for topology metrics (seed-level bootstrap by rho)
```

```
for rho, group in topology.groupby("rho"):

# Bootstrap cosine similarity

cosine_values = group["cross_T_cosine"].dropna().values

if len(cosine_values) &gt; 0:

indices = rng.integers(0, len(cosine_values), size=(n_bootstrap, len(cosine_values)))

bootstrap_means = np.mean(cosine_values[indices], axis=1)

cosine_ci_lower = float(np.percentile(bootstrap_means, 2.5))

cosine_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "cosine_ci_lower"] = cosine_ci_lower

topology_summary.loc[idx, "cosine_ci_upper"] = cosine_ci_upper


# Bootstrap Jaccard similarity

jaccard_values = group["cross_T_Jaccard"].dropna().values

if len(jaccard_values) &gt; 0:

indices = rng.integers(0, len(jaccard_values), size=(n_bootstrap, len(jaccard_values)))

bootstrap_means = np.mean(jaccard_values[indices], axis=1)
```

```
jaccard_ci_lower = float(np.percentile(bootstrap_means, 2.5))

jaccard_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "jaccard_ci_lower"] = jaccard_ci_lower

topology_summary.loc[idx, "jaccard_ci_upper"] = jaccard_ci_upper


# Bootstrap normalized distance

distance_values = group["cross_T_normalized_distance"].dropna().values

if len(distance_values) &gt; 0:

indices = rng.integers(0, len(distance_values), size=(n_bootstrap, len(distance_values)))

bootstrap_means = np.mean(distance_values[indices], axis=1)

distance_ci_lower = float(np.percentile(bootstrap_means, 2.5))

distance_ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = topology_summary[topology_summary["rho"] == rho].index

topology_summary.loc[idx, "distance_ci_lower"] = distance_ci_lower

topology_summary.loc[idx, "distance_ci_upper"] = distance_ci_upper
```

```
summaries["topology"] = topology_summary

print(f"✅ Stage 5: Topology Summaries - {len(topology_summary)} rho values")



# Recovery scaling summaries with bootstrap CI

if recovery is not None and not recovery.empty:

# Group by metric, horizon_system, branch, rho and compute summaries

# Include both raw event times AND censoring-aware restricted times

recovery_summary = recovery.groupby(

["rho", "branch", "horizon_system", "metric"],

as_index=False,

).agg(

# Raw event times (conditional on observed recovery)

mean_first_crossing=("first_crossing_time", "mean"),

std_first_crossing=("first_crossing_time", "std"),

n_first_crossing=("first_crossing_time", "count"),

mean_sustained=("sustained_time", "mean"),

std_sustained=("sustained_time", "std"),
```

```
n_sustained=("sustained_time", "count"),

mean_held=("held_time", "mean"),

std_held=("held_time", "std"),

n_held=("held_time", "count"),

mean_auc_deficit=("auc_deficit_vs_continue", "mean"),

std_auc_deficit=("auc_deficit_vs_continue", "std"),

n_auc_deficit=("auc_deficit_vs_continue", "count"),


# Censoring-aware restricted times (cohort-level estimates)

mean_restricted_first=("restricted_first_crossing_time", "mean"),

std_restricted_first=("restricted_first_crossing_time", "std"),

n_restricted_first=("restricted_first_crossing_time", "count"),

mean_restricted_sustained=("restricted_sustained_time", "mean"),

std_restricted_sustained=("restricted_sustained_time", "std"),

n_restricted_sustained=("restricted_sustained_time", "count"),

mean_restricted_held=("restricted_held_time", "mean"),

std_restricted_held=("restricted_held_time", "std"),
```

```
n_restricted_held=("restricted_held_time", "count"),


# Normalized times (t/T_window) - raw observed (conditional on recovery)
mean_first_over_Twindow=("first_crossing_over_Twindow", "mean"),
std_first_over_Twindow=("first_crossing_over_Twindow", "std"),
n_first_over_Twindow=("first_crossing_over_Twindow", "count"),
mean_sustained_over_Twindow=("sustained_time_over_Twindow", "mean"),
std_sustained_over_Twindow=("sustained_time_over_Twindow", "std"),
n_sustained_over_Twindow=("sustained_time_over_Twindow", "count"),
mean_held_over_Twindow=("held_time_over_Twindow", "mean"),
std_held_over_Twindow=("held_time_over_Twindow", "std"),
n_held_over_Twindow=("held_time_over_Twindow", "count"),


# Censoring-aware restricted normalized times (cohort-level estimates)
mean_restricted_first_over_Twindow=("restricted_first_over_Twindow", "mean"),
std_restricted_first_over_Twindow=("restricted_first_over_Twindow", "std"),
n_restricted_first_over_Twindow=("restricted_first_over_Twindow", "count"),
mean_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "mean"),
```

```
std_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "std"),

n_restricted_sustained_over_Twindow=("restricted_sustained_over_Twindow", "count"),

mean_restricted_held_over_Twindow=("restricted_held_over_Twindow", "mean"),

std_restricted_held_over_Twindow=("restricted_held_over_Twindow", "std"),

n_restricted_held_over_Twindow=("restricted_held_over_Twindow", "count"),


# Recovery probabilities

observed_rate_first=("first_crossing_observed", "mean"),

observed_rate_sustained=("sustained_observed", "mean"),

observed_rate_held=("held_observed", "mean"),

)


# Add bootstrap CI for recovery metrics

for (rho, branch, horizon_system, metric), group in recovery.groupby(["rho", "branch",
"horizon_system", "metric"]):

# Bootstrap first crossing time (conditional on observed)

first_crossing_values = group["first_crossing_time"].dropna().values

if len(first_crossing_values) &gt; 0:

indices = rng.integers(0, len(first_crossing_values), size=(n_bootstrap, len(first_crossing_values)))
```

```
bootstrap_means = np.nanmean(first_crossing_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_crossing"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_crossing"] = ci_upper


# Bootstrap restricted first crossing time (cohort-level, includes censored)

restricted_first_values = group["restricted_first_crossing_time"].dropna().values

if len(restricted_first_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_values), size=(n_bootstrap, len(restricted_first_values)))

bootstrap_means = np.nanmean(restricted_first_values[indices], axis=1)
```

```
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first"] = ci_upper


# Bootstrap restricted sustained time

restricted_sustained_values = group["restricted_sustained_time"].dropna().values

if len(restricted_sustained_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_values), size=(n_bootstrap,
len(restricted_sustained_values)))

bootstrap_means = np.nanmean(restricted_sustained_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
```

```
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained"] = ci_upper


# Bootstrap restricted held time

restricted_held_values = group["restricted_held_time"].dropna().values

if len(restricted_held_values) &amp;gt; 0:

indices = rng.integers(0, len(restricted_held_values), size=(n_bootstrap, len(restricted_held_values)))

bootstrap_means = np.nanmean(restricted_held_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_held"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held"] = ci_upper



# Bootstrap normalized first crossing time over T_window

first_over_Twindow_values = group["first_crossing_over_Twindow"].dropna().values

if len(first_over_Twindow_values) &amp;gt; 0:

indices = rng.integers(0, len(first_over_Twindow_values), size=(n_bootstrap,
len(first_over_Twindow_values)))

bootstrap_means = np.nanmean(first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))
```

```
idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_first_over_Twindow"] = ci_upper


# Bootstrap normalized sustained time over T_window

sustained_over_Twindow_values = group["sustained_time_over_Twindow"].dropna().values

if len(sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(sustained_over_Twindow_values), size=(n_bootstrap,
len(sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[
```

```
(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_sustained_over_Twindow"] = ci_upper


# Bootstrap normalized held time over T_window

held_over_Twindow_values = group["held_time_over_Twindow"].dropna().values

if len(held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(held_over_Twindow_values), size=(n_bootstrap,
len(held_over_Twindow_values)))

bootstrap_means = np.nanmean(held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;
```

```
(recovery_summary["branch"] == branch) &amp;amp;
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
(recovery_summary["metric"] == metric)
].index
recovery_summary.loc[idx, "ci_lower_held_over_Twindow"] = ci_lower
recovery_summary.loc[idx, "ci_upper_held_over_Twindow"] = ci_upper

# Bootstrap AUC deficit
auc_values = group["auc_deficit_vs_continue"].dropna().values
if len(auc_values) &gt; 0:
indices = rng.integers(0, len(auc_values), size=(n_bootstrap, len(auc_values)))
bootstrap_means = np.nanmean(auc_values[indices], axis=1)
ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))
ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))

idx = recovery_summary[
(recovery_summary["rho"] == rho) &amp;amp;
(recovery_summary["branch"] == branch) &amp;amp;
```

```
(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_auc_deficit"] = ci_lower

recovery_summary.loc[idx, "ci_upper_auc_deficit"] = ci_upper


# Bootstrap restricted normalized first crossing time over T_window

restricted_first_over_Twindow_values = group["restricted_first_over_Twindow"].dropna().values

if len(restricted_first_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_first_over_Twindow_values), size=(n_bootstrap,
len(restricted_first_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_first_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;
```

```
(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_restricted_first_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_first_over_Twindow"] = ci_upper



# Bootstrap restricted normalized sustained time over T_window

restricted_sustained_over_Twindow_values =
group["restricted_sustained_over_Twindow"].dropna().values

if len(restricted_sustained_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_sustained_over_Twindow_values), size=(n_bootstrap,
len(restricted_sustained_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_sustained_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))



idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_sustained_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_sustained_over_Twindow"] = ci_upper


# Bootstrap restricted normalized held time over T_window

restricted_held_over_Twindow_values = group["restricted_held_over_Twindow"].dropna().values

if len(restricted_held_over_Twindow_values) &gt; 0:

indices = rng.integers(0, len(restricted_held_over_Twindow_values), size=(n_bootstrap,
len(restricted_held_over_Twindow_values)))

bootstrap_means = np.nanmean(restricted_held_over_Twindow_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_restricted_held_over_Twindow"] = ci_lower

recovery_summary.loc[idx, "ci_upper_restricted_held_over_Twindow"] = ci_upper


# Bootstrap recovery probabilities (observed rates)

# First crossing observed rate

first_observed_values = group["first_crossing_observed"].dropna().values

if len(first_observed_values) &gt; 0:

indices = rng.integers(0, len(first_observed_values), size=(n_bootstrap, len(first_observed_values)))

bootstrap_means = np.nanmean(first_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)
```

```
].index

recovery_summary.loc[idx, "ci_lower_observed_rate_first"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_first"] = ci_upper


# Sustained observed rate

sustained_observed_values = group["sustained_observed"].dropna().values

if len(sustained_observed_values) &gt; 0:

indices = rng.integers(0, len(sustained_observed_values), size=(n_bootstrap,
len(sustained_observed_values)))

bootstrap_means = np.nanmean(sustained_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index
```

```
recovery_summary.loc[idx, "ci_lower_observed_rate_sustained"] = ci_lower

recovery_summary.loc[idx, "ci_upper_observed_rate_sustained"] = ci_upper


# Held observed rate

held_observed_values = group["held_observed"].dropna().values

if len(held_observed_values) &gt; 0:

indices = rng.integers(0, len(held_observed_values), size=(n_bootstrap, len(held_observed_values)))

bootstrap_means = np.nanmean(held_observed_values[indices], axis=1)

ci_lower = float(np.nanpercentile(bootstrap_means, 2.5))

ci_upper = float(np.nanpercentile(bootstrap_means, 97.5))


idx = recovery_summary[

(recovery_summary["rho"] == rho) &amp;amp;

(recovery_summary["branch"] == branch) &amp;amp;

(recovery_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_summary["metric"] == metric)

].index

recovery_summary.loc[idx, "ci_lower_observed_rate_held"] = ci_lower
```

```
recovery_summary.loc[idx, "ci_upper_observed_rate_held"] = ci_upper


summaries["recovery"] = recovery_summary

print(f"✅ Stage 5: Recovery Summaries - {len(recovery_summary)} groups")



return summaries

# ============================================================================
# Report Generation
# ============================================================================

def generate_text_report(

text

validation: dict[str, Any],

regime_contrasts: pd.DataFrame,

branch_did: pd.DataFrame,

topology: pd.DataFrame,

regime_summaries: pd.DataFrame,

did_summaries: pd.DataFrame,

topology_summaries: pd.DataFrame,
```

```
recovery_summaries: pd.DataFrame | None = None,

regime_summaries_secondary: pd.DataFrame | None = None,

did_summaries_secondary: pd.DataFrame | None = None,

recovery_did_summary: pd.DataFrame | None = None,

) -> str:

text

"""Generate human-readable text report."""

lines = []

lines.append("=" * 80)

lines.append("AIEF-1.4 T_window Sensitivity Study - Analysis Report")

lines.append("=" * 80)

lines.append("")



# Validation summary

lines.append("VALIDATION SUMMARY")

lines.append("-" * 40)

val = validation.get("validation", {})
```

```
lines.append(f" Status: {validation.get('status', 'unknown')}")

lines.append(f" Rows: {val.get('n_rows', 0)} (expected: {EXPECTED_N_ROWS})")

lines.append(f" Seeds: {val.get('n_seeds', 0)} (expected: {EXPECTED_N_SEEDS})")

lines.append(f" Rho values: {val.get('n_rho', 0)} (expected: {len(EXPECTED_RHO_VALUES)})")

lines.append(f" Branches: {val.get('n_branches', 0)} (expected: {len(EXPECTED_BRANCHES)})")

lines.append(f" Duplicates: {val.get('n_duplicates', 0)}")

lines.append(f" Missing combinations: {val.get('n_missing_combinations', 0)}")

lines.append(f" Baseline equivalence: {val.get('baseline_equivalence', 'unknown')}")

lines.append(f" Freezeable: {val.get('freezeable', False)}")

lines.append("")



# Regime contrasts (Primary - fixed_physical horizons)

lines.append("REGIME CONTRASTS (Δ = Y_{s,ρ,continue} - Y_{s,1,continue})")

lines.append("-" * 40)

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not regime_summaries.empty:
```

```
for rho in sorted(regime_summaries["rho"].unique()):

rho_data = regime_summaries[regime_summaries["rho"] == rho]

for outcome in PRIMARY_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No regime contrast data")

lines.append("")



# Secondary regime contrasts (normalized_EMA horizons)
```

```
lines.append("SECONDARY REGIME CONTRASTS (Normalized EMA horizons)")

lines.append("-" * 40)

if regime_summaries_secondary is not None and not regime_summaries_secondary.empty:

for rho in sorted(regime_summaries_secondary["rho"].unique()):

rho_data = regime_summaries_secondary[regime_summaries_secondary["rho"] == rho]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = rho_data[rho_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:
```

```
lines.append(" No secondary regime contrast data")

lines.append("")



# Branch DID (Primary - fixed_physical horizons)

lines.append("BRANCH DIFFERENCE-IN-DIFFERENCES")

lines.append("-" * 40)

lines.append("I_{s,ρ,b} = [Y_{s,ρ,b} - Y_{s,ρ,continue}] - [Y_{s,1,b} - Y_{s,1,continue}]")

lines.append("Primary outcomes: fixed_physical horizons")

lines.append("")

if not did_summaries.empty:

for rho in sorted(did_summaries["rho"].unique()):

rho_data = did_summaries[did_summaries["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in PRIMARY_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:
```

```
mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")

else:

lines.append(" No branch DID data")

lines.append("")



# Secondary Branch DID (normalized_EMA horizons)

lines.append("SECONDARY BRANCH DIFFERENCE-IN-DIFFERENCES (Normalized EMA)")

lines.append("-" * 40)

if did_summaries_secondary is not None and not did_summaries_secondary.empty:

for rho in sorted(did_summaries_secondary["rho"].unique()):
```

```
rho_data = did_summaries_secondary[did_summaries_secondary["rho"] == rho]

for branch in sorted(rho_data["branch"].unique()):

branch_data = rho_data[rho_data["branch"] == branch]

for outcome in SECONDARY_NORMALIZED_OUTCOMES:

out_data = branch_data[branch_data["outcome"] == outcome]

if not out_data.empty:

mean_val = float(out_data["mean"].iloc[0])

std_val = float(out_data["std"].iloc[0])

n_val = int(out_data["n"].iloc[0])

ci_lower = float(out_data.get("ci_lower", float('nan')).iloc[0])

ci_upper = float(out_data.get("ci_upper", float('nan')).iloc[0])



ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" rho={rho:.2f}, branch={branch}, {outcome}: {mean_val:.4f} ± {std_val:.4f} {ci_str}
(n={n_val})")

else:

lines.append(" No secondary branch DID data")

lines.append("")
```

```
# Cross-T topology

lines.append("CROSS-T TOPOLOGY METRICS")

lines.append("-" * 40)

if not topology_summaries.empty:

for *, row in topology*summaries.iterrows():

rho = float(row["rho"])

lines.append(f" rho={rho:.2f}:")

lines.append(f" Cosine similarity: {float(row['cosine_mean']):.4f} ± {float(row['cosine_std']):.4f}
(n={int(row['cosine_n'])})")

if "cosine_ci_lower" in row and not np.isnan(float(row.get("cosine_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['cosine_ci_lower']):.4f}, {float(row['cosine_ci_upper']):.4f}]")

lines.append(f" Jaccard similarity: {float(row['jaccard_mean']):.4f} ± {float(row['jaccard_std']):.4f}
(n={int(row['jaccard_n'])})")

if "jaccard_ci_lower" in row and not np.isnan(float(row.get("jaccard_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['jaccard_ci_lower']):.4f}, {float(row['jaccard_ci_upper']):.4f}]")

lines.append(f" Normalized distance: {float(row['distance_mean']):.4f} ±
{float(row['distance_std']):.4f} (n={int(row['distance_n'])})")

if "distance_ci_lower" in row and not np.isnan(float(row.get("distance_ci_lower", float('nan')))):

lines.append(f" 95% CI: [{float(row['distance_ci_lower']):.4f}, {float(row['distance_ci_upper']):.4f}]")
```

```
else:

lines.append(" No cross-T topology data")

lines.append("")



# Recovery scaling metrics

lines.append("RECOVERY SCALING METRICS")

lines.append("-" * 40)

lines.append("Cohort-level estimates (censoring-aware) and conditional-on-recovery estimates")

lines.append("")

if recovery_summaries is not None and not recovery_summaries.empty:

for *, row in recovery*summaries.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}:")
```

```
# Cohort-level estimates (includes censored) - restricted times

lines.append(f" Cohort-level estimates (all seeds, censoring-aware):")

lines.append(f" Mean restricted first crossing: {float(row['mean_restricted_first']):.4f} ±
{float(row['std_restricted_first']):.4f} (n={int(row['n_restricted_first'])})")

if "ci_lower_restricted_first" in row and not np.isnan(float(row.get("ci_lower_restricted_first",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first']):.4f},
{float(row['ci_upper_restricted_first']):.4f}]")

lines.append(f" Mean restricted sustained: {float(row['mean_restricted_sustained']):.4f} ±
{float(row['std_restricted_sustained']):.4f} (n={int(row['n_restricted_sustained'])})")

if "ci_lower_restricted_sustained" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained']):.4f},
{float(row['ci_upper_restricted_sustained']):.4f}]")

lines.append(f" Mean restricted held: {float(row['mean_restricted_held']):.4f} ±
{float(row['std_restricted_held']):.4f} (n={int(row['n_restricted_held'])})")

if "ci_lower_restricted_held" in row and not np.isnan(float(row.get("ci_lower_restricted_held",
float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held']):.4f},
{float(row['ci_upper_restricted_held']):.4f}]")

lines.append(f" AUC deficit: {float(row['mean_auc_deficit']):.4f} ± {float(row['std_auc_deficit']):.4f}
(n={int(row['n_auc_deficit'])})")

if "ci_lower_auc_deficit" in row and not np.isnan(float(row.get("ci_lower_auc_deficit", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_auc_deficit']):.4f},
{float(row['ci_upper_auc_deficit']):.4f}]")



# Cohort-level normalized times (restricted t/T_window)

lines.append(f" Cohort-level normalized times (restricted, all seeds):")

lines.append(f" Mean restricted first/t_window: {float(row.get('mean_restricted_first_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_first_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_first_over_Twindow', 0))})")

if "ci_lower_restricted_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_first_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_first_over_Twindow']):.4f},
{float(row['ci_upper_restricted_first_over_Twindow']):.4f}]")

lines.append(f" Mean restricted sustained/t_window:
{float(row.get('mean_restricted_sustained_over_Twindow', float('nan'))):.4f} ±
{float(row.get('std_restricted_sustained_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_sustained_over_Twindow', 0))})")

if "ci_lower_restricted_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_restricted_sustained_over_Twindow']):.4f},
{float(row['ci_upper_restricted_sustained_over_Twindow']):.4f}]")

lines.append(f" Mean restricted held/t_window: {float(row.get('mean_restricted_held_over_Twindow',
float('nan'))):.4f} ± {float(row.get('std_restricted_held_over_Twindow', float('nan'))):.4f}
(n={int(row.get('n_restricted_held_over_Twindow', 0))})")

if "ci_lower_restricted_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_restricted_held_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI: [{float(row['ci_lower_restricted_held_over_Twindow']):.4f},
{float(row['ci_upper_restricted_held_over_Twindow']):.4f}]")



# Recovery probabilities with CI

lines.append(f" Recovery probabilities (with 95% bootstrap CI):")

lines.append(f" First crossing observed: {float(row['observed_rate_first']):.4f}")

if "ci_lower_observed_rate_first" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_first", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_first']):.4f},
{float(row['ci_upper_observed_rate_first']):.4f}]")

lines.append(f" Sustained recovery observed: {float(row['observed_rate_sustained']):.4f}")

if "ci_lower_observed_rate_sustained" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_sustained", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_sustained']):.4f},
{float(row['ci_upper_observed_rate_sustained']):.4f}]")

lines.append(f" Held recovery observed: {float(row['observed_rate_held']):.4f}")

if "ci_lower_observed_rate_held" in row and not
np.isnan(float(row.get("ci_lower_observed_rate_held", float('nan')))):

lines.append(f" 95% CI: [{float(row['ci_lower_observed_rate_held']):.4f},
{float(row['ci_upper_observed_rate_held']):.4f}]")



# Conditional-on-recovery estimates (observed seeds only)

lines.append(f" Conditional-on-recovery estimates (observed seeds only):")
```

```
lines.append(f" First crossing: {float(row['mean_first_crossing']):.4f} ±
{float(row['std_first_crossing']):.4f} (n={int(row['n_first_crossing'])})")

if "ci_lower_first_crossing" in row and not np.isnan(float(row.get("ci_lower_first_crossing",
float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_crossing']):.4f},
{float(row['ci_upper_first_crossing']):.4f}]")

lines.append(f" Sustained: {float(row['mean_sustained']):.4f} ± {float(row['std_sustained']):.4f}
(n={int(row['n_sustained'])})")

if "ci_lower_sustained" in row and not np.isnan(float(row.get("ci_lower_sustained", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained']):.4f},
{float(row['ci_upper_sustained']):.4f}]")

lines.append(f" Held: {float(row['mean_held']):.4f} ± {float(row['std_held']):.4f}
(n={int(row['n_held'])})")

if "ci_lower_held" in row and not np.isnan(float(row.get("ci_lower_held", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held']):.4f},
{float(row['ci_upper_held']):.4f}]")



# Conditional-on-recovery normalized times (observed seeds only)

lines.append(f" Conditional-on-recovery normalized times (observed seeds only):")

lines.append(f" First crossing/t_window: {float(row['mean_first_over_Twindow']):.4f} ±
{float(row['std_first_over_Twindow']):.4f} (n={int(row['n_first_over_Twindow'])})")

if "ci_lower_first_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_first_over_Twindow", float('nan')))):
```

```
lines.append(f" 95% CI (conditional): [{float(row['ci_lower_first_over_Twindow']):.4f},
{float(row['ci_upper_first_over_Twindow']):.4f}]")

lines.append(f" Sustained/t_window: {float(row['mean_sustained_over_Twindow']):.4f} ±
{float(row['std_sustained_over_Twindow']):.4f} (n={int(row['n_sustained_over_Twindow'])})")

if "ci_lower_sustained_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_sustained_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_sustained_over_Twindow']):.4f},
{float(row['ci_upper_sustained_over_Twindow']):.4f}]")

lines.append(f" Held/t_window: {float(row['mean_held_over_Twindow']):.4f} ±
{float(row['std_held_over_Twindow']):.4f} (n={int(row['n_held_over_Twindow'])})")

if "ci_lower_held_over_Twindow" in row and not
np.isnan(float(row.get("ci_lower_held_over_Twindow", float('nan')))):

lines.append(f" 95% CI (conditional): [{float(row['ci_lower_held_over_Twindow']):.4f},
{float(row['ci_upper_held_over_Twindow']):.4f}]")

else:

lines.append(" No recovery scaling data")

lines.append("")



# Recovery interaction DID

lines.append("RECOVERY INTERACTION DID")

lines.append("-" * 40)
```

```
lines.append("I_{s,ρ,b}^recovery = [R_{s,ρ,b} - R_{s,ρ,continue}] - [R_{s,1,b} - R_{s,1,continue}]")

lines.append("")

if recovery_did_summary is not None and not recovery_did_summary.empty:

for *, row in recovery*did_summary.iterrows():

rho = float(row["rho"])

branch = str(row["branch"])

horizon_system = str(row["horizon_system"])

metric = str(row["metric"])

outcome = str(row["outcome"])

lines.append(f" rho={rho:.2f}, branch={branch}, {horizon_system}, {metric}, {outcome}:")

mean_val = float(row["mean"])

std_val = float(row["std"])

n_val = int(row["n"])

ci_lower = float(row.get("ci_lower", float('nan')))

ci_upper = float(row.get("ci_upper", float('nan')))


ci_str = f"[{ci_lower:.4f}, {ci_upper:.4f}]" if not np.isnan(ci_lower) else "N/A"

lines.append(f" DID: {mean_val:.4f} ± {std_val:.4f} {ci_str} (n={n_val})")
```

```
else:

lines.append(" No recovery DID data")

lines.append("")



lines.append("=" * 80)

lines.append("END OF REPORT")

lines.append("=" * 80)



return "\n".join(lines)

def generate_analysis_manifest(

text

csv_path: Path,

npz_path: Path,

manifest_path: Path,

validation: dict[str, Any],

output_files: dict[str, Path],

source_path: Path,
```

```
bootstrap_seed: int,

n_bootstrap: int,

command_line: str,

) -> dict[str, Any]:

text

"""Generate analysis manifest with complete provenance."""

# Compute hashes

csv_hash = file_sha256(csv_path)

npz_hash = file_sha256(npz_path)

manifest_hash = file_sha256(manifest_path)

source_hash = file_sha256(source_path)



output_hashes = {}

for name, path in output_files.items():

output_hashes[name] = file_sha256(path)



manifest = {
```

"analysis_version": ANALYSIS_VERSION,

"timestamp": int(time.time()),

"study_name": STUDY_NAME,

"source": {

"path": str(source_path),

"sha256": source_hash,

},

"command_line": command_line,

"bootstrap": {

"seed": bootstrap_seed,

"n_iterations": n_bootstrap,

},

"input_hashes": {

"scalar_csv": csv_hash,

"branch_npz": npz_hash,

"simulation_manifest": manifest_hash,

"condition_status": validation.get("condition_status_hash", "NOT_FOUND"),

"baseline_certificate": validation.get("baseline_certificate_hash", "NOT_FOUND"),

},

"validation": validation,

"output_hashes": output_hashes,

"environment": {

"python": sys.version,

"numpy": np.__version__,

"pandas": pd.__version__,

},

"notes": {

"primary_outcomes": list(PRIMARY_OUTCOMES),

"secondary_outcomes": list(SECONDARY_NORMALIZED_OUTCOMES),

"expected_n_seeds": EXPECTED_N_SEEDS,

"expected_rho_values": [float(r) for r in EXPECTED_RHO_VALUES],

"expected_branches": list(EXPECTED_BRANCHES),

"expected_n_regime_contrasts": EXPECTED_N_REGIME_CONTRASTS,

"expected_n_did": EXPECTED_N_DID,

"expected_n_topology": EXPECTED_N_TOPOLOGY,

"horizon_definitions": {

"fixed_physical": {

"immediate": "0-17.5 time units",

"recovery": "17.5-87.5 time units",

"late": "87.5-320 time units"

},

"normalized_EMA": {

"immediate": "0-1×T_window",

"recovery": "1-5×T_window",

"late": "5×T_window to end"

}

},

"interpretation_constraints": {

"primary_estimand": "fixed_physical horizons",

"secondary_estimand": "normalized_EMA horizons",

"recovery_metrics": "t_rec/T_window for both horizon systems"

}

},

}

return manifest

==
Main Execution
==

def parse_arguments() -> argparse.Namespace:

text

parser = argparse.ArgumentParser(

description=__doc__,

formatter_class=argparse.RawDescriptionHelpFormatter,

)

```
parser.add_argument(
"run_dir",
type=str,
help="run directory containing study outputs",
)
parser.add_argument(
"--output",
type=str,
default=None,
help="output directory for analysis (default: &lt;run_dir&gt;/analysis)",
)
parser.add_argument(
"--n-bootstrap",
type=int,
default=1000,
help="number of bootstrap iterations (default: 1000)",
)
parser.add_argument(
```

```
"--bootstrap-seed",

type=int,

required=True,

help="random seed for bootstrap reproducibility (REQUIRED)",

)

parser.add_argument(

"--overwrite",

action="store_true",

default=False,

help="overwrite existing output files (default: fail if outputs exist)",

)

return parser.parse_args()

def validate_bootstrap_args(args):

text

"""Validate bootstrap arguments."""

if args.n_bootstrap &lt;= 0:

raise ValueError("--n-bootstrap must be positive")
```

```
if args.bootstrap_seed is None:

raise ValueError("--bootstrap-seed is required")

return True

def main() -> None:

text

args = parse_arguments()



# Validate bootstrap arguments

validate_bootstrap_args(args)



# Parse run directory

run_dir = Path([args.run](http://args.run)_dir)

csv_files = list(run_dir.glob(f"{STUDY_NAME}_*_scalar_summary.csv"))

npz_files = list(run_dir.glob(f"{STUDY_NAME}_*_branch_diagnostics.npz"))

manifest_files = list(run_dir.glob("manifest.json"))



if not csv_files:
```

```
print(f"No CSV file found in {run_dir}")

sys.exit(1)

if not npz_files:

print(f"No NPZ file found in {run_dir}")

sys.exit(1)

if not manifest_files:

print(f"No manifest found in {run_dir}")

sys.exit(1)



csv_path = csv_files[0]

npz_path = npz_files[0]

manifest_path = manifest_files[0]



# Set up output directory with stale-file protection

if args.output is None:

output_dir = run_dir / "analysis"

else:
```

```
output_dir = Path(args.output)



# Check for stale files in output directory

expected_outputs = [

"twindow_paired_regime_contrasts.csv",

"twindow_paired_regime_contrasts_fixed_physical.csv",

"twindow_paired_regime_contrasts_normalized_EMA.csv",

"twindow_branch_interactions.csv",

"twindow_branch_interactions_fixed_physical.csv",

"twindow_branch_interactions_normalized_EMA.csv",

"twindow_cross_condition_topology.csv",

"twindow_recovery_by_seed.csv",

"twindow_recovery_summary.csv",

"twindow_regime_summary.csv",

"twindow_branch_DID_summary.csv",

"twindow_topology_summary.csv",

"twindow_regime_secondary_summary.csv",
```

```
"twindow_branch_DID_secondary_summary.csv",

"twindow_sensitivity_report.txt",

"analysis_manifest.json",

]



stale_files = []

if output_dir.exists():

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

stale_files.append(expected_file)



if stale_files and not args.overwrite:

print(f"❌ Output directory contains stale files: {stale_files[:5]}...")

print(f" Use --overwrite to force overwrite, or specify a different --output directory")

print(f" This prevents mixing outputs from different analysis versions.")

sys.exit(1)
```

```
# If --overwrite is passed, remove ALL expected files before starting

if stale_files and args.overwrite:

print(f" Removing {len(stale_files)} stale files from output directory...")

for expected_file in expected_outputs:

file_path = output_dir / expected_file

if file_path.exists():

try:

file_path.unlink()

except Exception as e:

print(f" WARNING: Could not remove stale file {expected_file}: {e}")



output_dir.mkdir(parents=True, exist_ok=True)



# Save command line for manifest

command_line = " ".join(sys.argv)



print(f"Analyzing {run_dir}")
```

```
print(f" CSV: {csv_[path.name](http://path.name)}")

print(f" NPZ: {npz_[path.name](http://path.name)}")

print(f" Manifest: {manifest_[path.name](http://path.name)}")

print(f" Output: {output_dir}")

print(f" Bootstrap: {args.n_bootstrap} iterations, seed={args.bootstrap_seed}")

print()



# Stage 1: Validation

print("Stage 1: Validating run...")

validation = validate_run(csv_path, npz_path, manifest_path)



if validation["status"] == "failed":

print("\n❌ VALIDATION FAILED - Cannot proceed with analysis")

for error in validation["errors"]:

print(f" {error}")

for warning in validation["warnings"]:

print(f" WARNING: {warning}")
```

```
sys.exit(1)



# Load CSV data

print("\nLoading scalar data...")

df = load_csv_atomic(csv_path)



# Get study parameters from manifest

with open(manifest_path, "r") as f:

manifest = json.load(f)



seeds = [str(s) for s in manifest["randomization"]["paired_seeds"]]

rho_values = [float(r) for r in manifest["study"]["rho_values"]]

branches = manifest["protocol"]["branches"]

b_c = float(manifest["model"]["B_c"])



print(f" Loaded {len(df)} rows")

print(f" Seeds: {len(seeds)}, Rho: {len(rho_values)}, Branches: {len(branches)}")
```

```
print(f" B_c: {b_c}")



# Stage 2: Regime Contrasts (Primary outcomes - fixed_physical)

print("\nStage 2: Computing regime contrasts...")

regime_contrasts = compute_regime_contrasts(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary regime contrasts

regime_contrasts["horizon_system"] = "fixed_physical"



# Stage 2b: Regime Contrasts (Secondary normalized-EMA outcomes)

print("\nStage 2b: Computing secondary regime contrasts...")

regime_contrasts_secondary = compute_regime_contrasts(df,
SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary regime contrasts

regime_contrasts_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary regime contrasts into long format

regime_contrasts_combined = pd.concat([regime_contrasts, regime_contrasts_secondary],
ignore_index=True)



# Stage 3: Branch DID (Primary outcomes - fixed_physical)

print("\nStage 3: Computing branch difference-in-differences...")

branch_did = compute_branch_did(df, PRIMARY_OUTCOMES)



# Add horizon_system column to primary branch DID

branch_did["horizon_system"] = "fixed_physical"



# Stage 3b: Branch DID (Secondary normalized-EMA outcomes)

print("\nStage 3b: Computing secondary branch difference-in-differences...")

branch_did_secondary = compute_branch_did(df, SECONDARY_NORMALIZED_OUTCOMES)



# Add horizon_system column to secondary branch DID

branch_did_secondary["horizon_system"] = "normalized_EMA"
```

```
# Combine primary and secondary branch DID into long format
branch_did_combined = pd.concat([branch_did, branch_did_secondary], ignore_index=True)


# Stage 4: Cross-T Topology
print("\nStage 4: Computing cross-T topology metrics...")
topology = compute_cross_t_topology(npz_path, df, seeds, rho_values, b_c)


# Stage 4b: Recovery Scaling (REQUIRED for production)
print("\nStage 4b: Computing recovery scaling metrics...")
recovery = compute_recovery_scaling(df, npz_path, b_c)


# Stage 4c: Recovery Interaction DID
print("\nStage 4c: Computing recovery interaction DID...")
recovery_did = compute_recovery_did(recovery,
n_bootstrap=args.n_bootstrap,
bootstrap_seed=args.bootstrap_seed)
```

```
# Stage 5: Seed-Level Summaries

print("\nStage 5: Computing seed-level summaries...")

summaries = compute_seed_level_summaries(

regime_contrasts, branch_did, topology, recovery,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5b: Secondary normalized-EMA summaries

print("\nStage 5b: Computing secondary seed-level summaries...")

secondary_summaries = compute_seed_level_summaries(

regime_contrasts_secondary, branch_did_secondary, pd.DataFrame(), None,

n_bootstrap=args.n_bootstrap,

bootstrap_seed=args.bootstrap_seed,

)



# Stage 5c: Recovery DID summaries
```

```
print("\nStage 5c: Computing recovery DID summaries...")

recovery_did_summaries = {}

if not recovery_did.empty:

recovery_did_summary = recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"],

as_index=False,

).agg(

mean=("recovery_difference_in_differences", "mean"),

std=("recovery_difference_in_differences", "std"),

n=("recovery_difference_in_differences", "count"),

)


# Add bootstrap CI

rng_did = np.random.default_rng(args.bootstrap_seed)

for (rho, branch, horizon_system, metric, outcome), group in recovery_did.groupby(

["rho", "branch", "horizon_system", "metric", "outcome"]

):
```

```
values = group["recovery_difference_in_differences"].dropna().values

if len(values) &gt; 0:

indices = rng_did.integers(0, len(values), size=(args.n_bootstrap, len(values)))

bootstrap_means = np.mean(values[indices], axis=1)

ci_lower = float(np.percentile(bootstrap_means, 2.5))

ci_upper = float(np.percentile(bootstrap_means, 97.5))


idx = recovery_did_summary[

(recovery_did_summary["rho"] == rho) &amp;amp;

(recovery_did_summary["branch"] == branch) &amp;amp;

(recovery_did_summary["horizon_system"] == horizon_system) &amp;amp;

(recovery_did_summary["metric"] == metric) &amp;amp;

(recovery_did_summary["outcome"] == outcome)

].index

recovery_did_summary.loc[idx, "ci_lower"] = ci_lower

recovery_did_summary.loc[idx, "ci_upper"] = ci_upper


recovery_did_summaries["recovery_did"] = recovery_did_summary
```

```
print(f"✅ Stage 5c: Recovery DID Summaries - {len(recovery_did_summary)} groups")



regime_summaries = summaries.get("regime_contrasts", pd.DataFrame())

did_summaries = summaries.get("branch_did", pd.DataFrame())

topology_summaries = summaries.get("topology", pd.DataFrame())

recovery_summaries = summaries.get("recovery", pd.DataFrame())

regime_summaries_secondary = secondary_summaries.get("regime_contrasts", pd.DataFrame())

did_summaries_secondary = secondary_summaries.get("branch_did", pd.DataFrame())

recovery_did_summary = recovery_did_summaries.get("recovery_did", pd.DataFrame())



# Save outputs - construct output_files incrementally

print("\nSaving outputs...")

output_files: dict[str, Path] = {}



# Paired regime contrasts (combined primary + secondary)

regime_contrast_combined_path = output_dir / "twindow_paired_regime_contrasts.csv"

save_csv_atomic(regime_contrast_combined_path, regime_contrasts_combined)
```

```
output_files["regime_contrasts_combined"] = regime_contrast_combined_path

print(f" Saved: {regime_contrast_combined_[path.name](http://path.name)}")



# Paired regime contrasts (primary only - fixed_physical)

regime_contrast_path = output_dir / "twindow_paired_regime_contrasts_fixed_physical.csv"

save_csv_atomic(regime_contrast_path, regime_contrasts)

output_files["regime_contrasts"] = regime_contrast_path

print(f" Saved: {regime_contrast_[path.name](http://path.name)}")



# Paired regime contrasts (secondary only - normalized_EMA)

regime_contrast_secondary_path = output_dir /
"twindow_paired_regime_contrasts_normalized_EMA.csv"

save_csv_atomic(regime_contrast_secondary_path, regime_contrasts_secondary)

output_files["regime_contrasts_secondary"] = regime_contrast_secondary_path

print(f" Saved: {regime_contrast_secondary_[path.name](http://path.name)}")



# Branch DID (combined primary + secondary)

branch_did_combined_path = output_dir / "twindow_branch_interactions.csv"
```

```
save_csv_atomic(branch_did_combined_path, branch_did_combined)

output_files["branch_did_combined"] = branch_did_combined_path

print(f" Saved: {branch_did_combined_[path.name](http://path.name)}")



# Branch DID (primary only - fixed_physical)

branch_did_path = output_dir / "twindow_branch_interactions_fixed_physical.csv"

save_csv_atomic(branch_did_path, branch_did)

output_files["branch_did"] = branch_did_path

print(f" Saved: {branch_did_[path.name](http://path.name)}")



# Branch DID (secondary only - normalized_EMA)

branch_did_secondary_path = output_dir / "twindow_branch_interactions_normalized_EMA.csv"

save_csv_atomic(branch_did_secondary_path, branch_did_secondary)

output_files["branch_did_secondary"] = branch_did_secondary_path

print(f" Saved: {branch_did_secondary_[path.name](http://path.name)}")



# Cross-T topology
```

```
if not topology.empty:

topology_path = output_dir / "twindow_cross_condition_topology.csv"

save_csv_atomic(topology_path, topology)

output_files["topology"] = topology_path

print(f" Saved: {topology_[path.name](http://path.name)}")



# Recovery scaling raw output

if not recovery.empty:

recovery_raw_path = output_dir / "twindow_recovery_by_seed.csv"

save_csv_atomic(recovery_raw_path, recovery)

output_files["recovery_by_seed"] = recovery_raw_path

print(f" Saved: {recovery_raw_[path.name](http://path.name)}")



# Summaries

if not regime_summaries.empty:

regime_summary_path = output_dir / "twindow_regime_summary.csv"

save_csv_atomic(regime_summary_path, regime_summaries)
```

```
output_files["regime_summary"] = regime_summary_path

print(f" Saved: {regime_summary_[path.name](http://path.name)}")



if not did_summaries.empty:

did_summary_path = output_dir / "twindow_branch_DID_summary.csv"

save_csv_atomic(did_summary_path, did_summaries)

output_files["did_summary"] = did_summary_path

print(f" Saved: {did_summary_[path.name](http://path.name)}")



if not topology_summaries.empty:

topology_summary_path = output_dir / "twindow_topology_summary.csv"

save_csv_atomic(topology_summary_path, topology_summaries)

output_files["topology_summary"] = topology_summary_path

print(f" Saved: {topology_summary_[path.name](http://path.name)}")



# Recovery summaries (censoring-aware)

if not recovery_summaries.empty:
```

```
recovery_summary_path = output_dir / "twindow_recovery_summary.csv"

save_csv_atomic(recovery_summary_path, recovery_summaries)

output_files["recovery_summary"] = recovery_summary_path

print(f" Saved: {recovery_summary_[path.name](http://path.name)}")



# Recovery DID (raw)

if not recovery_did.empty:

recovery_did_path = output_dir / "twindow_recovery_interactions.csv"

save_csv_atomic(recovery_did_path, recovery_did)

output_files["recovery_did"] = recovery_did_path

print(f" Saved: {recovery_did_[path.name](http://path.name)}")



# Recovery DID summaries

if not recovery_did_summary.empty:

recovery_did_summary_path = output_dir / "twindow_recovery_interactions_summary.csv"

save_csv_atomic(recovery_did_summary_path, recovery_did_summary)

output_files["recovery_did_summary"] = recovery_did_summary_path
```

```
print(f" Saved: {recovery_did_summary_[path.name](http://path.name)}")


# Secondary summaries
if not regime_summaries_secondary.empty:
regime_summary_secondary_path = output_dir / "twindow_regime_secondary_summary.csv"
save_csv_atomic(regime_summary_secondary_path, regime_summaries_secondary)
output_files["regime_summary_secondary"] = regime_summary_secondary_path
print(f" Saved: {regime_summary_secondary_[path.name](http://path.name)}")


if not did_summaries_secondary.empty:
did_summary_secondary_path = output_dir / "twindow_branch_DID_secondary_summary.csv"
save_csv_atomic(did_summary_secondary_path, did_summaries_secondary)
output_files["did_summary_secondary"] = did_summary_secondary_path
print(f" Saved: {did_summary_secondary_[path.name](http://path.name)}")


# Text report
report = generate_text_report(
```

```
validation, regime_contrasts, branch_did, topology,
regime_summaries, did_summaries, topology_summaries,
recovery_summaries, regime_summaries_secondary, did_summaries_secondary,
recovery_did_summary
)
report_path = output_dir / "twindow_sensitivity_report.txt"
save_text_atomic(report_path, report)
output_files["report"] = report_path
print(f" Saved: {report_[path.name](http://path.name)}")


# Analysis manifest
analysis_manifest = generate_analysis_manifest(
csv_path, npz_path, manifest_path, validation, output_files,
source_path=Path(__file__),
bootstrap_seed=args.bootstrap_seed,
n_bootstrap=args.n_bootstrap,
command_line=command_line,
```

```
)

manifest_path_out = output_dir / "analysis_manifest.json"

save_json_atomic(manifest_path_out, analysis_manifest)

output_files["analysis_manifest"] = manifest_path_out

print(f" Saved: {manifest_path_[out.name](http://out.name)}")



# Print report to console

print("\n" + "=" * 80)

print("ANALYSIS REPORT")

print("=" * 80)

print(report)



print("\n" + "=" * 80)

print("ANALYSIS COMPLETE")

print("=" * 80)

if name == "main":

text
```

main()

Verdict: GO for compile and staged testing

This analysis is now close to production-ready. The main estimands, fixed/normalized horizon separation, regime contrasts, branch DID, cross-TT topology, censoring-aware recovery, normalized recovery times, recovery DID, validation gates, deterministic bootstrap, and provenance are all implemented coherently.

I do not see an obvious remaining runtime blocker in the analysis logic, ignoring the copy/paste corruption. A few small corrections should be made before the first run.
1. Correct similarity target metadata

For similarity rows, the target depends on the horizon system:

Python

similarity_target_fixed
similarity_target_normalized

but the output currently says:

Python

"target_horizon_dependence":
"horizon_independent"

Change similarity rows to:

Python

"target_horizon_dependence":
"horizon_dependent"

Density can remain:

Python

"horizon_independent"

2. Validate secondary outcomes as finite

The analysis uses all three normalized-EMA secondary outcomes, but Stage 1 validates only the primary fixed-physical outcomes.

Add:

Python

```
for outcome in (
PRIMARY_OUTCOMES
+ SECONDARY_NORMALIZED_OUTCOMES
):
...
```

All six fields should be present and finite in all 800 scalar rows.
3. Add recovery-interaction files to stale-output protection

The output list now includes:

text

```
twindow_recovery_interactions.csv
twindow_recovery_interactions_summary.csv
```

but these names were not in the earlier expected_outputs stale-file list.

Add them so --overwrite removes them and a nonoverwrite run detects them.
4. Validate recovery DID count

compute_recovery_did silently skips missing matches. Add an expected-count check.

There are:

40 seeds;
4 nonbaseline rho values;
3 noncontinue branches;
2 horizon systems;
2 metrics;
13 outcomes:
10 continuous/restricted/AUC outcomes;
3 binary indicators.

Therefore expected recovery DID rows are:
40×4×3×2×2×13=
40×4×3×2×2×13=

24{,}960.
]

Add:

Python

```
expected = (
EXPECTED_N_SEEDS
* (len(EXPECTED_RHO_VALUES) - 1)
* len(NON_CONTINUE_BRANCHES)
* 2
* 2
* (
len(recovery_outcomes)
+ len(indicator_outcomes)
)
)

if len(did_rows) != expected:
raise RuntimeError(...)
```

5. Validate every recovery summary group has 40 seeds

For each:

text

rho
branch
horizon_system
metric

require:

Python

group["seed"].nunique() == 40

Restricted-time fields should have n=40n=40. Raw conditional event times may have smaller counts because of censoring.
6. Clarify duplicated density rows

Density targets and full-future AUC are horizon-independent, so fixed and normalized density rows are duplicates unless held-duration rules differ.

The metadata now documents this adequately. Ensure no later aggregation pools the two horizon rows as independent observations.
7. Add CIs for raw sustained/held times if reported

The report conditionally looks for:

text

ci_lower_sustained
ci_lower_held

but the current summary bootstrap may not populate those fields.

Either add those bootstraps or omit those CI checks. This is secondary because restricted times are the primary censoring-aware outcomes.
8. Recovery DID helper has unused bootstrap arguments

compute_recovery_did accepts:

Python

n_bootstrap
bootstrap_seed

and constructs an RNG, but does not use them. Bootstrapping occurs later in Stage 5c.

Remove those arguments from this helper to avoid implying that it performs inference.
9. Minor output details

Use np.trapezoid when available.
Add the two recovery interaction filenames to the manifest's expected output notes.
The analysis manifest cannot hash itself at creation; that is fine.
CSV atomic writing still fsyncs only the directory, not the temporary file. This is not a scientific blocker.

Readiness rating
Area Rating
Validation 9.5/10
Fixed-physical regime estimand 9.5/10
Normalized secondary estimand 9/10
Branch DID 9.5/10
Cross-TT topology 9/10
Recovery extraction 9/10
Censoring-aware summaries 9/10
Normalized recovery scaling 9/10
Recovery DID 8.5/10
Provenance/output separation 9/10
Current executable readiness 9/10
Overall: 9/10
Test order
1. Compile all files

Bash

```
python -m py_compile \
aief_experiment_engine_v1.py \
AIEF_1_4_Twindow_sensitivity.py \
AIEF_1_4_Twindow_sensitivity_analysis.py
```

2. Run the engine import/validation

Bash

```
python aief_experiment_engine_v1.py
```

3. Run baseline equivalence only

Use the driver’s new baseline-only path. Inspect the certificate before running the full grid.
4. After the grid completes and is marked freezeable, run analysis

Bash

```
python AIEF_1_4_Twindow_sensitivity_analysis.py \
<run-directory> \
--bootstrap-seed 20260808 \
--n-bootstrap 5000
```

At this point, actual test output—not another static rewrite—should determine the next corrections.
Summary of previous conversation

This session is being continued from a previous conversation that ran out of context. The summary below covers the earlier portion of the conversation.
Goal

Continue the SSRH/AIEF computational-mathematics research program from its current proof-oriented and experiment-correction state.

Immediate goals, in order:

Run the first actual tests of the new three-file, fully regenerated TwindowTwindow sensitivity stack:

text

aief_experiment_engine_v1.py
AIEF_1_4_Twindow_sensitivity.py
AIEF_1_4_Twindow_sensitivity_analysis.py

First compile all three, then run the parameterized engine's mandatory frozen-baseline equivalence gate.
Do not launch the five-level TwindowTwindow grid until the baseline certificate reports exact reproduction of the frozen run.
After a successful baseline, run the fully regenerated five-level grid:
$\rho=\mathrm{Twindow}/T_*\in\{0.5,0.75,1,1.5,2\}, T_*=25\bar{\omega}=17.502917152858814\ldots$
$\rho=\mathrm{Twindow}/T_*\in\{0.5,0.75,1,1.5,2\}, T_*=\bar{\omega}25=17.502917152858814\ldots$
Analyze:
fixed-physical primary horizons;
normalized-EMA secondary horizons;
paired regime contrasts;
branch difference-in-differences;
cross-TT topology;
recovery scaling trec/Twindowtrec/Twindow.
Freeze the successful outputs and update the evidence lineage.
Only after TwindowTwindow sensitivity, design the separately regenerated timestep/safeguard study.

Already completed experimentally:

matched-phase correction of B_carrier_only versus full_reset;
V4 two-channel alignment closure;
exact controller-fiber theorem package;
conditional proposal/radial filter formalization.

Instructions

Be rigorous and skeptical; do not merely agree with report prose.
Always separate:
exact implementation identities;
saved finite-replay facts;
controlled counterfactual evidence;

conditional mathematical lemmas;
open asymptotic/general claims.
The seed/graph is the inferential unit.
Edges, components, perturbation directions, parameter levels, reset realizations, and amplitudes within a seed are not independent replicates.
Repeated workflow-version runs using the same seeds are not independent experiments.
Do not suggest another broad seed cohort until existing analyses and regenerated parameter studies are exhausted.
Do not call BB the unique or dominant memory carrier:
reset_B reconstructs mature topology nearly exactly when phase/history/EMA are retained;
the matched-phase correction shows BB is a useful conditional scaffold when the other mature channels are reset.
Keep separate:
threshold persistence;
topology identity;
continuous-weight similarity;
density;
storage;
reconstruction;
maintenance.
Do not conflate:
coordinate change, mirror flow, Bregman proximal point, proximal Galerkin;
topology similarity and Kuramoto order parameter;
hard admissibility and a soft regularizer;
adaptive-component stationarity and full delayed-system equilibrium.
Do not use ALPHA=0 as “instantaneous resonance”; with

Python

```
z_inst = np.exp(1j * dphi)
R = np.abs(z_inst)
```

every instantaneous off-diagonal magnitude is 1.
The implemented admissibility map is not known to be an orthogonal Euclidean projection.
Mirror AIEF is a new model:
$w˙=H−1F/β$,
$w˙=H−1F/β$, not a coordinate rewrite.
Bregman divergence is not a global geodesic distance.
Entropy geometry is natural, not proved canonical.
Do not silently change:

text

N
dt
tau
T_window
omega
eta
mu
gamma
capacity
B_c
delay indexing
EMA update
positivity map
symmetrization
global gate
minimum.outer endpoint scaling
fixed +1e-12 safeguard
RNG streams
seed pairing
split-state protocol

A changed T_window, dt, or safeguard requires a separately versioned, fully regenerated trajectory.
Never reuse a split state under incompatible dynamics.
For coding:
preserve reproducibility;
preserve source hashes/manifests;
use atomic outputs;
preserve paired seeds;
record tolerances and caveats;
prefer new scripts over mutation of frozen production drivers;
compile/help/self-test/smoke-test before a full run.
The user prefers iterative technical dialogue, often saying “think about this,” “ponder,” and asking for code critique. Be precise, candid, conservative, and direct.
When the user asks to review pasted code “overlooking copy paste errors,” ignore HTML/Markdown corruption such as:

text

from **future**

if **name**
->
<
handle.read

and focus on actual logic/science.
Do not create a file when the user explicitly asks for critique “in the chat.”
Quasiperiodicity/KAM/Ott–Antonsen/full-attractor geometry remain downstream.

Discoveries
Core production model

Python

N = 12
T_sim = 800
dt = 0.01
tau = 0.5

omega = np.array([
1.00, 1.02, 1.50, 2.00, 2.01,
1.01, 1.03, 1.51, 2.02, 2.00,
1.00, 1.04
])

Parameters:

text

a=1.0
q=1.5
delta=0.05
eta=0.04
gamma=0.015
mu=0.03
B_c=0.12
B_init_scale=0.0005
D_B_star=0.30
T_window=25/mean(omega)=17.502917152858814...

alpha=exp(-dt/T_window)=0.9994288298464774...

Capacity:
c=a−δq=0.633333…
c=qa−δ=0.633333…

Phase:
θ˙i=
θ˙i=

\omega_i+
\sum_jB_{ij}\sin(\theta_j(t-\tau)-\theta_i(t)).
]

EMA:
Cijn+1=
Cijn+1=

\alpha C_{ij}^n+
(1-\alpha)e^{i\Delta\phi_{ij}},
\qquad
R_{ij}=|C_{ij}|.
]

Weights:
B˙=(η+μ)R−μ−γB.
B˙=(η+μ)R−μ−γB.

Implemented proposal:

Python

```
proposal = B + dt * F
proposal = np.maximum(proposal, 0.0)
proposal = 0.5 * (proposal + proposal.T)
np.fill_diagonal(proposal, 0.0)
```

Global scaling when any proposal row violates:

Python

```
scale = np.minimum(
1.0,
capacities / (row_sum + 1e-12),
)
projected = proposal * np.minimum.outer(
scale,
scale,
)
```

The map is not an orthogonal Euclidean projection.
AIEF-1.3

Frozen production script:

text

AIEF_1_3_falsification_updated.py

Independent 50-seed cohorts:

text

runs/c3da1ac707bf/
runs/73dca37cf93f/

Pooled baseline approximately:

text

D_B ~0.306
Corr_physical ~0.995
Effective_edges ~14.6
Total edge weight ~3.64

Key ablations:

text

shuffle_R:
D_B=0, P_mean=0, P_degenerate=1 every seed

no_mu:
D_B~0.035, total weight~3.78, effective edges~34.4

no_gap:
D_B~0.727, total weight~127.9, G_min~-32.8

Closure passed. Baseline P_future was exactly binary for every unique edge and seed.
Original AIEF-1.4 memory ablation

Production:

text

AIEF_1_4_memory_ablation_updated.py

Frozen run:

text

runs/5a9f093a2fe0/

Branches:

text

continue
reset_ema
reset_B
reset_B_ema

B_carrier_only
full_reset
future_shuffle

Historical caveat:
B_carrier_only and full_reset used different branch-specific randomized phase/history draws.

Late summaries:

text

branch P_late ratio Sim_late Retention Emergence
continue 1.000 0.999 1.000 0.000
reset_ema 0.990 0.993 0.990 0.000
reset_B 1.000 0.999 1.000 0.000
reset_B_ema 1.080 0.944 0.987 0.038
B_carrier_only 0.945 0.948 0.908 0.015
full_reset 1.035 0.884 0.920 0.046
future_shuffle 0.000 0.475 0.000 0.000

Sustained 90%90% recovery:

text

continue 40/40
reset_ema 39/40
reset_B 40/40
reset_B_ema 38/40
B_carrier_only 34/40
full_reset 25/40
future_shuffle 0/40

Conclusions:

no single state variable is the unique register;
reset_B reconstructs nearly exactly;
future edge-specific identity is required for maintenance.

Matched-phase correction — completed

The successful corrected experiment used two matched reset blocks per seed:

original B_carrier_only reset realization;
original full_reset reset realization.

Within each block, phase/history/zero EMA were identical and only initial BB differed.

Validation:

text

Total seeds: 40
Seeds succeeded: 40
Seeds failed: 0
Initial matching passed: 80/80
Anchor validation passed: 120/120
Censoring horizon: 319.5100
Run scope: full_cohort
Freezeable: True

Primary seed-level two-block continuous contrasts, preserved BB minus zero BB:

text

P_mean_late:
mean = -0.0176095
95% bootstrap CI [-0.0267873, -0.00882897]

Weighted_similarity_late:
mean = 0.0641537
CI [0.0405751, 0.0889752]

Weighted_similarity_final:
mean = 0.0638799
CI [0.0402338, 0.0887457]

Retention_selected_late:
mean = 0.00728708
CI [-0.0168658, 0.0316646]

Emergence_unselected_late:
mean = -0.0270618
CI [-0.03626, -0.0178676]

D_B_final:
mean = -0.0179924
CI [-0.0274621, -0.00890152]

Total_edge_weight:
mean = -0.0829795
CI [-0.13534, -0.0293594]

Effective_edges:
mean = -0.826419
CI [-1.45916, -0.206972]

Nonzero_edge_fraction:
mean = -0.0179924
CI [-0.0272727, -0.00946496]

Minimum_similarity:
mean = 0.100776
CI [0.080042, 0.119867]

AUC_similarity_deficit_vs_continue:
mean = -31.7407
CI [-38.8781, -25.0103]

Mean_similarity_deficit_vs_continue:
mean = -0.0993451
CI [-0.121684, -0.0782796]

restricted_time_to_similarity_90:
mean = -154.412

CI [-188.927, -121.143]

restricted_time_to_density_90:
mean = -86.8438
CI [-117.125, -61.6853]

Binary two-block average effects:

text

sustained_similarity_90_recovered:
mean = 0.2375
CI [0.1125, 0.375]
17 positive, 20 zero, 3 negative

sustained_density_90_recovered:
mean = -0.075
CI [-0.15, 0]
2 positive, 30 zero, 8 negative

held_similarity_90_for_1Twindow_recovered:
mean = 0.25
CI [0.125, 0.375312]
17 positive, 21 zero, 2 negative

Block-specific sustained similarity:

text

carrier reset:
B preserved 0.850
B zero 0.600
paired difference 0.250

full-reset reset:
B preserved 0.850
B zero 0.625
paired difference 0.225

Reset-realization interactions were small; all principal continuous interaction intervals included zero. Example:

text

Weighted_similarity_late:
mean interaction=0.00905696
CI [-0.0221536, 0.0432124]

Interpretation:

mature BB improves recovery of prior continuous-weight organization when phase/history and EMA are reset;
it suppresses emergence and yields somewhat sparser/lower-weight networks;
it does not clearly increase selected-edge retention;
it is a conditional scaffold, not a unique memory register;
original unmatched result was not mainly driven by a lucky reset realization.

Exceptional K3 and operator geometry

Exceptional seed:

text

4619122171158768524

Nodes/edges:

text

nodes [2,7,11]
25=(2,7)
29=(2,11)
59=(7,11)

Fixed saved winners:

text

25 -> node7
29 -> node2
59 -> node7

No effective positive-edge winner change over 32,001 states.

Controller rows:
J={2,7},CJ=(110101),
J={2,7},CJ=(111001),
kerCJ=
kerCJ=

\operatorname{span}{(1,-1,-1)}.
]

Analytic carrier:
v=13(1,−1,−1).
v=3
1(1,−1,−1).

Analytic fixed left covector:
u∗=32(1,0,−1),u∗Tv=1.
u∗=23
(1,0,−1),u∗Tv=1.

Pointwise left covector after right closure:
u(r)=
u(r)=

\frac{\sqrt3}{1+r}(1,0,-r),
\qquad
r=\beta_{25}/\beta_{59}.
]

Exact selected block:

K=c0(s7−β250−β25−β29s2−β290−β590s7−β59),c0=1−γdt.
K=c0
s7−β25−β29−β590s2−β290−β250s7−β59
,c0=1−γdt.

Right closure:
Kv=λv ⟺ s2=s7.
Kv=λv⟹s2=s7.

Fixed-left closure:
u∗TK=λu∗T ⟺ β25=β59.
u∗TK=λu∗T⟹β25=β59.

Proposal differences:
s2−s7=
s2−s7=

c\frac{x_{59}-x_{29}}
{(q_2+\varepsilon)(q_7+\varepsilon)},
]
β25−β59=
β25−β59=

\frac{s_7}{q_7+\varepsilon}
(x_{25}-x_{59}).
]

Strict branch cone:
dL=x25−x59>0,dR=x59−x29>0.
dL=x25−x59>0,dR=x59−x29>0.

At exact equality, all scales tie and:
CK3=
CK3=

\begin{pmatrix}
1&1&0\
1&0&1\
0&1&1

\end{pmatrix},
\qquad
\ker C_{K_3}={0}.
]